



Faculty of Computer Science, Electrical Engineering and Mathematics
Department of Computer Science
Research Group Secure Software Engineering

Assisting GDPR Compliance through Static Analysis of Android Apps

Mugdha Khedkar

Dissertation

Submitted in partial fulfillment of the requirements for the degree of
Doktor der Naturwissenschaften (Dr. rer. nat.)

Advisor

Prof. Dr. Eric Bodden

Paderborn, May 22, 2026

Mugdha Khedkar

Assisting GDPR Compliance through Static Analysis of Android Apps

Dissertation, May 22, 2026

Advisor: Prof. Dr. Eric Bodden

Paderborn University

Research Group Secure Software Engineering

Department of Computer Science

Faculty of Computer Science, Electrical Engineering and Mathematics

Warburger Straße 100

33098 Paderborn

*Two roads diverged in a wood, and I—
I took the one less traveled by,
And that has made all the difference...*

— Robert Frost

Abstract

Android applications collecting data from users must comply with legal frameworks to ensure data protection. This requirement has become even more important since the implementation of the General Data Protection Regulation (GDPR) by the European Union in 2018. In practice, GDPR compliance involves conducting effective privacy assessments, requiring close collaboration between stakeholders with diverse expertise-including developers, privacy experts, legal experts, and auditors.

This thesis introduces a multi-layered definition of privacy-related data, grounded in GDPR's notion of personal data, and presents **Privacy-Relevant Input Classification Engine (PRICE)**, a command-line tool for statically labeling privacy-related data collected by Android apps. Using PRICE, we examine data collection patterns across Android apps and assess how accurately these apps report their data collection practices. Our analysis reveals that apps most frequently collect data capable of partially identifying users, highlighting the need for greater consistency in privacy-aware development and reporting. Complementing this analysis, we investigate developers' experiences with Google Play Store's Data Safety Section (DSS) through a survey and analysis of online developer discussions. This study identifies key challenges developers face in completing the DSS, including difficulties in identifying privacy-related data and limited understanding of the form.

Building on these findings, the thesis explores how static program analysis can be used to automate and support privacy assessments. We introduce **Assessor View**, a static analysis-based web tool designed to assist multiple stakeholders involved in privacy evaluations. Assessor View provides a unified view of privacy-related data flows and supports collaboration across technical and non-technical roles. Qualitative evaluations with technical and legal participants indicate that the tool's warnings and guidance are valuable to Data Protection Officers and privacy experts, representing a significant step toward improving communication between legal and technical experts and streamlining privacy assessments.

Finally, we extend both PRICE and Assessor View to more directly support stakeholder needs by enabling accurate, source code-driven report generation. These extensions aim to reduce manual effort, improve consistency between implemented behavior and reported disclosures, and support reliable and transparent privacy assessments.

Collectively, these contributions advance the state of the art in privacy-aware software engineering and provide a foundation for more systematic, automated, and legally compliant privacy assessments of Android applications.

Zusammenfassung

Android-Anwendungen, die Nutzerdaten erfassen, müssen die gesetzlichen Bestimmungen zum Datenschutz einhalten. Diese Anforderung hat seit dem Inkrafttreten der Datenschutz-Grundverordnung (DSGVO) durch die Europäische Union im Jahr 2018 noch an Bedeutung gewonnen. In der Praxis erfordert die Einhaltung der DSGVO die Durchführung effektiver Datenschutzprüfungen und damit eine enge Zusammenarbeit zwischen Beteiligten mit unterschiedlichen Fachkenntnissen - darunter Entwickler, Datenschutzexperten, Rechtsexperten und Auditoren.

Diese Arbeit führt eine mehrschichtige Definition datenschutzrelevanter Daten ein, die auf dem Begriff der personenbezogenen Daten der DSGVO basiert, und stellt die **Privacy-Relevant Input Classification Engine (PRICE)** vor, ein Kommandozeilen-Tool zur statischen Kennzeichnung datenschutzrelevanter Daten, die von Android-Apps erfasst werden. Mithilfe von PRICE untersuchen wir Datenerfassungsmuster in Android-Apps und bewerten, wie genau diese Apps ihre Datenerfassungspraktiken dokumentieren. Unsere Analyse zeigt, dass Apps am häufigsten Daten erfassen, die Nutzer teilweise identifizieren können, was die Notwendigkeit größerer Konsistenz bei datenschutzbewusster Entwicklung und Berichterstattung unterstreicht. Ergänzend zu dieser Analyse untersuchen wir die Erfahrungen von Entwicklern mit dem Bereich Datensicherheit (DSS) des Google Play Stores anhand einer Umfrage und der Analyse von Online-Entwicklerdiskussionen. Diese Studie identifiziert zentrale Herausforderungen, mit denen Entwickler beim Ausfüllen des DSS konfrontiert sind, darunter Schwierigkeiten bei der Identifizierung datenschutzrelevanter Daten und ein begrenztes Verständnis des Formulars.

Aufbauend auf diesen Erkenntnissen untersucht die Arbeit, wie statische Programmanalyse zur Automatisierung und Unterstützung von Datenschutzbewertungen eingesetzt werden kann. Wir stellen **Assessor View** vor, ein webbasiertes Tool auf Basis statischer Analyse, das verschiedene an Datenschutzbewertungen beteiligte Akteure unterstützt. Assessor View bietet eine einheitliche Sicht auf datenschutzrelevante Datenflüsse und fördert die Zusammenarbeit zwischen technischen und nicht-technischen Rollen. Qualitative Evaluierungen mit technischen und juristischen Teilnehmern zeigen, dass die Warnungen und Hinweise des Tools für Datenschutzbeauftragte und Datenschutzexperten wertvoll sind und einen wichtigen Schritt zur Verbesserung der Kommunikation zwischen juristischen und technischen Experten sowie zur Optimierung von Datenschutzbewertungen darstellen.

Schließlich erweitern wir sowohl PRICE als auch Assessor View, um die Bedürfnisse der Stakeholder direkter zu unterstützen, indem wir eine präzise, quellcodebasierte Berichterstellung ermöglichen. Diese Erweiterungen zielen darauf ab, den manuellen

Aufwand zu reduzieren, die Konsistenz zwischen implementiertem Verhalten und gemeldeten Offenlegungen zu verbessern und zuverlässige und transparente Datenschutzbewertungen zu unterstützen.

Zusammengenommen erweitern diese Beiträge den Stand der Technik im Bereich datenschutzbewusster Softwareentwicklung und schaffen eine Grundlage für systematischere, automatisierte und rechtskonforme Datenschutzbewertungen von Android-Anwendungen.

Acknowledgments

Although this thesis bears my name, it has been shaped by many people who accompanied me on the journey toward this PhD.

First, I would like to thank my advisor, Eric Bodden. His constant understanding and support throughout this tumultuous journey has made all the difference in my career. Thank you, Eric, for never giving up on my ideas and for giving me the freedom and confidence to explore unfamiliar and unconventional directions. Your trust has meant a great deal to me, and I look forward to continuing our mentor-mentee relationship in the years ahead.

I am also deeply grateful to my colleague, Michael Schlichtig, who has been an excellent collaborator. Some of my best memories from this PhD are of the time we spent developing new ideas together. His encouragement, especially in the early stages of my PhD, gave me the confidence to stay on this path. Thank you, Michael, for being not only a collaborator, but also a friend.

I am grateful to my collaborator and co-author, Mohamed Soliman. From him, I learned the value of attention to detail and the craft of writing clear and well-structured research papers. Thank you, Mohamed, for your guidance and mentorship.

This work would not have been possible without the contributions of many researchers whose work came before mine. In particular, the main contributions of this thesis build on the tools Jicer and SootFX. I am grateful to Felix Pauck for open-sourcing Jicer, and to Kadiray Karakaya for open-sourcing SootFX.

An empirical thesis such as this one would be incomplete without the people who participated in its studies and surveys. I would like to sincerely thank the Android developers, students, privacy experts, and Data Protection Officers who shared their time, perspectives, and feedback. The privacy community welcomed me warmly, and I am truly grateful for that.

I had the good fortune to work with many students over the years. I am grateful to Saurabh Raut, Amit Kumar, Ambuj Mondal, Amit Kumar, Shashi Prakash, Santhosh Mohan, Nihad Atikishiyev, Hardik Shetty, and Akshaya Nair for their dedication and hard work.

Many of the contributions presented in this thesis were previously published at international conferences and in journals. I thank the anonymous reviewers whose thoughtful comments helped improve the work presented here. I would also like to thank Mohamed Soliman, Awais Rashid, Ben Hermann, and Arnab Sharma for serving on my thesis committee.

I am especially grateful to Sammy for his technical support and for always responding so quickly, even to the smallest or silliest of requests. Many thanks to Nicole for all her support, especially in helping with the many business trips I applied for. I will always remember their kindness and help.

I would like to thank Tapti Palit for the fruitful collaboration over the past two years.

My colleagues and friends from the office kept me sane despite the hardships of my journey. Sriteja was my first friend in Paderborn, and I am so happy that we have remained such close friends over the years. Whether it was encouraging each other after every rejection or turning to one another for advice, thank you, Sriteja, for always having my back. Thanks Ashwin and Kadiray for being my seniors, mentors, and first office mates. We truly were the first “expats of HNI”, and I am grateful to have been part of that community. I certainly would not have made it through this time without Jonas’s coffee breaks. Thank you, Jonas, for being the glue that holds this group together. Thank you, Stefan, for your friendship and support, and also for introducing me to Michelle. Thanks to Martin and Faiza for being my neighbors. I will never forget Martin’s help when I first moved to Paderborn. Thank you Martin, you may never fully know how much your friendship and guidance have meant to me. Thanks Markus and Marcu for always having something fun to say, and thank you, Palani, Rose, Vaibhav, Santhosh, Leen, Ranjith and Fabian, for being such a wonderful part of the new generation. Because of all of you, I am certain that our group will continue to grow and flourish. A PhD can be a lonely journey, and it is important to remind each other that we are in this together.

My personal village has stood by me, again and again, through every season of this journey. Words feel insufficient when I try to thank my husband, Rahul. This achievement is as much his as it is mine. Pursuing our PhDs on opposite sides of the world while sustaining a long-distance marriage was no small undertaking, and I have not taken a single moment with him for granted. I know I would have stepped away from this path years ago if not for his steady voice reminding me, time and again, “*You’re doing so well. Keep it up.*” Thank you, Rahul, for being a partner in the truest sense of the word. The world would be a kinder and happier place if every partner carried even a fraction of your understanding.

My twin sister, Sneha, has been my best friend since the very first day of my life. She has stood firmly beside me through every decision and every rejection I faced during my PhD. I truly do not know where I would be without her sisterhood, and I cannot imagine a life in which she is not on the other end of a phone call. Thank you, Sneha, for supporting me unconditionally through every phase of my PhD and for always being my personal cheerleader.

There are many privileges in belonging to an academic family, but among the greatest is the patience and understanding my parents have shown me throughout this journey. Their faith in me has never wavered. Thank you, Aai and Baba, for believing in me, in my decisions, and in my abilities, and for standing by me with unwavering support.

I am deeply grateful to my friends, Samiksha and Neha, for being steady voices of encouragement across time zones. Finally, I would like to thank my entire family. None of this would have been possible without your understanding, support, and love.

Publications

This dissertation is an original work. However, parts of it have already been published in conference and journal papers, of which the author of this thesis is also the lead author. In particular, this includes the following work:

1. **Mugdha Khedkar**. “Static Analysis for Android GDPR Compliance Assurance”. In: *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 2023, pp. 197–199. DOI: 10.1109/ICSE-Companion58688.2023.00054
Parts of Chapter 1 are taken from this *doctoral symposium* paper.
2. **Mugdha Khedkar** and Eric Bodden. “Toward an Android Static Analysis Approach for Data Protection”. In: *Proceedings of the IEEE/ACM 11th International Conference on Mobile Software Engineering and Systems*. MOBILESoft ’24. Lisbon, Portugal: Association for Computing Machinery, 2024, pp. 65–68. DOI: 10.1145/3647632.3651389
Parts of Chapter 1 are taken from this *idea* paper.
3. **Mugdha Khedkar**, Ambuj Kumar Mondal, and Eric Bodden. “A Study of Privacy-Related Data Collected by Android Apps”. In: *Automated Software Engineering* 33.2 (2026), p. 45. DOI: 10.1007/s10515-025-00589-3
Parts of Chapter 3 and Chapter 4 are taken from this *journal* paper.
4. **Mugdha Khedkar**, Ambuj Kumar Mondal, and Eric Bodden. “Do Android App Developers Accurately Report Collection of Privacy-Related Data?” In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops*. ASEW ’24. Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 176–186. DOI: 10.1145/3691621.3694949
Parts of Chapter 4 are taken from this *workshop* paper.
5. **Mugdha Khedkar**, Michael Schlichtig, Mohamed Soliman, and Eric Bodden. “Challenges in Android Data Disclosure: An Empirical Study.” In: *2026 IEEE/ACM 13th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. 2026. DOI: 10.1145/3795077.3795118
Parts of Chapter 4 are taken from this *conference* paper.

6. **Mugdha Khedkar**, Michael Schlichtig, and Eric Bodden. “Advancing Android Privacy Assessments with Automation”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops*. ASEW ’24. Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 218–222. DOI: 10.1145/3691621.3694953
Parts of Chapter 5 are taken from this *idea* paper.
7. **Mugdha Khedkar**, Michael Schlichtig, Nihad Atakishiyev, and Eric Bodden. “Between Law and Code: Challenges and Opportunities for Automating Privacy Assessments”. In: *Automated Software Engineering* 33.56 (2026), p. 45. DOI: 10.1007/s10515-026-00601-4
Parts of Chapter 5 are taken from this *journal* paper.
8. **Mugdha Khedkar**, Michael Schlichtig, and Eric Bodden. “Source Code-Driven GDPR Documentation: Supporting RoPA with Assessor View”. In: *2026 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 2026, pp. 935–939. DOI: 10.1109/SANER67736.2026.00109
Parts of Chapter 6 are taken from this *tool demonstration* paper.

Contents

1. Introduction	1
1.1. Contributions	2
1.2. Overview	5
2. Background	7
2.1. General Data Protection Regulation (GDPR)	7
2.2. Data Collection in Android apps	8
2.3. Program Slicing	10
2.4. Jicer	11
2.5. Data Privacy Vocabulary (DPV)	13
2.6. Privacy-Related Documentation	16
2.6.1. Data Safety Section (DSS)	16
2.6.2. Record of Processing Activities (RoPA)	18
3. A Study of Android Data Collection	21
3.1. Privacy-Related Data	21
3.2. Identifier Datasets	23
3.3. Privacy-Relevant Input Classification Engine (PRICE)	26
3.3.1. Sanity Checks	28
3.3.2. Limitations	29
3.4. Analyzing Data Collection Practices using PRICE	31
3.4.1. RQ1. Data Collection by Random Apps	31
3.4.2. RQ2. UI-Based Collection by Domain Apps	34
3.4.3. Discussion	37
3.4.4. Limitations and Threats to Validity	39
3.5. Related Work	39
3.6. Summary	40
4. Understanding Privacy-Aware Reporting Practices	41
4.1. Analyzing Data Disclosures using PRICE	41
4.1.1. RQ1. Analysis of Data Safety Sections	42
4.1.2. RQ2. Comparison between Actual and Reported Data Collection	43
4.1.3. Discussion	45
4.1.4. Limitations and Threats to Validity	48

4.2. Challenges Faced by Developers: An Empirical Study	48
4.2.1. Study Methodology	48
4.2.2. Study Results	53
4.2.3. Discussion	62
4.2.4. Limitations and Threats to Validity	64
4.3. Related Work	65
4.4. Summary	66
5. Exploring Automation of Privacy Assessments	69
5.1. Assessor View	69
5.1.1. An Example	75
5.1.2. Implementation	78
5.1.3. Early Experiments	81
5.1.4. Limitations	84
5.2. Stakeholder Evaluations using Assessor View	85
5.2.1. Evaluation with Developers	86
5.2.2. Evaluation with Privacy Professionals	93
5.3. Related Work	113
5.4. Summary	115
6. Tool Support for Privacy-Aware Reporting	117
6.1. AutoPRICE for DSS Completion	117
6.1.1. Early Experiment	120
6.1.2. Feature Extensions	126
6.1.3. Limitations and Threats to Validity	130
6.1.4. Related Work	130
6.2. Extending Assessor View for RoPA Generation	131
6.2.1. Feature Extensions	132
6.2.2. Evaluation: Semi-Structured Interviews	134
6.2.3. Limitations and Threats to Validity	138
6.2.4. Related Work	139
6.3. Summary	139
7. Conclusion and Future Directions	141
8. Replication Package	145
8.1. Tool 1: Privacy-Relevant Input Classification Engine (PRICE)	145
8.2. Empirical Study: Challenges Faced by Developers	145
8.3. Tool 2: Assessor View	146
8.4. Tool 3: AutoPRICE	146

Bibliography	149
List of Acronyms	165
List of Figures	167
List of Tables	171
A. Appendix: Developer Survey Questionnaire	173
B. Appendix: Study Materials for Assessor View Evaluation (Developers)	179
C. Appendix: Study Materials for Assessor View Evaluation (Privacy Experts)	187
C.1. Interview Briefing	187
C.2. Interview Questionnaire	187
C.3. Codebook	192
D. Appendix: Study Questionnaire for AutoPRICE Evaluation	197
E. Appendix: Study Materials for Assessor View RoPA Generation	203
E.1. Interview Briefing	203
E.2. Interview Questionnaire	203
E.3. Codebook	204

Introduction

We use many Android applications in our daily life. As of April 2026, there are approximately 1.79 million apps available on the Google Play Store [26e]. Among these apps, almost all collect data from their users, and some apps belong to particularly privacy-sensitive categories.

Since 2018, all Android apps that collect data from users residing in the European Union must comply with the General Data Protection Regulation (GDPR) [18f]. The GDPR imposes several obligations on the collection, storage, and processing of personal data. Article §25 [18h] of the GDPR highlights *data protection by default and design*, making data protection an integral part of the application development process. In addition to technical and organizational measures, the GDPR introduces strong transparency obligations: under Article §13 [18g], data controllers (e.g., software providers or organizations) must inform data subjects about data collection and processing through disclosures such as privacy policies and, in the Android ecosystem, Google Play Store's Data Safety Section (DSS) [22c]. The growing demand for privacy by design [Cav09], both by end users and by GDPR necessitates that app developers use state-of-the-art technical measures to protect their users' privacy. The legal description of GDPR is very complex and lengthy and hence it can be difficult for app developers to understand. Since they lack legal expertise, app developers may find it difficult to understand how to protect users' data, or even *which* data they should protect [Hor+24].

Although developers are responsible for protecting users' data, demonstrating compliance with the GDPR typically requires assessments that involve multiple stakeholders with diverse expertise. Under Article §35 [18k] of the GDPR, the data controller is responsible for ensuring that a Data Protection Impact Assessment (DPIA) [18d] is carried out whenever processing operations may pose high risks to individuals' rights and freedoms. In particular, Article §30 [18i] requires organizations to maintain a Record of Processing Activities (RoPA) [18q] describing how personal data is collected, stored, and processed. RoPAs often serve as a basis for DPIAs. The task of conducting the DPIA and completing the RoPA typically falls on privacy professionals such as Data Protection Officers (DPOs) or legal experts, who must evaluate data protection risks in collaboration with technical teams. Hence, assuring GDPR compliance is an inter-disciplinary process.

Over the years, static program analysis has emerged as a promising technical approach for supporting security by design. It examines an application’s source or compiled code without executing it, enabling systematic reasoning about data flows, Application Programming Interface (API) usage, and potential security or privacy risks early in the development lifecycle. Compared to manual inspections or documentation based assessments, static analysis can scale to large numbers of applications and produce objective and repeatable evidence about the types of data an app may collect, process, or transmit [Arz+14; Li+15; Gor+19]. These characteristics make static analysis particularly well suited for privacy assessments, where identifying accessed personal data and understanding their intended purposes are essential for meeting GDPR requirements such as data minimization, transparency, and accountability.

Despite its potential, existing static analysis tools are rarely designed with non-technical stakeholders in mind [NSB22]. Most tools target security specialists or program analysis researchers and present results at a low level of technical abstraction, limiting their accessibility to developers, DPOs, legal professionals, and other actors involved in DPIAs. Consequently, a gap remains between the technical capabilities of static analysis and its effective use in real-world privacy assessments. Bridging this gap requires approaches that (i) systematically classify data collected by apps as privacy-related according to the GDPR, (ii) present analysis results in forms that are understandable and actionable for technical and non-technical stakeholders alike, and (iii) support the documentation and reporting activities mandated by the GDPR. Addressing these challenges is a central focus of this thesis.

1.1 Contributions

This thesis makes the following contributions (cf. Figure 1.1):

Contribution 1: Understanding Privacy-Related Data Collection in Android Apps. A key technical gap, especially in the post-GDPR setting, is the absence of an operational, analysis-ready definition of what constitutes “sensitive” or “privacy-related data” in Android apps. This thesis addresses that gap by introducing a *multi-layered definition* of privacy-related data grounded in GDPR’s concept of personal data [18]. The definition maps concrete identifiers (e.g., Internet Protocol (IP) address, email address) to (i) a GDPR-based risk rating and (ii) a data category aligned with Google Play Store’s Data Safety Section (DSS), enabling a direct connection between low-level program behavior and the categories developers must disclose. Based on this definition, we construct a dataset and develop *Privacy-Relevant Input*

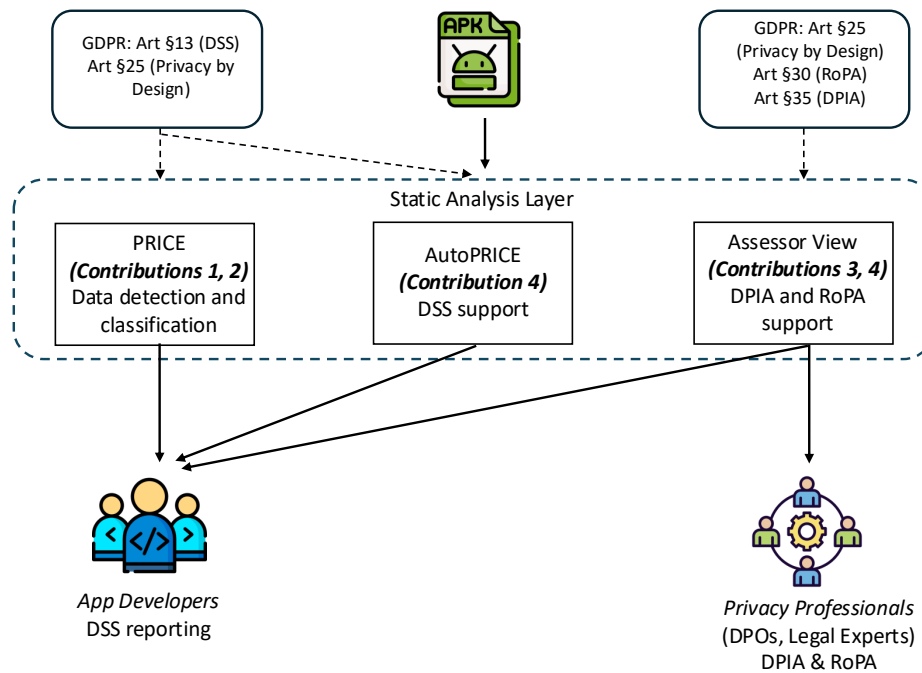


Figure 1.1.: Overview of stakeholders and static analysis-based support for GDPR compliance in Android applications. Static analysis bridges technical artifacts and interdisciplinary stakeholders by supporting DSS reporting, DPIAs, and RoPA generation.

Classification Engine, a command-line static analysis tool that detects and labels privacy-related data collected by Android applications through source code, user interfaces, and permissions.

Using PRICE, we empirically analyze data collection practices of Android applications. A large-scale quantitative study of 7,500 *Android apps* shows that apps primarily collect personal and financial information via user interfaces, while device, location, and browsing data are predominantly accessed through system APIs. Our results further reveal that partially identifying personal data, such as age or postal codes, constitute the most frequently collected data types. Although system APIs consistently collect substantial amounts of privacy-related data, user interfaces exhibit more diverse and application-specific collection patterns.

To further investigate these patterns, we perform a qualitative analysis of the 100 most popular apps in each of the following domains: *Messaging and Social Media*, *Banking and Finance*, *News and Entertainment*, *Sports and Games*, *Technology and Education*, *E-Commerce and Shopping*, and *Health and Fitness*. This analysis shows that Messaging and Social Media apps frequently collect personal and authentication-

related data directly from users, whereas Sports and Games apps collect the least amount of personal data through their user interfaces.

Contribution 2: Examining Developers’ Privacy Reporting Practices. The second contribution of this thesis examines how Android developers *report* data collection practices in Google Play Store’s Data Safety Section (DSS) [22c]. Using PRICE, we analyze which data categories are consistently disclosed and identify discrepancies between statically detected data collection and developer-reported DSS information.

Through detailed case studies of two *Messaging and Social Media* apps (Signal and Instagram), we illustrate instances of *under-reporting* and *over-reporting*, and identify inaccurately disclosed data categories. To further understand the causes of these discrepancies, we conduct an *empirical study* exploring how developers complete the DSS form and the challenges they encounter.

Our findings show that developers largely rely on manual methods to classify privacy-related data into DSS categories, or in some cases omit classification altogether. They frequently consult online resources when completing the form. While developers generally express confidence in identifying the data their applications collect, this confidence declines significantly when translating technical knowledge into DSS-compliant disclosures. We identify recurring challenges, including difficulties in identifying privacy-related data, limited understanding of the DSS form, and concerns about app rejection due to mismatches with Google’s privacy requirements.

Contribution 3: Exploring Static Program Analysis for Automating Privacy Assessments. Ensuring GDPR compliance through assessments requires collaboration among groups with a diverse expertise. However, a clear research gap remains in understanding the challenges faced by Data Protection Officers (DPOs), privacy experts, and legal experts—those responsible for conducting Data Protection Impact Assessments (DPIAs) [18d] and ensuring organizational compliance. Understanding their perspective is essential, particularly to automate parts of the assessment process.

To address this gap, we design and implement *Assessor View*, a static analysis-based web tool that visualizes source code components using Data Privacy Vocabulary (DPV) [Pan+24] and provides multiple visualization modes tailored to both technical and legal stakeholders. We employ Assessor View as a research probe to explore how automation can support privacy assessments.

We conduct qualitative studies involving *two stakeholder groups*: (i) developers, and (ii) privacy professionals (including DPOs and legal experts). Our results identify key challenges in current privacy assessment practices, such as knowledge gaps, ineffective communication between legal and technical teams, the lack of source-code-based privacy analysis tools, and the late involvement of privacy assessors in the development lifecycle. Hands-on sessions using Assessor View demonstrate that its GDPR-related warnings and guidance are valuable to privacy professionals, and that its design effectively supports their assessment tasks. Overall, our findings suggest that Assessor View represents a promising first step toward improving communication and partial automation in privacy assessment processes.

Contribution 4: Providing Automated Report-Generation Support for Stakeholders. While PRICE (*Contribution 1*) and Assessor View (*Contribution 3*) provide valuable analytical capabilities, they do not directly support the automated generation of legally required documentation from software artifacts. The final contribution of this thesis addresses this limitation by *extending both tools* to provide direct reporting support for key stakeholders.

We introduce *AutoPRICE*, a static analysis-based web tool that uses PRICE at the back-end to assist Android app developers in completing the DSS form. In addition, we extend *Assessor View* to assist privacy professionals (including DPOs and legal experts) by automatically generating Record of Processing Activities (RoPA) [18q], a legally mandated inventory describing how personal data is processed. *Semi-structured interviews* with 6 DPOs indicate that Assessor View’s RoPA feature provides a good overview of the processed data and enhances interdisciplinary communication between stakeholders. These extensions enable more accurate, efficient, and practical reporting, directly supporting multiple stakeholders in fulfilling GDPR documentation requirements.

1.2 Overview

The remainder of this thesis is organized around the four core contributions discussed in Section 1.1.

Chapter 2 provides the necessary background for this work, introducing key concepts from data privacy, the GDPR, and static program analysis.

Chapter 3 introduces our definition of privacy-related data and describes the construction of a dataset grounded in this definition. It then introduces Privacy-Relevant Input Classification Engine (PRICE), a command-line tool that detects and labels the data collected by Android apps. Using PRICE, the chapter empirically analyzes data collection practices of Android applications (*Contribution 1*).

Chapter 4 uses PRICE to examine whether Android developers accurately disclose collected data in Google Play Store's Data Safety Section (DSS). The chapter further presents an empirical study aimed at understanding how developers complete the DSS form and identifying the challenges they encounter during this process, thereby providing insights into developers' reporting practices (*Contribution 2*).

Chapter 5 broadens the scope of the thesis by exploring how static program analysis can be used to automate privacy assessments. It introduces Assessor View, a static analysis-based web tool designed to support multiple stakeholders involved in privacy assessments, and presents qualitative evaluations of its potential for automating privacy assessments and audits (*Contribution 3*).

Chapter 6 extends PRICE and Assessor View to provide direct tool support for relevant stakeholders, enabling more practical and accurate reporting (*Contribution 4*). Chapter 7 concludes the thesis.

Background

This chapter presents the key concepts on which this thesis is based. We first introduce the European Union’s General Data Protection Regulation (GDPR) in Section 2.1. Next, we discuss the data collection techniques used by Android apps (cf. Section 2.2), followed by an overview of static program slicing (cf. Section 2.3) and Jicer, the Android slicer used in this thesis (cf. Section 2.4). We then introduce the Data Privacy Vocabulary (DPV) in Section 2.5. Finally, we discuss the privacy-related documentation required for Android apps to support GDPR compliance in Section 2.6.

2.1 General Data Protection Regulation (GDPR)

On 25 May 2018, the European Union enforced the General Data Protection Regulation (GDPR) which has been called the *toughest privacy and security law in the world* [18r]. The GDPR introduces a strict definition of *personal data* and imposes obligations on the collection, storage, and processing of personal data.

Article §4 [18l] of the GDPR defines personal data as “*any information relating to an identified or identifiable natural person*” where an identifiable natural person can be identified, directly or indirectly, by reference to an identifier such as a name, an identification number, etc.

Article §5 [18m] of the GDPR highlights the importance of *data minimization*, which requires that personal data be collected only to the extent necessary for specified and legitimate purposes. Once personal data is collected, the regulation obliges organizations and application developers to implement appropriate technical and organizational measures to ensure its protection. Among these measures, Privacy by Design [Cav09], formalized in Article §25 [18h] of the GDPR, mandates that privacy safeguards be integrated into systems and processes from the earliest stages of software development. This principle requires developers to make deliberate and proactive decisions about how personal data is collected, processed, and stored in order to minimize risks and ensure regulatory compliance.

These measures include effectively disguising personal data using a *pseudonymization* function. Pseudonymization aims to protect personal data by replacing parts of it with unique, non-identifying *pseudonyms*. Personal data can then no longer be attributed to a specific data subject without the use of pseudonyms [Kon19]. Even though pseudonymized data still qualifies as personal data, pseudonymization techniques mitigate data protection risks. By pseudonymizing personal data, organizations may continue to use the data for their purposes while reducing the risk of severe consequences in case of a privacy breach [MJ19].

In addition to protection measures, the GDPR also imposes transparency obligations. Article §13 [18g] of the GDPR mandates that personal data collection and processing be reported to the data subject through documents such as privacy policies.

Taken together, these requirements highlight the need for systematic methods that enable organizations to identify, protect, and transparently document the processing of personal data within software systems.

2.2 Data Collection in Android apps

An Android app may acquire privacy-related data from the user either directly or indirectly. The most direct way to gain access to data is to ask the user to provide it via various input screens and user interfaces.

Android uses eXtensible Markup Language (XML) to describe the user interface layout of an application. The XML layout files play a crucial role in defining the buttons, text fields, and layouts that constitute the application's user interface. A layout XML file defines the structure and visual appearance of a screen and defines the position, appearance, and characteristics of its components and how they interact with each other. *EditText* is a widely used input field that allows users to enter text-based data. It supports various input types, including but not limited to plain text, numbers, email addresses, and passwords.

Figure 2.1 illustrates a screen from Amazon shopping app [26a] that collects credit card information from the user. It consists of *editText* fields for capturing the name and number of the credit card, and dropdown fields to select the expiration date. Attributes such as *android:inputType* define the allowed input type (e.g., text, number, password), while *android:hint* provides a placeholder or instructional text. These attributes describe the type of data to be captured. In Figure 2.1, using the label and hint texts, one can guess that the input field is asking the user to input credit card details.

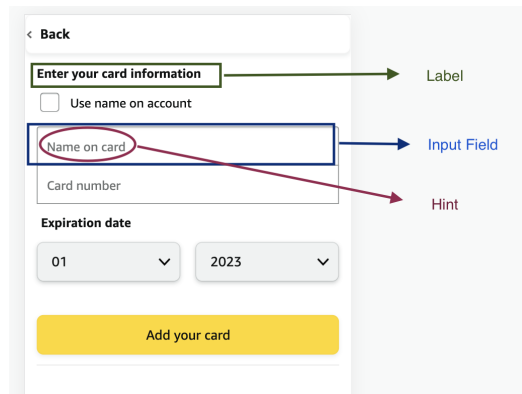


Figure 2.1.: Example: Input fields with labels and Edit Texts from Amazon shopping app [26a].

Since system calls such as *Element.getText()* return the values assigned to these attributes (IDs, hints, labels), static analysis tools [RAB14; Arz+14] classify them as sensitive sources. Hence, these tools effectively end up tainting all user inputs that are returned by such sources. However, this approach may lack precision, as not all user inputs are equally sensitive, and some may not be sensitive at all. Since there are no specific guidelines in Application Programming Interface (API) design to regulate this data, it becomes challenging to monitor and identify whether the data actually is privacy-related. User interfaces are a common means of directly collecting data from the user, emphasizing the need for a more accurate risk analysis of such user input.

An Android app may also acquire data from the Operating System (OS) through system API calls. Some of this data might be privacy-related, such as the GPS information collected through system API calls such as *getLastKnownLocation()*.

To be able to access certain system APIs in the first place, Android apps may request appropriate permissions from the user at install time or runtime. They are declared in the app's *AndroidManifest.xml* file and relate to various system permissions to read, write, or administer the resources. These permissions can be set and reset using the Android settings for the app.

For the remainder of this thesis, we adopt a conservative definition of collected data. This includes data collected via user interface screens, via system API calls in the source code, or as Manifest permissions.

2.3 Program Slicing

Since its introduction in the 1980s [Wei84; Wei82; Xu+05], the technique of program slicing has been used to assist developers in tasks such as debugging, testing, finding bugs, and understanding the code. Program slicing systematically extracts a subset of a program that is relevant to a particular program point, thereby helping developers focus on code elements that influence specific behaviors. A program slice consists of the parts of a program that potentially affect the values computed at some point of interest referred to as a slicing criterion [Xu+05].

Two primary slicing techniques are used in program analysis: *static slicing* and *dynamic slicing*, each offering distinct advantages and limitations. Static slicing analyzes the source code of a program without executing it [Wei84]. By considering all possible execution paths, it identifies data and control dependencies across the entire program, providing a comprehensive view of how information may flow through the code. This makes static slicing particularly useful for general-purpose analysis and debugging, as it enables developers to trace all potential dependencies, including those that may not manifest in any single execution. Key advantages of static slicing include its holistic coverage and its independence from runtime inputs or test cases. However, this comprehensive nature can also result in over-approximation, potentially introducing false positives by including dependencies that do not occur in practice.

In contrast, dynamic slicing analyzes the program during its execution [Azi+19]. It restricts its scope to the execution paths actually taken for a given input, yielding highly precise insights into program behavior under specific scenarios. By focusing only on executed statements, dynamic slicing reduces over-approximation and minimizes false positives. However, its precision comes at the cost of completeness: dynamic slicing depends heavily on the quality and coverage of the provided test cases and may fail to capture relevant behaviors that are not exercised during execution. Furthermore, reliance on runtime information can limit scalability, particularly for large or complex systems where exhaustive testing is infeasible.

Taken together, static and dynamic slicing provide complementary perspectives on program behavior. While static slicing offers a broad overview of all potential data flows within a program, dynamic slicing delivers precise insights into concrete executions. In this thesis, we focus on the design and implementation of static slicing techniques, motivated by the hypothesis that whole-program, static coverage is better suited to supporting systematic analyses and enabling stronger forms of assurance, including potential legal guarantees [NNH99].

2.4 Jicer

Jicer is a static Jimple slicer [PW21] that is specifically designed for Android applications. It is currently the only available static slicer tailored to this domain. It takes Android Application Package (APK) files as input and produces program slices as APK files. Unlike standard Java slicers which are not directly applicable to Android, Jicer addresses several Android-specific challenges, including operation on Dalvik bytecode, awareness of Android activity lifecycles, the absence of explicit main methods, support for inter-component and inter-application communication, handling of callback-based execution, and extensive use of library methods.

In this thesis, we integrate Jicer on the back-end of the implemented static analysis based tool. Specifically, we leverage Jicer’s ability to treat selected system API calls—identified by our analysis algorithm—as slicing sources and to perform forward slicing from these points.

How Jicer Works

Jicer’s slicing approach builds on the classical dependence graph-based slicing algorithm introduced by Horwitz et al. [HRB88]. It first constructs a Program Dependence Graph (PDG) for every method, where nodes correspond to program statements and edges represent control and data dependencies. To support inter-procedural analysis, individual PDGs are combined into a System Dependence Graph (SDG), which is sufficient for modeling basic inter-procedural programs [HRB88].

However, Java’s object-oriented features and the Android application model introduce additional challenges that go beyond the classical SDG formulation. To address these, Jicer extends the traditional approach by transforming a PDG into an Application Dependence Graph (ADG), which incorporates Android-specific constructs and execution semantics [PW21].

Slicing is then performed on the ADG by first identifying the statements that constitute the slicing criterion. Jicer is configurable and supports the generation of program slices based on a specified slicing criterion and slicing direction. It provides three slicing modes: (i) forward slicing, which propagates from a given program point or system API call; (ii) backward slicing, which propagates backward from a program point or system API call; and (iii) chop slicing, which extracts the slice between two specified program points. Once the slicing criterion has been identified, the ADG is traversed using the standard slicing algorithm proposed by Horwitz et al. [HRB88] to compute the relevant slice.

Finally, Jicer generates the sliced application by removing all statements not included in the computed slice. This step is implemented using the Soot framework [Val+10], producing a reduced APK that preserves the behavior relevant to the slicing criterion. Jicer's analysis is flow-, context-, field-, object-, and thread-sensitive, while path sensitivity—known to be undecidable in static analysis—is not supported [Wei84].

An Example

Consider the example in Listing 2.1. The program computes the product of a sequence of integers and prints the result stored in variable *x*. It also maintains a second variable *y*, which is used independently to control a conditional output.

```
1  int x = 1;
2  int y = 0;
3  for (int i = 1; i <= 5; i++)
4  {
5      x = x * i;
6      y = y + 4;
7  }
8  if(y > 4)
9  {
10     System.out.println(y);
11 }
12 System.out.println("Product is" + x);
```

Listing 2.1: A short example to demonstrate program slicing algorithm.

Assume that the slicing criterion is the assignment to variable *x* at line 1. The goal of the forward slice is to **identify all program statements that may be affected by the value of *x***.

The forward slicing algorithm operates by traversing the System Dependence Graph starting from the slicing criterion and following outgoing dependency edges. In this example, the initial assignment to *x* (line 1) influences the update of *x* inside the loop, where *x* is repeatedly multiplied by the loop variable *i* (line 5). This updated value of *x* then influences the output statement that prints the product (line 12).

In contrast, statements involving the variable *y* do not depend on *x* and are therefore excluded from the slice. Although *y* is updated within the same loop and later used in a conditional statement (line 8), it is independent of the computation and propagation of *x*. The resulting forward slice consists of the statements on lines 1, 3, 4, 5, 7, and 12 (cf. Listing 2.3).

```

1  int x = 1;
2  int y = 0;
3  for (int i = 1; i <= 5; i++)
4  {
5      x = x * i;
6      y = y + 4;
7  }
8  if (y > 4)
9  {
10     System.out.println(y);
11 }
12 System.out.println("Product is" + x);

```

Listing 2.2: Original code.

```

1  int x = 1;
2
3  for (int i = 1; i <= 5; i++)
4  {
5      x = x * i;
6
7  }
8
9
10
11
12 System.out.println("Product is" + x);

```

Listing 2.3: Forward slice with all statements that may be affected by the value of x.

Figure 2.2.: Comparison of the original code and the forward slice with respect to variable x.

2.5 Data Privacy Vocabulary (DPV)

The Data Privacy Vocabulary (DPV) [Pan+24] is a semantic framework for representing machine-readable metadata about the use and processing of personal data based on legislative requirements such as the GDPR [25d]. It provides a standardized way to describe privacy-relevant information in a structured and interoperable manner.

DPV is organized around a set of top-level abstract concepts, collectively referred to as *core concepts*. Each core concept serves as the root of a hierarchical taxonomy, where child concepts provide increasingly specific refinements. Figure 2.3 illustrates a subset of these core concepts. For example, the concept *PersonalData* represents personal data at an abstract level and can be specialized into concepts such as *SensitivePersonalData*, which may be further refined as *SpecialCategoryPersonalData*.

The goal of the DPV is to provide a rich collection of concepts for each of the top concepts so as to enable their application within real-world use cases. To this end, DPV introduces the concept *Process* to represent how multiple core concepts are combined and applied within a particular use case. Relationships between concepts are expressed using predicates that capture their semantic connections. For example, the predicate *hasPersonalData* indicates whether a use case involves the collection of personal data, while *hasProcessing* denotes the presence of processing operations (cf. Figure 2.3).

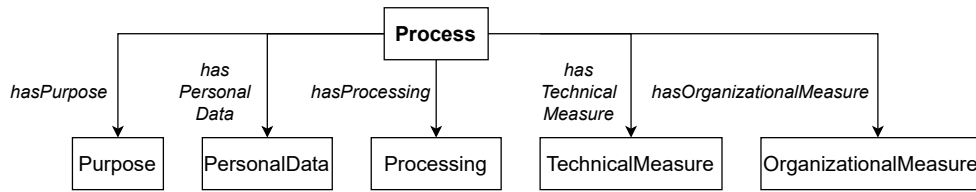


Figure 2.3.: A small subset of DPV core concepts and the possible relationships between them. The complete documentation can be found in the DPV primer [25d].

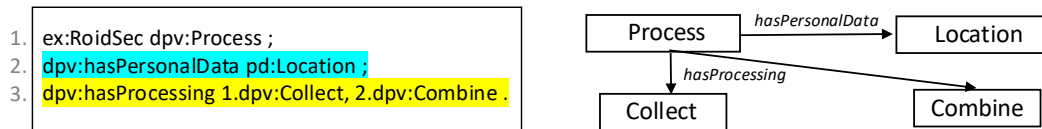


Figure 2.4.: An example DPV model and DPV diagram.

In this thesis, DPV is used to represent the source code of an Android application. Since we use program slicing (cf. Section 2.4) to retain only those parts of the program that are relevant to specific collected data, each resulting program slice is treated as an independent use case and described using the DPV concept *Process*. Accordingly, a program slice may involve multiple core concepts and defined relationships among them.

Figure 2.4 presents the DPV representation of a program slice, corresponding to a use case. The first representation, the DPV model, indicates that the slice processes personal data related to **location** and performs the following processing operations: **collection** and **combination** with other data. Alongside the DPV model, this thesis also uses a DPV Diagram to provide a graphical representation of the same information. In contrast to the DPV model, the DPV Diagram does not explicitly specify the order of processing operations; instead, it offers a high-level summary of the app’s behaviour.

Additional examples of program slices are presented in Table 2.1. The first program slice is from a real-world Android app Steam¹. It uses Google Play API methods to collect **email address** and then writes it to Google storage. According to DPV definitions, this program slice has a *third-party data source* and involves the processing operations of **collection** and **storage**. According to Article §25 [18h] of the GDPR, personal data should be pseudonymized before being stored or processed. DPV model and DPV diagram clearly show the absence of pseudonymization before storing the email address, suggesting a **potential violation** of Article §25. Addi-

¹<https://play.google.com/store/apps/details?id=com.valvesoftware.android.steam.community&hl=en>

Table 2.1.: Some examples of DPV from real-world apps.

Program Slice Property	DPV model	DPV diagram
Email address collected by Google Play Services (third party library) and written to Google storage.	1. <code>ex:App dpv:Process ;</code> 2. <code>dpv:hasPersonalData pd:Email ;</code> 3. <code>dpv:hasDataSource dpv:ThirdParty ;</code> 4. <code>dpv:hasProcessing 1.dpv:Collect, 2.dpv:Store .</code>	
Email address collected, and used to send emails.	1. <code>ex:App dpv:Process ;</code> 2. <code>dpv:hasPersonalData pd:Email ;</code> 3. <code>dpv:hasProcessing 1.dpv:Collect, 2.dpv:Use ;</code> 4. <code>dpv:hasPurpose dpv:CommunicationManagement .</code>	
Phone number collected, and hashed.	1. <code>ex:App dpv:Process ;</code> 2. <code>dpv:hasPersonalData pd:Phone ;</code> 3. <code>dpv:hasProcessing 1.dpv:Collect ;</code> 4. <code>dpv:hasTechnicalMeasure dpv:2.HashFunction .</code>	

tionally, the presence of a third-party data source in DPV model and DPV diagram indicates that the app must adhere to Chapter §5 [18o] of the GDPR to ensure lawful third-party data transfer.

In the next example, the TaintBench [Luo+21] app `Beita_com_beita_contact`² collects `email address` and uses it to send emails. The DPV model records processing operations as `collection` and `usage`, and identifies the purpose of data collection as *communication management* (line 4 in DPV model). Since neither DPV model nor DPV diagram indicates pseudonymization, the app appears to **violate** Article §25.

The final example is from a TaintBench app `Overlay_android_samp`³, which `collects` a `phone number` and *hashes* it. DPV model and DPV diagram demonstrate the use of a `hash function` as a technical measure before processing the data, thus **adhering** to Article §25.

²https://github.com/TaintBench/beita_com_beita_contact

³https://github.com/TaintBench/overlay_android_samp

2.6 Privacy-Related Documentation

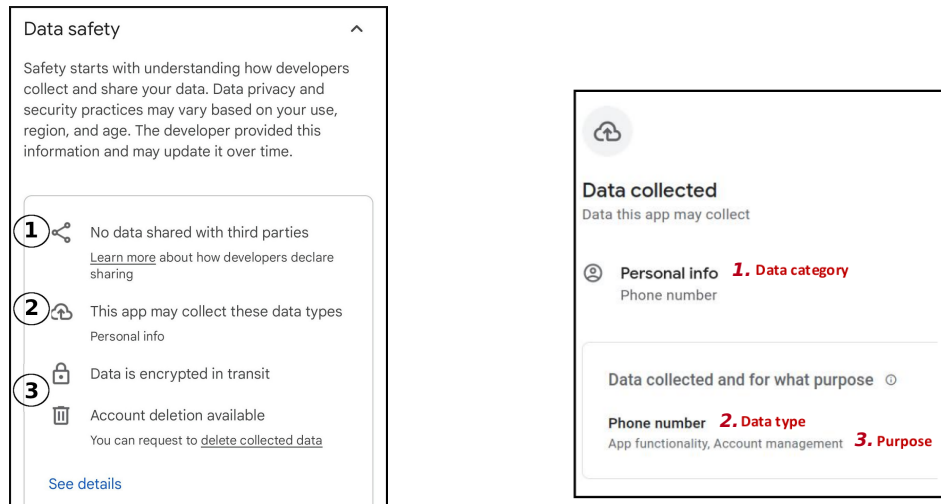
This section summarizes the documentation that is required for an Android app to adhere to GDPR. This includes Data Safety Section (DSS), the documentation developers need to complete before publishing any app on the Google Play Store (cf. Section 2.6.1), and Record of Processing Activities (RoPA), the documentation that companies need to complete to ensure GDPR compliance (cf. Section 2.6.2).

2.6.1 Data Safety Section (DSS)

Among its key provisions, Article §13 [18g] of the GDPR mandates that users be informed of the collection and processing of personal data through documents such as privacy policies. However, such privacy policies are long and vague, and may not always be authored by the app developers. Several studies [Yu+16; Zim+17; Wan+18; Sla+16; TS23] have consistently shown discrepancies between privacy policies and the actual source code, undermining their accuracy and misleading users.

In response to these discrepancies, researchers suggested the use of privacy nutrition labels [Kel+09; KCS13]. Following this, the Apple App Store introduced their labels in 2020 [20]. In 2022, researchers introduced a tool [Gar+22] that helps iOS developers generate privacy labels by identifying data flows through code analysis.

In 2022, Google Play Store introduced their labels by launching the Data Safety Section (DSS) [22c], requiring app developers to disclose how their apps collect, share, and protect user data. The DSS serves as a standardized disclosure mechanism, summarizing an app's privacy practices in three main areas: *data sharing*, *data collection*, and *security practices* (cf. Figure 2.5a). Within the DSS, data is classified into broad *data categories* (e.g., location, personal information, financial information), which are further divided into specific *data types* (cf. Table 2.2). Developers must specify whether these data types are collected or shared, and for what *purposes*, chosen from a predefined list (cf. Table 2.3). These disclosures are displayed on the app's Play Store page, directly informing end-users about its privacy practices (cf. Figure 2.5b). A recent study [Yan+25] involving 50 participants revealed that a majority (76%) considered privacy labels, such as those provided by DSS and Apple, to be a key factor in their decision to download an app.



(a) Data Safety Section, showing ① data sharing, ② data collection, and ③ security practices (encryption and data deletion). (b) *Data collected* part of the DSS, showing 1. data category, 2. data type, and 3. purpose.

Figure 2.5.: An example of the DSS of Signal [22k].

To populate these disclosures, app developers are required to complete a DSS form in the Google Play Console [25j]. An example of the DSS form is shown in Figure 2.6. In this form, developers first select all relevant *data types* their app collects or shares. For each selected type, they must provide detailed information, including:

- whether the data is collected, shared, or both (1. in Figure 2.6);
- whether it is processed ephemerally (2. in Figure 2.6);
- whether its collection is optional for users (3. in Figure 2.6); and
- the purposes for which the data is collected (selected from a fixed set: cf. Table 2.3).

Completing and submitting this form is mandatory for publishing any app on the Play Store, including apps that claim not to collect user data. Submitted forms are reviewed by Google as part of the app approval process to ensure compliance with the platform’s privacy reporting requirements.

A recent study by Mozilla [23b] evaluated popular Android apps by grading them based on the presence or absence of discrepancies between their Data Safety Sections and privacy policies. Khandelwal et al. [Kha+23] observed the evolution of the Data Safety Section and highlighted under-reporting, over-reporting and inconsistencies in data practices. They examined how app developers interact with the Data Safety Section, using a dataset of snapshots without analyzing user interfaces or source code.

Table 2.2.: Google Play Store’s DSS data categories and data types [22f].

Data Category	Data Types
Location	Approximate location, Precise location
Personal Info	Name, Email address, User IDs, Address, Phone number, Race and ethnicity, Political or religious beliefs, Sexual orientation, Other info
Financial Info	Payment info, Purchase history, Credit score, Other financial info
Health & Fitness	Health info, Fitness info
Messages	Emails, SMS/MMS, Other in-app or chat messages
Photos & Videos	Photos, Videos
Audio Files	Voice or sound recordings, Music files, Other audio files
Files & Docs	Files and documents
Calendar	Calendar events
Contacts	Contacts
App Activity	App interactions, In-app search history, Installed apps, Other user-generated content, Other actions
Web Browsing	Web Browsing history
App Info & Performance	Crash logs, Diagnostics, Other performance data
Device or Other IDs	Device or other IDs

Table 2.3.: Google Play Store’s predefined DSS purposes [22h].

Data Purpose	Description
App functionality	Used for features that are available in the app.
Analytics	Used to collect data about how users use the app.
Developer communications	Used to send news or notifications about the app or the developer.
Advertising or marketing	Used to display or target ads or marketing communications.
Fraud prevention, security, and compliance	Used for fraud prevention, security, and legal compliance.
Personalization	Used to customize the app, such as showing recommended content.
Account management	Used for the setup or management of a user’s account.

In 2022, Google introduced Checks [CH22], a paid service that assists app developers in completing the Data Safety Section. However, its cost could hinder many app developers from using its services. Recently, open-source alternatives like Matcha [Li+24] and Privado.ai [22j] have been introduced to assist developers in creating accurate Google Play data safety labels.

2.6.2 Record of Processing Activities (RoPA)

Article §30 [18i] of the GDPR requires organizations to document how personal data is collected, stored, and processed, including details such as processing purposes, data categories, recipients, and retention periods. This documentation, known as the Record of Processing Activities (RoPA) [18q], serves as both a compliance record and a foundation for conducting DPIAs.

Email address

Is this data collected, shared, or both?

1.

Select all that apply. Some types of data collection and sharing are exempt. [View definitions and exemptions](#)

☒ Collected
This data is transmitted off the user's device, either to you (the developer) or a third party. This includes data that is processed ephemeraly, or stored for longer.

☐ Shared
This data is transferred to a third party, either on or off the user's device

Is this data processed ephemeraly?

2.

Processing data "ephemeraly" means accessing and using data while it is only stored in memory, and is retained for no longer than necessary to service the specific request in real-time.

Data collected in this way must still be disclosed, but will not be shown to users on your store listing. [Learn more](#)

☐ Yes, this collected data is processed ephemeraly

☒ No, this collected data is not processed ephemeraly

Is this data required for your app, or can users choose whether it's collected?

3.

[Learn more](#)

☐ Data collection is required (users can't turn off this data collection)

☒ Users can choose whether this data is collected
Data will be shown as optional on your store listing

Figure 2.6.: An example of the DSS form for the data type *Email address* [25f], showing **1.** data collection and/or sharing, **2.** ephemeral processing, and **3.** optional data collection.

Under the GDPR, organizations with more than 250 employees are generally required to maintain a RoPA. However, the obligation is not limited to large organizations. Smaller organizations must also maintain a RoPA where their processing activities are not occasional, involve special categories of personal data [18n], include data relating to criminal convictions and offences [26c], or are otherwise likely to result in risks to the rights and freedoms of individuals. In other words, many small businesses are subject to this requirement if their data processing goes beyond purely incidental activities.

To fulfill its purpose, the RoPA must accurately reflect an organization's actual data processing practices. It should enable the organization to clearly identify:

- The actors involved (controller, processors, representative, joint controller, etc.) in the data processing;
- The categories of data processed;
- The purpose of the processing;
- The duration of personal data retention;
- The technical and organizational security measures implemented [19c].

To support compliance, several data protection authorities, including the French CNIL [18a] and the UK ICO [26f] have published standard templates and guidance for maintaining a RoPA. Failure to maintain or provide adequate records of processing activities may result in administrative fines, as stipulated by the GDPR.

A Study of Android Data Collection

This chapter introduces the first major contribution of this thesis: an analysis of data collection practices of Android applications (*Contribution 1*). First, we propose a multi-layered definition of privacy-related data. This is then used to construct a dataset that classifies possible inputs to an Android app.

Next, we present **Privacy-Related Input Classification Engine (PRICE)**, a command-line tool that detects the data collected by Android app User Interface (UI) screens, source code, and Manifest permissions, and labels it as privacy-related data. We then use PRICE to conduct a quantitative study on 7,500 Android apps, to examine which data is collected the most by Android apps. To further examine UI-based data collection, we complement this quantitative study with a qualitative analysis of the 100 most popular apps each from the domains: *Messaging and Social Media*, *Banking and Finance*, *News and Entertainment*, *Sports and Games*, *Technology and Education*, *E-Commerce and Shopping*, and *Health and Fitness*.

3.1 Privacy-Related Data

The terms “private” and “sensitive” are subjective, lacking a precise definition of which data is considered sensitive within the context of an Android app [Kob+23]. In legal contexts, GDPR’s definition of personal data is directly derived from personal identifiers, which are clearly defined by the Article 29 Working Party [07]. These identifiers consist of information that holds a particularly privileged and close relationship with an individual, allowing for identification [Kon19]. While certain identifiers are sufficient to identify an individual only in some processing contexts, the others may always identify the user. Partial identifiers may only identify users when combined with additional information [Kon19]. However, until Kober et al. [Kob+23] provided a sound definition of sensitive data derived from the definition of personal data of several legal frameworks (including GDPR), there was no such definition for a technical framework such as Android. We build on their assessment and expand on their definition of sensitive data, considering the following categories of privacy-related data collected by an Android app:

1. *Directly identifiable personal data* : This personal data can uniquely identify an individual or a device without the use of any additional information. Examples include an email address, a passport number, a phone number, an International Bank Account Number (IBAN), an IP address. Such data consists of a single piece of information that is sufficient for identification, and hence it is crucial to pseudonymize it before processing.
2. *Partially identifiable personal data* : This type of personal data can identify an individual or a device when combined with additional information. Browsing histories are one example, as they could be linked to some social network profiles such as X or Facebook or Instagram accounts [Su+17]. Kurtz et al. [Kur+16] show that users of iOS devices could be singled out through their personalized device configurations, despite the absence of any device identifiers. Hence, this data should be pseudonymized before processing. Other examples include a postal code, postal address, age, and gender.
3. *Access data* : This category includes data that can grant access to an individual's device but cannot identify an individual's device. For example, a password, a bank Personal Identification Number (PIN), a Card Verification Value (CVV), and a One-Time Password (OTP). While it is not categorized as personal data, it still needs protection since it can cause harm if leaked.
4. *Context-dependent data* : This type of data may identify an individual or a device depending on its content and the context in which it is used. Protection is necessary, but its exact identifiability cannot be determined statically. For example, the content of an email or a message may contain privacy-related information that is challenging to identify without running the app.

We summarize these data categories in Table 3.1, assigning a risk ranking to each category. A certain data can get only one risk rank. However, this risk-based classification is too coarse-grained to precisely report data collection in Android apps. To address this issue, we augment data with additional information. To ensure accurate labeling, these augmentations employ a detailed classification and are defined as a pair: Privacy Label $\rightarrow$ Identifier.

We label potential data sources (API calls or keywords) with a privacy label, which combines privacy relevance and data category (cf. Table 3.1). We introduced these data categories by carefully examining the categories in Google Play Store's Data Safety Section [22c]. We also studied the sensitive Android API sources labeled by SuSi [RAB14] and then classified these data categories according to their pri-

Table 3.1.: Classification of privacy-related data.

Privacy-Relevance	Data Categories	Utility	Risk Rank
Directly identifiable personal data	Personal information, Device or other IDs, Financial information	Direct identification of user	1
Partially identifiable personal data	Personal information, Location data, Device data, Audio data, Browsing data, App activity, Photos and videos, Session data, Calendar data	Identification possible in combination with other data	2
Access data	Authentication, Email authentication, Network authentication, Payment authentication	Authentication	3
Context-dependent data	Message, UI, Audio, Photos and videos, Email	Unknown statically	4

Table 3.2.: Examples of privacy-related data.

Data	Privacy label → Identifier
Tax ID	Directly identifiable financial information → Unique ID
Family name	Partially identifiable personal information → Name
Bank PIN	Access payment authentication data → Password

vacy relevance. In some cases, we introduced some new data categories, such as **partially identifiable device data**. For every piece of data, we then decided its appropriate category.

The second component links the data source to an identifier, manually classified based on Android API documentation and Google Play Store's Data Safety Section. Table 3.2 gives some concrete examples of privacy-related data labeled according to this classification.

Due to the subjectivity and ambiguity of data identifiers, this classification was done manually. Automating it will be an interesting but non-trivial task, and is beyond the scope of this thesis.

3.2 Identifier Datasets

To label sources as potential sources of privacy-related data, we require a dataset that can be used as a classification criterion. We constructed two different datasets: an *Identifier Keywords Dataset* for labeling UI data elements, and an *Identifier API Dataset* for labeling system API calls within the app code.

Initial corpus. In July 2023, we referred to Google Play Store and AppBrain statistics [14b] to select the 50 most downloaded apps each from these categories: *Messaging and Social Media*, *Finance and Banking*, *News and Entertainment*, *Sports and Games*, *Technology and Education*, *E-Commerce and Shopping*, and *Health and Fitness*. We focused on these categories because they represent the major top-level categories of mobile apps and encompass a diverse range of data-collection behaviors. They include both high-risk categories (e.g., banking, health) and general-purpose categories (e.g., social media, entertainment), allowing us to capture representative patterns of identifiability across different types of applications. We used ApkMirror [14a] to access these 350 apps crawled from the Google Play Store, and manually downloaded these 350 APKs.

We automatically extracted all the keywords from user input fields along with their metadata (editText id, input type, label, and hint) from the UI layout files of the selected 350 apps. To generate the initial dataset of system API methods, we extended SootFX [KB21] to detect potential data sources (getter methods) from the source code (APK). We first focused on commonly used Android and Java APIs, such as *android.location*, *android.accounts* and *javax.net*. We then used the AppBrain statistics [14b] to select the most installed third-party libraries across categories such as analytics and advertisements. Table 3.3 gives some examples of these libraries.

To distinguish first-party from third-party data collection, we analyze the package namespaces of all methods that access privacy-related data: methods belonging to the app’s own package are labeled as first-party collection, while calls into known third-party Software Development Kit (SDK) packages (e.g., those listed in Table 3.3 and sourced from AppBrain) are labeled as third-party collection. This separation follows the disclosure boundaries expected in Google Play Store’s Data Safety Section (DSS). We conservatively treat any collection operation performed within an SDK as third-party, regardless of DSS-specific exemptions (cf. Section 4.1.3), since developers remain responsible for declaring third-party data practices.

Finally, we combined the data sources extracted by SootFX with sources generated by SuSi [RAB14] and the predefined sources used by FlowDroid [Arz+14] to produce the initial dataset of potential data collection methods.

Data annotation. After constructing the initial corpus for our datasets, we filtered the metadata keywords and manually annotated them according to the data categories described in Section 3.1. The author of this thesis and a second researcher independently performed an initial round of annotations. We then included feedback from a broader group of domain-adjacent researchers.

Table 3.3.: Third-Party Libraries in the Identifier API Dataset. Not an exhaustive list.

Category	Library
Analytics	Google, Firebase
Authentication	Google, Firebase, Apache
Advertisements	Adjust, Applovin, AppsFlyer, Adcolony, Mopub
Image Processing	Google ZXing
Network	okHttp, Apache, gRPC, Volley
Email	Apache James
Location	Baidu
Phone number validation	Google i18n

Table 3.4.: Part of the *Identifier Keywords Dataset*.

Keyword(s)	Privacy label → Identifier
IBAN, Account number	Directly identifiable financial information → Account
First name, Family name	Partially identifiable personal information → Name
PIN, OTP	Access payment authentication data → Password
Chat	Context-dependent data → Message

To do this, we selected a set of 50 keywords which the the author of this thesis and the second researcher identified as potentially ambiguous and subjective to human judgement, and conducted a session with 22 research scholars from the Computer Science department at our university. We first introduced the data classification scheme and then asked participants to complete a survey evaluating these keywords along with their assigned categories. For each keyword, participants indicated whether they agreed with the proposed classification. Participants agreed unanimously on 45 of the 50 keywords. Two keywords (first name, family name) were reclassified from **directly identifiable personal info** to **partially identifiable personal info** based on the feedback. In addition, responses indicated the need for an additional category to capture data items that did not fit well into existing directly identifiable categories; we therefore introduced **directly identifiable other data**, which includes items such as signatures, biometric data, and images.

The final *Identifier Keywords Dataset* was constructed by combining the updated annotations from this survey process. A part of this dataset is illustrated in Table 3.4.

The author of this thesis and a second researcher manually examined the official API documentation to annotate system API calls from the initial corpus, classifying them into the data categories described in Section 3.1. In cases where documentation was not available, we studied the labels SuSi [RAB14] associated with these calls. This annotated information formed the basis for the *Identifier API Dataset*, as illustrated in Table 3.5.

Table 3.5.: Part of the *Identifier API Dataset*.

Method signature	Privacy label → Identifier
<i>android.net.IpPrefix:</i> <i>java.net.InetAddress</i> getAddress()	Directly identifiable device or other IDs → IP Address
<i>android.location.Location:</i> <i>double</i> getLatitude()	Partially identifiable location data → Approximate location
<i>com.google.android.gms.auth.api.identity:</i> <i>SignInPassword:</i> <i>java.lang.String</i> getPassword()	Access email authentication data → Password
<i>android.app.Activity:</i> <i>android.view.View</i> findViewById(int)	Context-dependent UI data → Text field

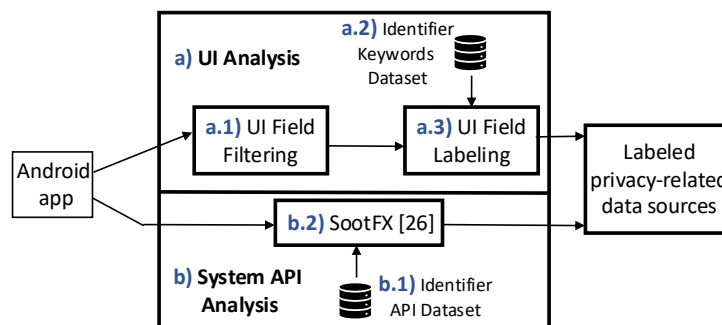


Figure 3.1.: Workflow of Privacy-Relevant Input Classification Engine (PRICE).

We decided to annotate the datasets manually since it was non-trivial to automate this, especially for cases that required extensive discussion among the researchers. For instance, we categorized IP addresses under **directly identifiable device or other IDs** rather than combining it with other device information and classifying it as **partially identifiable device data**. We carefully discussed ambiguous cases, which could have been classified under multiple categories. We classified postal address as **partially identifiable personal information** and the more general keywords such as postal code, city, or country as **partially identifiable location data**.

3.3 Privacy-Relevant Input Classification Engine (PRICE)

Privacy-Related Input Classification Engine (PRICE) is a command-line tool that accepts an APK as input and generates labeled privacy-related data sources found in the source code (cf. Figure 3.1). We now discuss the modules needed to detect privacy-related data in the source code.

a) UI Analysis. Analyzing UI screens statically is challenging because the behavior of fields depends on human interaction. However, by exploring the attributes associated with these fields which are defined in the layout XML file for that screen, we can deduce the fields and the data they capture to a certain extent [Hua+15; Nan+15].

We first decompile the APK file using *apktool* [10] and extract all XML files that present user screens. Next, we filter all user input fields and their associated attributes such as identifier, label, hint, and input type (a.1). Attributes such as *android:inputType* define the allowed input type (e.g., text, number, password), while *android:hint* is an instructional text and suggests the format of data. Attributes such as label give additional information about the data to be inserted. All *editText* fields have a unique identifier that is used to fetch the value and trigger events. It is reasonable to assume that this identifier contains text defining the data it collects (e.g., an identifier *txt_name* suggests a field for collecting a name). Combining the identifier with the input type and hint provides a good starting point for heuristics to determine information sensitivity [Hua+15; Nan+15].

Our field labeling analysis (a.3) starts by examining the type of the user input field. Input fields with types such as *Password* are immediately classified as privacy-related information. If the input type does not reveal the nature of the information, we check the unique ID of the *editText* field and compare it with the *Identifier Keywords Dataset* (a.2). In many cases, this unique ID contains important information about the data. In case we are still unable to decide the sensitivity, we examine the hint attribute and the text associated with the field. Combining these attributes gives a good idea about the input type. We then use the *Identifier Keywords Dataset* to assign a privacy label to the input field. Additionally, we disclose all the permissions declared in the app's *AndroidManifest.xml* file. The analysis is currently limited to textual data collected by an Android app. Identifying and classifying more complex data (such as images, audio, video, and biometrics) is out of the scope of the analysis.

b) System API Analysis. To analyze the privacy relevance of system API methods in an Android app, we extend SootFX [KB21], a static code-feature extraction tool for Java and Android. SootFX is a standalone tool that uses the Soot framework [Val+10] to extract features such as methods, classes, and the call graph of the target APK. SootFX allows the extraction of whole-program features such as the number of method calls or the number of reachable statements in a target program.

For PRICE, we introduce a new whole-program feature that uses our *Identifier API Dataset* (b.1) as input and checks the APK for privacy-sensitive system API calls. To this end, SootFX (b.2) uses the Soot Pointer Analysis Research Kit (SPARK) call graph construction algorithm [LH03] to extract the static call graph of the APK, and matches every method call with the methods described in the *Identifier API Dataset*.

If a match is found, it looks for the privacy label corresponding to the method call and labels it as a privacy-related data source.

3.3.1 Sanity Checks

Before conducting our experiments, we performed a sanity check to assess the reliability of PRICE, since all subsequent analyses depend on its ability to identify privacy-related data. For User Interface (UI), PRICE processes each app in an average of 12.18 seconds. We manually examined 25 randomly selected apps spanning multiple domains, producing 536 UI-based data records out of approximately 24,000 extracted records. In this sample, we identified 379 true positives, 30 false positives, 98 true negatives, and 29 false negatives, resulting in a precision of **0.926** and a recall of **0.928** (F1-score: **0.927**).

When mislabeling occurs, it is primarily caused by substring-based matches in short tokens. For example, the UI text “zip” may trigger a match on the substring “ip”, which can lead PRICE to incorrectly label the field as collecting an IP address. Similar errors arise when short identifiers (e.g., “id”, “pin”) appear as parts of longer, unrelated words or abbreviations. These cases suggest that the errors are related to the ambiguity introduced by short keyword patterns. In future work, these false matches could be reduced by adding stronger token-boundary constraints and context-aware disambiguation (e.g., lightweight Natural Language Processing (NLP) or domain-specific rules) to ensure that labels are inferred from whole-word matches and surrounding context rather than substrings alone.

For system API analysis, we rely on SootFX [KB21], an established static analysis framework. SootFX detects system API sources in approximately 71.42 seconds per app. As SootFX is widely validated and used in prior work [KB21], we treat its identification of API-level data flows as ground truth for the purpose of our analyses.

3.3.2 Limitations

PRICE has several technical and methodological limitations. First, the accuracy of its results depends on how precisely one constructs the datasets. Since PRICE relies on curated keyword and API datasets, any missing, ambiguous, or incorrectly classified entries may affect the quality of the analysis. In addition, the degree of identifiability relies on human subjectivity and it may vary with different domains and compliance regimes. The proposed definition of data focuses on GDPR but modifying the datasets is sufficient to adapt the analysis to alternative compliance regimes.

Another limitation stems from the annotation process. After independently annotating the initial corpus, the author of this thesis and another researcher identified a subset of 50 keywords on which they disagreed or considered ambiguous. Instead of relying on a formal inter-annotator agreement metric, we sought feedback from a broader group of 22 domain-adjacent researchers. Although this approach helped refine several classifications and led to the introduction of an additional category, it may not fully eliminate subjectivity in interpreting keyword meaning. Moreover, the selection of 50 keywords for external review may limit the generalizability of the feedback.

Since certain keywords in the *Identifier Keywords Dataset* may have different meanings in different contexts, they may produce false positives. In early experiments, we observed that the UI analysis automatically labeled the keywords “body” and “height” as *partially identifiable health and fitness data*. In reality, input fields in messaging and e-commerce apps use the keyword “body” when describing the body of a chat or an email, and many apps use the keyword “height” to describe the dimensions of an image, making these input fields *context-dependent data* instead. The mislabeling initially occurred because we prioritized labeling risk 2 data (partially identifiable data) before moving on to risk 4 data (context-dependent data), causing these keywords to be labeled incorrectly. To address this issue, we adjusted the order of precedence levels of risk 2 health data in different domains, reducing false positives by 0.5%. We consider the remaining false positives to be insignificant. However, in the future, we aim to explore more advanced Natural Language Processing (NLP) techniques similar to UiRef’s disambiguation strategy [And+17] to further mitigate the risk of false positives.

Despite the process of third-party library selection we outlined in Section 3.2 and Table 3.3, the *Identifier API Dataset* may miss out on some libraries. In future work, we will explore whether library detection tools [17; 16; HLT18] can help expand coverage to a wider range of third-party libraries.

The UI analysis is currently limited to textual data collected by an Android app, but it can be broadened to include a wider range of UI widgets. It focuses on scanning the layout files of Android apps, and does not associate privacy-related input fields with their corresponding variables in the source code. Therefore, while PRICE can identify potential privacy-related data collection points in the UI, it does not establish how this data is subsequently processed by the app.

For system API analysis, SootFX matches the method signatures in the call graph with the labeled methods present in the *Identifier API Dataset*. Since it uses keyword matching to detect the presence of data sources including those from some third-party libraries, its analysis will not be able to handle code obfuscation in cases where the obfuscation renames also calls to these third-party APIs.

The extent of this limitation depends on the type of API being analyzed. Android framework and SDK APIs are largely unaffected by application-level obfuscation, because developers cannot rename platform-level method signatures belonging to Android system services or framework classes. As a result, calls to standard Android APIs remain identifiable in the call graph. In contrast, third-party libraries packaged with an application may be renamed by obfuscation tools such as ProGuard [26h]. If such libraries are obfuscated without appropriate keep rules, signature-based and keyword-based matching may fail to recognize relevant method calls. This can result in false negatives, where privacy-related data sources are missed.

This limitation mainly affects the completeness of the API-based analysis rather than the reliability of the matches that are detected. When SootFX identifies a known API source, the detection remains valid; however, privacy-relevant sources in obfuscated third-party libraries may go undetected. To mitigate this limitation, our approach combines API analysis with UI analysis. The UI analysis detects privacy-related input fields from Android layout files and textual UI elements, which can provide evidence of privacy-related data collection even when source code variables or third-party API calls are difficult to recover due to obfuscation. Therefore, the UI and system API analyses complement each other and reduce, although do not fully eliminate, the risk of missed detections caused by obfuscation.

The practical impact of this limitation may also be reduced by the prevalence of obfuscation in real-world Android applications. Wormke et al. [Wer+18], in a large-scale study of 1.7 million free Android applications from Google Play, found that only 24.92% of apps were obfuscated by developers. This suggests that while obfuscation is an important limitation to acknowledge, it may not affect the majority of applications in typical datasets.

3.4 Analyzing Data Collection Practices using PRICE

We next use PRICE to automatically investigate how privacy-related data is collected by Android apps. The experiments reported in this section were conducted in 2023 on a Linux virtual machine with an Intel(R) Xeon(R) Platinum 8462Y+ 8-core processor with 128 GB memory. Our empirical evaluation addresses the following research questions:

RQ1. *How do Android apps collect privacy-related data?*

This question examines the mechanisms through which Android apps collect privacy-related data, including system API calls, user interface inputs, and manifest-declared permissions. By identifying and quantifying these collection channels, we aim to uncover overall data collection trends and establish a comprehensive understanding of how privacy-related data flows within Android apps. This analysis serves as a foundation for accurately characterizing data collection practices and for subsequent evaluations of privacy disclosures.

RQ2. *Which data categories are directly collected from users by the most downloaded apps across different domains?*

This question focuses specifically on data collected through user interfaces, as these inputs often reflect explicit interactions between users and applications. By analyzing UI-based data collection across app domains, we aim to identify domain-specific patterns and differences in the types of personal data requested from users. Understanding these patterns helps reveal how user-facing data collection varies by application context and highlights domains where users may be exposed to more invasive data collection practices.

3.4.1 RQ1. Data Collection by Random Apps

To answer RQ1, we evaluate PRICE on 7,500 randomly chosen Android apps. We downloaded these apps using *AndroZoo* [All+16], a research database of Android applications created and maintained by University of Luxembourg. Since GDPR was introduced in 2018, we selected apps that were released on Google Play Store after October 2018. We downloaded APKs with a maximum size of 300 MB.

We present the general results in Table 3.6. We observe that 95.32% of the Android apps that PRICE could successfully parse collect data from user interfaces. Among them, **31.52%** of apps collect privacy-related data through user interfaces. We ob-

Table 3.6.: Statistics of 7,500 apps. All tables in this section present percentages between 50-100 in red, 20-50 in yellow, and below 20 in green.

	#Apps	%Apps
Successful analysis (UI)	6922	92.3%
Successful analysis (system API)	7150	95.33%
Parsing errors (UI)	578	7.7%
Parsing errors (system)	350	4.66%
With UI fields	6600	95.32%
With privacy-related UI fields	2182	31.52%
With system API privacy-related data	7150	100%
With both UI and system API privacy-related data	2182	31.52%
With permissions	6378	92.14%
With UI and system API privacy-related data but no permissions	57	0.82%

Table 3.7.: UI privacy-related data distribution (2182 apps).

Data	#Apps	%Apps
Directly identifiable personal data	694	31.81%
Partially identifiable personal data	1094	50.14%
Access data	596	27.31%
Context-dependent data	987	45.23%

Table 3.8.: System API privacy-related data distribution (7150 apps).

Data	#Apps	%Apps
Directly identifiable personal data	7150	100%
Partially identifiable personal data	7150	100%
Access data	7150	100%
Context-dependent data	5821	81.41%

serve that *all* of the apps that PRICE could successfully parse collect at least *some* privacy-related data through system API calls in their source code, details given below.

All apps that collect privacy-related data through user interfaces collect some privacy-related data through system APIs, too (cf. Table 3.6). We summarize privacy-related data distribution in Table 3.7 and Table 3.8.

A notable observation is that **50.14%** of the apps that gather privacy-related data through user interfaces collect *partially identifiable personal data* (risk 2), while only **31.81%** of apps collect *directly identifiable personal data* (risk 1). In contrast, all apps that have reachable privacy-related data sources through system APIs collect personal data.

To examine this observation in detail, we provide a detailed breakdown of personal data in Table 3.9 and Table 3.10. We observe that user interfaces collect *directly identifiable and partially identifiable personal information* almost to the same extent

Table 3.9.: UI personal data distribution (2182 apps).

Data	Category	#Apps	%Apps
Directly identifiable personal data	Personal information	629	28.83%
	Device or other IDs	233	10.68%
	Financial information	130	5.96%
Partially identifiable personal data	Personal information	764	35.01%
	Financial information	757	34.69%
	Location data	357	16.36%
	Health and fitness	177	8.11%
	Calendar data	143	6.55%

Table 3.10.: System API personal data distribution (7150 apps).

Data	Category	#Apps	%Apps
Directly identifiable personal data	Device or other IDs	7150	100%
	Personal information	1551	21.69%
Partially identifiable personal data	Personal information	7150	100%
	Location data	7150	100%
	Browsing data	7150	100%
	Device data	3260	45.59%
	Audio data	1751	24.49%
	Session data	259	3.62%
	App activity	253	3.54%
	Photos and videos	51	0.71%
	Calendar data	15	0.21%

as *partially identifiable financial information*. By our observation, user interfaces commonly collect personal information such as names, email addresses, and phone numbers, along with access data like passwords.

In contrast, calls to system APIs mostly collect device, location and browsing information through the OS. The identifier API dataset links one category to multiple identifiers, which are then associated with several system API calls. Consequently, the presence of even one dominant system call indicates data collection. We can attribute the 100% data collection in four categories to specific identifiers such as IP address and Media Access Control (MAC) address (device or other IDs), user data from Uniform Resource Locators (URLs) (personal information), approximate location (location data) and Hypertext Transfer Protocol (HTTP) cookie name (browsing data).

Our experiment reveals that only **27.31%** of all Android apps collect access data through user interfaces. The high percentage of system API-based access data collection (cf. Table 3.8) can be traced back to predominant network-based authentication present in the source code, such as the use of the `java.net.PasswordAuthentication:char[] getPassword()` system call.

Table 3.11.: Statistics of 700 most downloaded apps (100 from each domain).

	#Apps	%Apps
Successful analysis (UI)	683	97.57%
Successful analysis (system API)	668	95.43%
Parsing errors (UI)	17	2.42%
Parsing errors (system API)	32	4.57%
With UI fields	677	99.12%
With privacy-related UI fields	552	80.81%
With system API privacy-related data	668	100%
With both UI and system API privacy-related data	552	80.81%
With permissions	665	97.36%
With UI and system API privacy-related data but no permissions	3	0.44%

Moreover, we observe that **45.23%** of all apps collect context-dependent data through user interfaces, emphasizing the importance of dynamic analysis to effectively assess the risks associated with collection of such data in specific contexts. The high percentage (81.41%) of context-dependent data collection through system API calls can be attributed to the presence of the *findViewById(...)* method call, which connects the source code with the text fields in the layout files.

RQ1 summary: Through user interfaces, Android apps predominantly collect personal and financial information; and device, location and browsing information through system APIs. *Partially identifiable personal data* (risk 2) is the most frequently collected data.

3.4.2 RQ2. UI-Based Collection by Domain Apps

To answer RQ2, we extend our experiment to analyze in depth 100 apps each from the following domains: *Messaging and Social Media*, *Banking and Finance*, *News and Entertainment*, *Sports and Games*, *Technology and Education*, *E-Commerce and Shopping*, and *Health and Fitness*. In July 2023, we referred to Google Play Store and AppBrain statistics [14b] to select 50 most downloaded apps from each domain. We used AppBrain and AndroidRank statistics [11] again in October 2023 to choose 50 additional most downloaded apps from these domains. We manually downloaded these APKs, and we evaluate this research question by running PRICE on these 700 APKs.

We present the general results in Table 3.11 and the privacy-related data distribution in Table 3.12. We observe that **76.63%** of the apps that gather privacy-related data through user interfaces collect *partially identifiable personal data*, while **62.32%** of apps collect *directly identifiable personal data*.

Table 3.12.: UI privacy-related data distribution (552 most downloaded apps).

Data	#Apps	%Apps
Directly identifiable personal data	344	62.32%
Partially identifiable personal data	423	76.63%
Access data	259	46.92%
Context-dependent data	521	94.38%

Table 3.13.: Domain-wise UI privacy-related data distribution. Highlighted: highest percentage of apps collecting data.

Data	Percentage of apps that collect data (per domain)						
	E-commerce and Shopping	Messaging and Social Media	Banking and Finance	Sports and Games	Technology and Education	Health and Fitness	News and Entertainment
Directly identifiable personal data	71.11%	84.34%	57.89%	39.22%	52.27%	67.09%	56.47%
Partially identifiable personal data	83.33%	91.57%	76.32%	54.90%	78.41%	81.01%	65.88%
Access data	55.56%	60.24%	55.26%	29.41%	42.05%	48.10%	32.94%
Context-dependent data	97.78%	97.59%	96.05%	84.31%	93.18%	97.47%	91.76%

The domain-wise privacy-related data distribution is summarized in Table 3.13. Since the system API privacy-related data distribution is uniformly high due to the constant presence of some specific method calls in the source code, we focus on privacy-related data that is collected directly from the user via user interfaces (cf. Table 3.13).

We examine the different categories of personal data collected by user interfaces for different domains. The results are illustrated in Table 3.14, and can be summarized as follows:

- The highest percentage of apps collecting *directly identifiable financial information* (34.44%) and *partially identifiable location data* (61.11%) belong to the domain ***E-commerce and Shopping***.
- The ***Messaging and Social Media*** domain has the highest proportion of apps collecting *directly identifiable personal information* (73.49%), *directly identifiable device or other IDs* (54.22%) and *partially identifiable personal information* (77.11%).
- The highest percentage of apps collecting *partially identifiable financial information* (44.74%) belong to the domain ***Banking and Finance***.
- Among all the various domains, ***Sports and Games*** apps collect the least amount of personal data.

Table 3.14.: Domain-wise UI personal data distribution. Highlighted: highest percentage of apps collecting data. Risk 1: directly identifiable, Risk 2: partially identifiable.

Data	Category	Percentage of apps that collect data (per domain)						
		E-com and Shopping	Messaging and Social Media	Banking and Finance	Sports and Games	Technology and Education	Health and Fitness	News and Entertainment
Risk 1	Personal info	65.56%	73.49%	46.05%	33.33%	42.05%	54.43%	45.88%
	Financial info	34.44%	25.30%	26.32%	3.92%	15.91%	17.72%	8.24%
	Device or other IDs	35.56%	54.22%	27.63%	15.69%	25%	36.71%	23.53%
Risk 2	Personal info	65.56%	77.11%	52.63%	41.18%	63.64%	64.56%	55.29%
	Location data	61.11%	48.19%	34.21%	17.65%	21.59%	27.85%	21.18%
	Financial info	34.44%	14.46%	44.74%	0%	6.82%	22.78%	3.53%
	Calendar data	18.89%	13.25%	19.74%	0%	10.23%	21.52%	5.88%
	Health, fitness	3.33%	7.23%	2.63%	1.96%	3.41%	17.72%	4.71%

- Unsurprisingly, **Health and Fitness** apps collect *partially identifiable calendar data* (21.52%) and *partially identifiable health and fitness information* (17.72%).
- The most collected data category among all the domains is *partially identifiable personal information*, closely followed by *directly identifiable personal information*.

According to our observations (cf. Table 3.13), 60.24% of **Messaging and Social Media** apps collect access data through their user interfaces, which is the highest among all domains. They are followed by **E-Commerce and Shopping** apps (55.56%) and **Banking and Finance** apps (55.26%). This is surprising, since one would reasonably expect shopping and banking apps to authenticate their users at least as much as messaging and social media apps.

Our dataset contains various identifiers that can *partially identify* a user. This suggests that one can define “dangerous combinations of data” based on these categories and alert app developers about them. For example, a user’s first name is insufficient to identify them, but combining it with other information like postal address or family name or age could potentially identify the user. In a broader sense, combining multiple instances of risk 2 data could result in risk 1 data.

To this end, we automatically analyze the report generated by PRICE and examine some combinations of partial identifiers collected by the selected 700 apps. The results, summarized in Table 3.15, reveal that **E-Commerce and Shopping** apps col-

Table 3.15.: Identifier combinations collected by most downloaded apps, Highlighted: highest percentage of apps collecting the identifier combination.

Identifier combination	Percentage of apps that collect the identifier combination						
	E-com and Shopping	Messaging and Social Media	Banking and Finance	Sports and Games	Technology and Education	Health and Fitness	News and Entertainment
Name, Approx location	41.11%	33.73%	19.74%	11.76%	18.18%	22.78%	12.66%
Phone number, Email address	34.44%	34.94%	25%	11.76%	7.95%	22.78%	16.46%
Photo, Text	23.33%	27.71%	18.42%	13.73%	18.18%	26.58%	24.71%
Name, Postal address	21.11%	14.46%	14.47%	3.92%	7.95%	8.86%	6.33%
Approx location, Postal address	21.11%	9.64%	10.53%	3.92%	5.68%	6.33%	5.06%
Name, Card details	23.33%	6.02%	22.37%	0%	4.55%	7.59%	1.27%
Email address, Card details	20%	6.02%	14.47%	1.96%	6.82%	7.59%	1.27%
Card details, Bill amount	10%	2.41%	19.74%	0%	0%	1.27%	12.66%
Name, Birthday	4.44%	4.82%	1.32%	0%	0%	2.53%	1.27%

lect the highest percentage of partial identifier combinations. These include *names* along with other partial identifiers related to location (*postal address*, *approximate location*) and financial information such as *card details*. Unsurprisingly, **Banking and Finance** apps collect multiple partial identifiers related to financial information. Our results indicate that **Messaging and Social Media** apps collect context-dependent data by combining photos and texts/SMSs. It may be sensible to assign a higher risk ranking to context-dependent data collected by a health and banking app as compared to context-dependent data collected by other app domains.

RQ2 summary: Through user interfaces, *Messaging and Social Media* apps predominantly collect personal and access data. In contrast, *Sports and Games* apps collect the least amount of privacy-related data. Many apps collect combinations of partial identifiers from users, with those in the *E-Commerce and Shopping* domain collecting the highest percentage of such combinations.

3.4.3 Discussion

Our quantitative study on 7,500 Android apps reveals that most apps collect data that can *partially identify* a user (risk 2 data). While system API methods constantly collect privacy-related data, user interfaces demonstrate more controlled patterns of data collection. A more focused study on various domain apps confirms that partially identifiable data is the most commonly collected type, with the user interfaces of highly downloaded apps analyzed in RQ2 collecting more privacy-related data than

Table 3.16.: UI privacy-related data distribution for random apps (RQ1) vs. most downloaded apps (RQ2).

Data	%Random Apps (RQ1)	%Most Downloaded Apps (RQ2)
Directly identifiable personal data	31.81%	62.32%
Partially identifiable personal data	50.14%	76.63%
Access data	27.31%	46.92%
Context-dependent data	45.23%	94.38%

those of randomly selected apps from RQ1 (cf. Table 3.16). This finding highlights the need for more refined UI analysis in privacy assessments, as existing approaches that focus solely on APIs may overlook significant data flows originating from app screens.

Our study also reveals domain-specific risks. *E-Commerce and Shopping* apps collect the highest proportion of partial identifier combinations. This raises important questions about how developers prevent these data elements from being combined in ways that could lead to user identification. For example, if multiple partially identifiable data points are processed together, developers need to ensure appropriate pseudonymization to prevent re-identification. Historical studies, such as Sweeney [Swe00], have shown that combinations of postal code, gender, and date of birth can uniquely identify a large portion of the population in the United States. In our analysis, we identified three apps—Finopaytech (*Banking and Finance*), Aliexpress (*E-Commerce and Shopping*), and Meetme (*Messaging and Social Media*) — that directly collect these three partial identifiers. These findings highlight the urgent need for heuristics or automated checks to assess the risk associated with combinations of partial identifiers when analyzing Android apps via source code.

Finally, our evaluation highlights actionable opportunities for supporting developers through tooling. The prototype reveals nuanced data collection patterns in user interfaces that can inform the design of privacy-aware apps. A potential plugin could leverage these insights to alert developers in real-time about the data collected on specific screens, guiding them toward “privacy by design” practices and helping ensure accurate reporting.

Takeaway. The user interfaces of the most downloaded apps collect significant amounts of privacy-related data, with *partially identifiable data* (risk 2) being the most frequently collected.

3.4.4 Limitations and Threats to Validity

While the number of apps in our experiments might appear limited, we ensure a comprehensive evaluation by including the most popular apps from different domains actively used by people. Since we had to manually download domain-specific apps to answer RQ2, we chose 100 apps per domain. However, selecting the top apps per domain could threaten external validity. Therefore, our findings may not generalize to less popular apps, apps from other domains, apps distributed outside the studied marketplace, or apps targeting different user populations.

A threat to internal validity arises from the manual steps involved in app selection and domain assignment. Since domain-specific apps were manually downloaded, researcher judgment may have influenced which apps were included. We mitigated this threat by using a consistent selection criterion and selecting 100 apps per domain. Nevertheless, alternative sampling strategies could lead to different results.

Another threat to internal validity comes from the dependence of the experiment on the analyzed app versions. Android apps are frequently updated, and different versions of the same app may include different UI elements, third-party libraries, permissions, or data collection behavior. Therefore, the results reflect the specific versions of the apps analyzed in this experiment.

Finally, the experiment was conducted in 2023. Since then, apps, Android APIs, and third-party libraries may have changed. Consequently, the findings may not fully reflect the current Android ecosystem. A replication with newer app versions may produce different results.

3.5 Related Work

Several studies analyze the text in user interfaces to detect privacy policy violations and malicious activities [Wan+18; Hua+14; Pan+13]. SUPOR [Hua+15] and UIPicker [Nan+15] can identify sensitive data collected through user interfaces of Android apps. Unfortunately, these are in-house proprietary tools and do not focus on GDPR's definition of personal data. UiRef [And+17] analyzes user interfaces to detect sensitive inputs and shows promising results, but its algorithm needs adaptation to the post-GDPR definition of privacy-related data.

Many previous studies on taint analysis [Arz+14; TØ22; Li+15] have used predefined sets of methods which are potential sources of system API privacy-related data. Follow-up work then proposed machine-learning approaches for automatically classifying and categorizing methods into sources [RAB14; PNB19; Pis+20; Sam+23].

In 2023, Kober et al. [Kob+23] provided a sound definition of sensitive data derived from the definition of personal data of several legal frameworks (including GDPR). They further publicly provided a list of sensitive sources from the Android framework which we used in our work.

Although related research exists, there remains a **research gap** in systematically identifying and labeling all collected data as privacy-related categories relevant to the post-GDPR world. Our work addresses this gap by introducing the *Identifier Dataset* and PRICE, which together support the classification of privacy-related data collected by Android applications.

3.6 Summary

In this chapter, we introduced a multi-layered definition of privacy-related data to correctly report data collection in Android apps. This data definition focuses on GDPR's definition of personal data. Building on this, we described the construction of a dataset of privacy-sensitive data classes that can be used by researchers to classify inputs to Android apps. The dataset accounts for data collected both through user interfaces and via system APIs.

We also presented PRICE (**P**rivacy-**R**elated **I**ntput **C**lassification **E**ngine), a command-line prototype that statically extracts and labels privacy-related data collected via app source code, user interfaces, and permissions. Using PRICE, we conducted a quantitative study of 7,500 Android apps. Our results show that apps most frequently collect data that can partially identify users. While system APIs consistently collect large amounts of location and device privacy-related data, user interfaces exhibit some more diverse data collection patterns. A more focused study on various domains of apps reveals that the largest fraction of apps collecting personal data belong to the domain of *Messaging and Social Media*. In contrast, *Sports and Games* apps collect the least amount of privacy-related data. Finally, our findings indicate that location data is collected particularly frequently, especially by apps in the *E-commerce and Shopping* domain.

Overall, this chapter provides a detailed characterization of personal data collection in Android apps in the post-GDPR landscape. It also highlights the need for more fine-grained analyses of user interfaces, as existing approaches that focus solely on APIs may miss significant data flows originating from app screens.

Understanding Privacy-Aware Reporting Practices

This chapter describes the second contribution of this thesis: an assessment of whether Android developers correctly report collected data in Google Play Store’s Data Safety Section (DSS), and an investigation of the challenges they encounter during DSS form completion (*Contribution 2*).

To this end, we first use PRICE, the static analysis tool we introduced in Chapter 3, to observe if Android apps accurately report the data that they collect via DSS. Using two widely used Messaging and Social Media applications (Signal and Instagram) as illustrative case studies, we demonstrate how developers may both *under-report* and *over-report* data collection, and we identify specific categories of data that are inaccurately disclosed.

Building on these observations, we then conduct an *empirical study* aimed at understanding the broader challenges developers face in privacy-related data disclosure. This study combines insights from a developer survey with an analysis of online discussions related to the DSS.

4.1 Analyzing Data Disclosures using PRICE

We begin by using PRICE to compare data collected by Android apps with the data reported via the Data Safety Sections. This comparison allows us to evaluate the extent to which developers accurately disclose their apps’ data collection practices.

To this end, we conduct an experiment to analyze how app developers disclose collected data through their Data Safety Sections. For this experiment, we focus on popular apps. Our empirical evaluation addresses the following research questions:

RQ1. *How do Data Safety Sections report data collection and sharing?*

This research question analyzes the information disclosed in Data Safety Sections to identify common reporting practices, patterns, and trends among popular Android apps. By examining these disclosures, we aim to understand how developers describe their data collection and sharing behaviors and to what extent DSS entries provide consistent and informative privacy reporting.

RQ2. *How does the reported data collection compare to the data actually collected by the source code?*

This research question uses PRICE (cf. Section 3.3) to systematically compare data disclosed in Data Safety Sections with the data collection behavior inferred from app source code. The goal is to assess the accuracy of DSS disclosures and to determine whether developers tend to under-report or over-report specific categories of privacy-related data. This comparison provides insight into potential challenges developers may face when completing DSS forms and highlights discrepancies between reported and actual data collection practices.

For our experiment, we initially selected 70 Android apps. In November 2023, we referred to AppBrain statistics [14b] to select 10 popular apps from each of the following (7) domains: *Messaging and Social Media*, *Banking and Finance*, *News and Entertainment*, *Sports and Games*, *Technology and Education*, *E-Commerce and Shopping*, and *Health and Fitness*. Based on the findings from RQ1, we narrowed down our focus to 20 apps in RQ2.

4.1.1 RQ1. Analysis of Data Safety Sections

To answer RQ1, we manually examine the Data Safety Sections of 70 apps to understand how data collection and sharing is reported. The Data Safety Section classifies all user data into different *data categories*, which are further divided into specific *data types* (cf. Table 2.2). This data can be collected or shared for 7 different *purposes* (cf. Table 2.3). For RQ1, we use the classification proposed by Khandelwal et al. [Kha+23] to categorize Data Safety Sections that report the collection of all data types or attribute most data types to 6 or 7 purposes as *over-reporting* practices. Conversely, sections that claim no data collection or report only one data type are categorized as *under-reporting* practices.

Data collection. Among all the apps that claim to collect data, only two apps (Signal [22k] and Telegram [26i]) claim not to collect *device or other IDs*. The most frequently reported data categories are *device or other IDs*, *personal information*, and *location*. *Sports and Games* apps claim to collect the highest number of data types and categories. Surprisingly, 11% of all DSS forms we analyze *over-report* the data

collection, reporting the collection of all data types (Instagram, details below) or attributing most data types to all (7) or nearly all (6) purposes. In contrast, 13% of the forms *under-report* the data collection, either claiming no data collection or reporting the collection of just one data type (Signal, details in RQ2).

Data sharing and security practices. 37% of the Data Safety Sections we analyze claim that the app does not share any data with third parties, even though their privacy policies indicate otherwise. Western Union¹ claims that data is not collected but is shared. SkyMap² claims that no data is collected or shared, but it can be encrypted. However, the same app claims that users cannot request data to be deleted. We categorize these cases as instances that *inconsistently report* data collection and sharing.

RQ1 summary: Many apps appear to inconsistently report data collection and sharing via their Data Safety Sections. This is consistent with a large-scale study conducted by Khandelwal et al. [Kha+23].

4.1.2 RQ2. Comparison between Actual and Reported Data Collection

To answer RQ2, we analyze the app source code and compare its data collection with the Data Safety Section. For this experiment, we compare the results of PRICE with the Data Safety Sections of 20 apps across all domains. This study focuses on the 20 apps we previously identified as over-reporting and under-reporting data collection (**RQ1**). In addition to detecting privacy-related data in the source code and user interfaces, we statically extract the permissions declared in the app's *AndroidManifest.xml* file. Given that the Android developer documentation references app permissions as a means to complete the DSS form [22d], we use permissions as another means to examine how effectively these apps report data collection.

Since the data labels detected by PRICE are similar to the data categories in the DSS form, our experiment allows us to examine 10 distinct data categories out of the 14 present in the Data Safety Section. Our results, illustrated in Table 4.1, validate that **70%** of apps reporting *device or other IDs* do indeed collect such data, making it the most consistently reported category, closely followed by *personal information* (**65%** apps correctly report it). This is encouraging, since these two categories comprise personal data (risks 1 and 2). In contrast, **50%** of apps reporting *health and fitness data* and *calendar data* do not use the correct manifest permissions, making these

¹<https://www.westernunion.com/>

²<https://play.google.com/store/apps/details?id=com.google.android.stardroid/>

Table 4.1.: Comparison between our results and the Data Safety Sections of 20 popular apps. Highlighted apps are discussed in detail.

	Categories in the Data Safety Section									
	Device or other IDs	Personal info	Audio	Contacts	Location	Photos and videos	Financial info	Messages	Health and fitness	Calendar
★: collected, ○: reported, ⊛: collected and reported										
Signal	★	⊛	★	★	★	★		★		★
SkyMap	★	★			★					
Western Union	★	★	★	★	★	★				
Yazio	⊛	⊛			★	⊛			★	
Temu	⊛	⊛			★	⊛	⊛			
DuckDuckGo	★	★	★		★	★				
Covpass	★	★			★					
Trust	⊛	⊛			★		★	○		
Paytm	★	⊛	★	⊛	★	★	★			★
Instagram	⊛	⊛	⊛	⊛	⊛	⊛	⊛	○	○	○
Gmail	⊛	⊛	⊛	⊛	⊛	⊛	○	⊛		⊛
Discord	⊛	○	⊛	⊛	⊛	⊛		○		
Youtube	⊛	⊛	⊛	○	⊛	⊛	⊛	⊛	★	
Flo	⊛	⊛			⊛	○	⊛	○	○	
Candy Crush Saga	⊛	⊛			⊛	★	○	○		
Babbel	⊛	○	⊛		⊛	★	○			
Google Maps	⊛	⊛	⊛	⊛	⊛	⊛		⊛	★	
Netflix	⊛	○	⊛		⊛	★				
Fitbit	⊛	⊛	★	⊛	⊛	★	⊛	⊛	⊛	★
Amazon	⊛	⊛	⊛	★	⊛	⊛	○	○	⊛	

the most inconsistently reported data categories. We observe that all apps which under-report data collection miss reporting *location* through the Data Safety Section. This is concerning since this category encompasses risk 2 personal data. We now discuss two apps in detail:

We observe that Signal only reports the collection of phone number (personal information) in their Data Safety Section. In contrast, our experiment reveals that the app's user interfaces also collect location data (country name) and access data such as username, password, PINs. Furthermore, the app's source code uses system APIs that collect device IDs such as IP address and MAC address, personal information borrowed from Google such as client ID, email address, user ID, name, profile photo. It also collects audio, location, and browsing data. Our findings indicate that Signal collects device data such as Subscriber Identity Module (SIM) card information, but the DSS form does not have a separate category to report such data.

We observe that Signal declares 70 permissions in its manifest file. A careful examination of these permissions reveals that their Data Safety Section should include the collection of audio, contacts, location, messages, and calendar data. This confirms our hypothesis that *Signal under-reports data collection in their DSS form*.

In contrast to Signal, Instagram [26g] reports the collection of **all** data categories in their DSS form. However, our results reveal that Instagram’s user interfaces only collect financial information, specifically card number and CVV. The source code uses system APIs which collect device IDs such as IP address and MAC address, location, and personal information. Instagram also collects device data such as mobile network information, but the DSS form does not have a separate category to report such data.

We extract the 40 permissions that Instagram declares in its manifest file. A careful examination of these permissions reveals that their Data Safety Section should report collecting personal information, audio, contacts, location, and photos and videos. Consequently, the inclusion of other data categories in their existing form such as calendar, health data appears redundant, thus confirming our hypothesis that *Instagram over-reports data collection in their Data Safety Section*.

RQ2 summary: Our experiment reveals evidence of over- and under-reporting by Android app developers. We also identify the most inconsistently reported data categories (location, health and fitness, and calendar).

4.1.3 Discussion

Our findings indicate that app developers struggle to accurately report data collection, despite the existence of tool support [Li+24; 22j; CH22].

One major reason for this issue could be the suboptimal design of Google Play Store’s DSS form. While our experiment considers data collected via user interface screens, system API calls in the app code, or manifest permissions as collected data, Google’s definition of *collected data* is abstract and includes many exemptions [22g]. For example, one does not need to disclose the collection of anonymized data, although the effectiveness of common anonymization techniques is disputed in the digital privacy community [19b]. A study by Mozilla [23b] questioned Google’s decision and suggested eliminating the definition of anonymized data from the DSS form. In contrast, pseudonymized data needs to be disclosed, while end-to-end encrypted data does not need reporting. This raises the question of how app developers determine if data has been properly anonymized or pseudonymized. Additionally, data

processed locally on the user's device and not sent off the device does not need to be disclosed either, which seems questionable, as substantial information might still be inferred from such data.

Another reason for under-reporting may be that the DSS form lacks a separate category for data collected by third-party libraries used by the app. This could lead developers to believe they do not need to disclose data collected by these libraries [Kha+23], potentially giving users a false sense of privacy. This third-party collection information might only be available in the privacy policy of the app (if at all), which is often too long and verbose for users to read. Privacy-conscious users might want to know which data is collected by third-party analytics libraries used by the app, and adding this information is crucial towards enhancing transparency.

After examining the data collected by real-world Android apps, we believe that **all** personal data collected by an Android app should be declared in the *data collected* section of the DSS form, irrespective of whether it is pseudonymized or shared with third parties.

The data shared with third parties can then be reported separately in the *data shared* section of the form. Furthermore, the form should be reordered to follow a risk-based reporting of collected data. For example, directly identifiable personal data such as *device or other IDs* should be placed on top, followed by the less risky categories. A comparison of our results with the Data Safety Section reveals that the data categories in the DSS form are not expressive enough to report all the data the system APIs collect from the device. Data categories in the form should be updated after conducting a thorough research of the system APIs that Android apps use to collect data. We make the Identifier Datasets open source³ and public so that they can be used to enhance transparency and clarity in the data reporting documents. We further suggest the inclusion of the following new data categories:

1. **Device data:** Android APIs and other libraries collect some data from the device. This data can be partially identifiable and hence is different from the identifiable category of *Device or other IDs*. We suggest including a new data type for such data. Examples of such data are SIM card information, bluetooth information, and mobile network information.
2. **Session data:** Media players, advertising platforms and analytics libraries collect session data to analyse user behavior, which does not have a distinct data type in the DSS form.

³Available at <https://zenodo.org/records/14046829>

3. Sub-types in the data category Device or other IDs: We observe *Device or other IDs* to be a very coarse representation, and recommend distinguishing these IDs by their purposes. For example, IP address may be used for gaining access to location and installation ID may be used for advertising and analytics.

Khandelwal et al.'s study [Kha+23] noted a lack of educational resources and demos to aid app developers in completing the Data Safety Section. Our findings indicate that app developers may be unaware of Google's guidelines for Android developers [22d]. These guidelines include a list of manifest permissions and system APIs that should directly correspond to specific data safety labels. For example, Signal (discussed in Section 4.1.2) declares **70** permissions, many of which should directly translate to data safety labels. Nevertheless, Signal has the most under-reported data collection among the 20 apps discussed in Table 4.1.

Furthermore, we observe that Google's guidelines for Android developers [22d] and Google's data collection exemptions for the Data Safety Section [22g] can be complementary, potentially causing confusion for app developers. According to the Android developers' guide, Signal's usage of permissions should translate to data safety labels. However, if the data collected via those permissions is encrypted in the source code, app developers might refer to Google's data safety exemptions and choose to not report it as part of the Data Safety Section. Our observations highlight the need for standardized guidelines and rules before introducing more tools to automate the reporting process.

Our experiments also show inconsistencies in reporting to Google Play Store's Data Safety Section. Location, health and fitness, and calendar data are among the most inconsistently reported categories. Our previous domain-specific analysis (cf. Section 3.4.2) further indicates that personal information, device or other IDs, and location are frequently collected, yet often under-reported, especially in E-Commerce and Shopping apps, where location data is commonly collected but *rarely disclosed accurately*. These results point to a gap between actual app behavior and DSS reporting, emphasizing the need for greater consistency in privacy-aware app development and disclosure practices.

Takeaway. App developers struggle to accurately report data collection, either due to the suboptimal design of Google Play Store's DSS form, lack of standardized guidelines, or insufficient existing tool support.

4.1.4 Limitations and Threats to Validity

While the number of apps in our experiment might appear limited, we ensure a comprehensive evaluation by including the most popular apps from different domains actively used by people. However, selecting top apps per domain could threaten external validity, so our findings may not be generalizable to other domains.

A threat to internal validity arises from the app selection process. Since the experiment focuses on the 70 most popular apps, the results depend on the ranking or popularity criteria used at the time of collection. Different popularity metrics, app store rankings, geographic regions, or collection dates could lead to a different set of applications and potentially different results.

Finally, the experiment was conducted in November 2023. Since then, some Android app versions and their Data Safety Sections may have been updated. Consequently, the results reflect the specific app versions and Data Safety disclosures analyzed in this experiment, and a replication using newer versions may produce different results.

4.2 Challenges Faced by Developers: An Empirical Study

In this section, we report on an empirical study conducted with the goal to *explore how developers classify the privacy-related data their apps collect into DSS data categories, how confident they are when completing the DSS forms, and the challenges they encounter along the way*. By achieving this goal, we aim to identify opportunities to design improved tools, documentation, and processes that better support developers in producing accurate privacy disclosures.

4.2.1 Study Methodology

We conducted an empirical study to achieve our goal. The study was designed to answer the following research questions:

RQ1. *What methods and resources do developers use to categorize privacy-related data collected by their app into the DSS data categories?*

This question focuses on understanding how developers currently approach data classification during the DSS completion process, and whether they use any tools or resources for this purpose. By examining these practices, we aim to identify areas where tooling or official documentation can be improved to better support developers in this task.

RQ2. *How confident are developers in their ability to accurately and effectively complete the DSS forms?*

This question seeks to assess developers' confidence in their ability to complete the DSS form accurately and in compliance with privacy requirements. Understanding developers' confidence levels can reveal points of uncertainty or ambiguity in the reporting process, helping guide the design of more effective support tools and clearer guidance.

RQ3. *What are the challenges developers face when completing the DSS forms?*

This question focuses on the challenges developers face to complete the DSS forms. By identifying these challenges, we aim to uncover opportunities to improve developer support mechanisms and enhance the overall transparency and accuracy of privacy reporting practices.

Figure 4.1 provides an overview of the study design. Our approach combines two complementary data sources: (i) a survey of Android developers capturing their self-reported practices and perceptions, and (ii) a qualitative analysis of online developer discussions that reveal naturally occurring challenges in community settings. The survey directly addresses RQ1 and RQ2, while both data sources collectively answer RQ3 through thematic analysis.

Developer Survey

To directly capture Android developers' experiences with completing the DSS form, we conducted an online survey. Developer surveys are commonly used in software engineering research to generate discussions of subjects where individual experiences are critical [SN23; Li+22; SSM25; Fra+24b; Per+24].

Survey Questionnaire. We designed the questionnaire with a focus on the research questions. These items explored how developers identify privacy-related data collected by their apps (**RQ1**), how confident they are in completing the DSS form (**RQ2**), and the obstacles they face during this process (**RQ3**). The survey included 5-point Likert-type items, multiple-choice questions, and open-ended prompts. A compact version of the questionnaire is presented in Table 4.2. Full questionnaire is available in Appendix A.

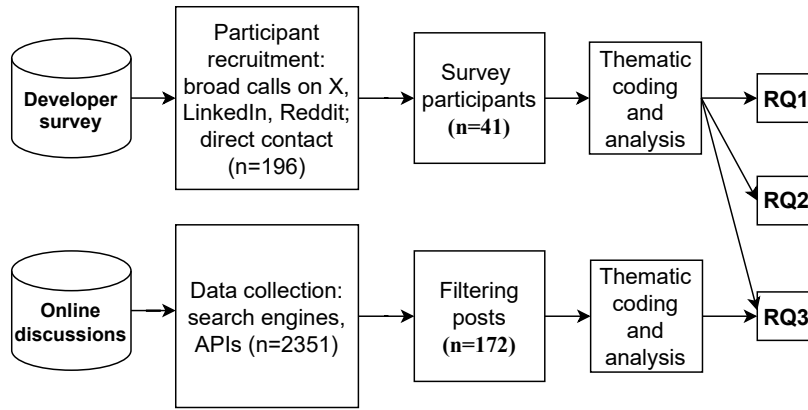


Figure 4.1.: Our empirical study design.

Participant Recruitment. We distributed our survey in three rounds between August and December 2024. In the first round, we made broad calls for participation on X, LinkedIn, and Reddit. We received 19 responses, with only 4 respondents having apps published in the European Union. After a few months, we undertook the second round, during which the author of this thesis directly contacted over 150 randomly selected Android developers via LinkedIn. We received 15 responses, i.e., obtained a response rate of 10%. In the third round, we contacted 46 additional app developers who were engaged in discussions about Android development, Google Play, and the DSS on GitHub and Reddit. We received 7 responses (15% response rate). In total, we obtained data from 41 participants.

Survey Participants. Our participants have a median of 5 years of Android app development experience (average = 5.54). 37 participants (90%) confirmed that they have previously completed the DSS form, and 23 participants (56%) have published app(s) in the European Union. We share detailed demographics in Table 4.3. We refer to our survey participants as P01–P41 (cf. Section 4.2.2).

Thematic Coding and Analysis. We used an iterative open coding approach [CS90] to perform thematic analysis [CB14] of open-ended survey responses. The author of this thesis carried out inductive coding to derive emergent themes, which were then compiled into a codebook. A second researcher independently reviewed the annotations, and disagreements were discussed and resolved through refinement of the final coding scheme. During this discussion, we refined 5 subthemes, resulting in re-alignment of 1 code (out of 24 codes).

Table 4.2.: Mapping between research questions and survey questions.

RQ	Question	Question type
RQ1: methods & resources	How do you categorize the privacy-related data collected by your app for the DSS?	Multiple-choice, open-ended
	What resources do you use to help you understand and complete the DSS?	
RQ2: confidence levels	How well do you understand the privacy-related data your app collects (e.g., location, personal, financial) that must be disclosed in the DSS?	5-point Likert scale
	How familiar are you with GDPR?	
	How confident are you that your app's data collection in the DSS aligns with GDPR requirements?	
	How well do you understand Google's definitions and categorizations of data types within the DSS?	
	How challenging is it to fill out the Data Collection part of the DSS?	
RQ3: challenges	Which aspects of the DSS do you find challenging?	Multiple-choice, open-ended
	What specific challenges have you faced while filling out the DSS?	
	Describe any technical or general issues you have encountered during the submission of the DSS.	Open-ended
	Have you ever had an app rejected due to issues with the DSS? If yes, please describe the situation.	

Table 4.3.: Survey participants' demographics (based on optional responses).

Experience (years)	No experience	4 (9.75%)
	Less than 1 year	5 (12.19%)
	1-3 years	8 (19.51%)
	4-6 years	5 (12.19%)
	7-10 years	10 (24.39%)
	More than 10 years	7 (17.07%)
Published app in EU (*)	Yes	23 (56%)
	No	18 (43.9%)

Analysis of Developer Discussions

Our analysis of the survey data revealed that many participants rely on online forums and developer communities as key resources when completing the DSS form (see Section 4.2.2). To capture a broader range of perspectives and enable data triangulation [RH09], we therefore complemented the survey with a qualitative analysis of online developer discussions.

Data Collection. We used literature [Li+21a; Höl+24; Iqb+21; Ant+23; TVS20; Li+21b; SSM25] to systematically identify relevant public forums and developer communities where Android privacy and data reporting topics are actively discussed. We also conducted pilot searches to refine our choice of sources and search terms. Our search focused on popular communities such as r/android (3.1M members), r/androidapps (436K), and r/androiddev (261K) on Reddit, as well as r/AndroidDev (34.5K) on Discord.

Table 4.4.: Summary of developer discussions collected across platforms. “Raw posts” refers to all extracted discussions (may include duplicates and irrelevant posts), and “Final posts” refers to those retained after filtering.

Online Platform	Extraction Source(s)	Raw posts	Final posts	Unique Developers (Final)
Stack Overflow	API + Google	1505	56	268
Reddit	Google	235	65	283
Discord	Local search	29	28	31
GitHub	GitHub + Google	572	22	48
Hacker News	Google	10	1	12
Total	—	2351	172	642

To capture posts relevant to the DSS form, we experimented with several keyword combinations and finalized the following query: (*DSS OR “data safety section” OR “data safety form”*). Using this query, we collected posts from multiple developer platforms (cf. Table 4.4). We leveraged both Google and local search engines, and used the Stack Exchange API [25t] to automatically extract posts tagged with relevant keywords. Data collection took place between January and September 2025, resulting in 2,351 discussions in total. These included potential duplicates and irrelevant discussions. Further details on data collection including search queries and Python scripts using developer community APIs are provided in the artifacts.

Filtering Posts. We applied a two-stage filtering process to ensure that only discussions authored by app developers and those directly related to the DSS were retained. Posts were included if they explicitly discussed experiences with DSS completion, privacy reporting, or app rejections related to the DSS or privacy policies. Posts were excluded if they were unrelated to the DSS reporting process (e.g., news articles or general privacy discussions) or authored by non-developers (e.g., lawyers or users commenting on app privacy). For Stack Overflow, we used the Stack Exchange API to further filter out posts discussing unrelated platforms (e.g., iOS or Apple labels). After filtering, our final dataset comprised 56 Stack Overflow posts, 65 Reddit threads, 28 Discord discussions, 22 GitHub issues and discussions, 1 post from Hacker News [25k], all posted after the DSS was introduced in 2022.

Measuring Engagement. To measure user engagement, we used Stack Exchange API [25t], Reddit API [25p], and GitHub API [25i] to automatically count unique developers interacting with the posts. For Discord, participant counts were obtained manually, while for Hacker News, we used a custom Python script to identify unique commenters. Altogether, our dataset comprised 172 posts, involving 642 distinct developers (cf. Table 4.4).

Thematic Coding and Analysis. We followed the same process of qualitative analysis as the developer survey.

4.2.2 Study Results

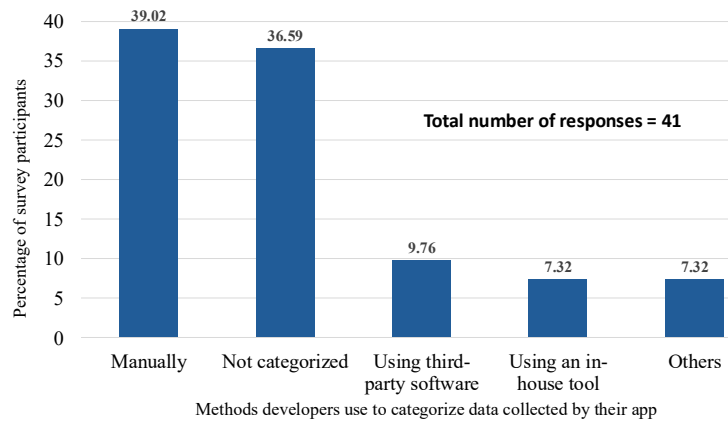
We discuss the results of the developer survey in the context of all three research questions, and analysis of developer discussions in the context of the third research question.

RQ1: Methods and Resources

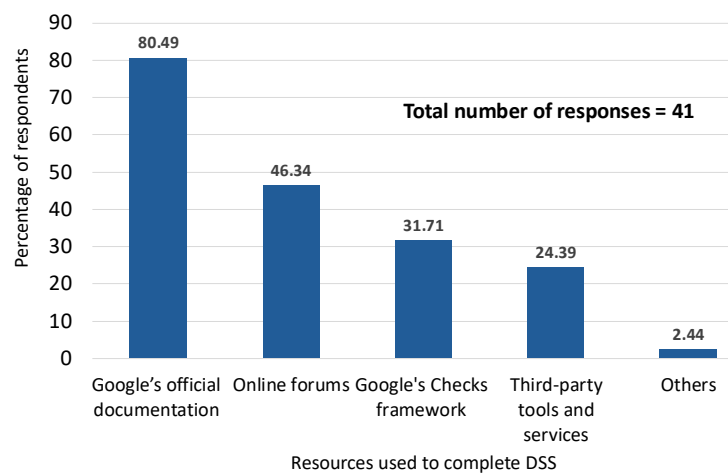
The DSS form requires developers to disclose collected information under broad categories (e.g., location, personal information, financial information) that are further divided into specific data types (cf. Table 2.2). We asked our survey participants what methods they use to classify the privacy-related data collected by their apps in accordance with these categories. The results are shown in Figure 4.2a. **39.02%** participants reported classifying data manually. P03 explained, *“The data is categorized manually to ensure accuracy and compliance with Google Play’s guidelines, allowing for thorough review and customization based on the app’s specific data collection practices.”*

However, more than a third of participants (**36.59%**) indicated that they do not perform any data categorization. In practice, this could mean that they select all data categories as collected, select none, or just guess which categories apply for their app. Each of these behaviors could result in inaccurate or incomplete disclosure, giving end users a false sense of privacy about how their data is handled. Several participants attributed this to the perception that their apps did not collect personal data. For instance, P22 said, *“I did not need to store any customer related data.”* P24 said, *“(I am) not collecting any data.”* P20 added a pragmatic perspective: *“I try to avoid doing that (completing the DSS form) as much as possible. Usually just by not collecting any data. It makes the user experience worse but affordable.”* Others, such as P16, emphasized the simplicity of their applications: *“My app is simple, so no need to care about this (DSS form).”*

Only **9.76%** reported relying on third-party solutions for categorization, highlighting the limited availability of automated, context-aware approaches. P41 noted, *“Many developers lack tools to automatically track and audit data usage, making it challenging to identify all collected or shared data.”* A smaller group (**7.32%**) reported using in-house tools and other approaches, such as permissions, to support categorization. P20 explained, *“App permissions (runtime or manifest-based) provide the most accurate info.”*



(a) Bar chart illustrating what methods survey participants use to categorize data collected by their apps.



(b) Bar chart showing the percentage of participants who used resources to complete the DSS form. Participants could choose multiple options for this question.

Figure 4.2.: Methods and resources participants use to categorize privacy-related data collected by their app into the DSS data categories (RQ1).

We next inquired about all the resources participants rely on to complete the DSS form (cf. Figure 4.2b). Most participants rely on Google's official documentation (80.49%), followed by online forums and communities (46.34%). Other reported resources include Google's Checks framework (31.71%) and third-party services (24.39%). P39 used *Privado* [22j] to complete the DSS form.

RQ1 summary: Developers use existing online resources to complete the DSS form. Most either manually categorize the collected data or omit categorization altogether.

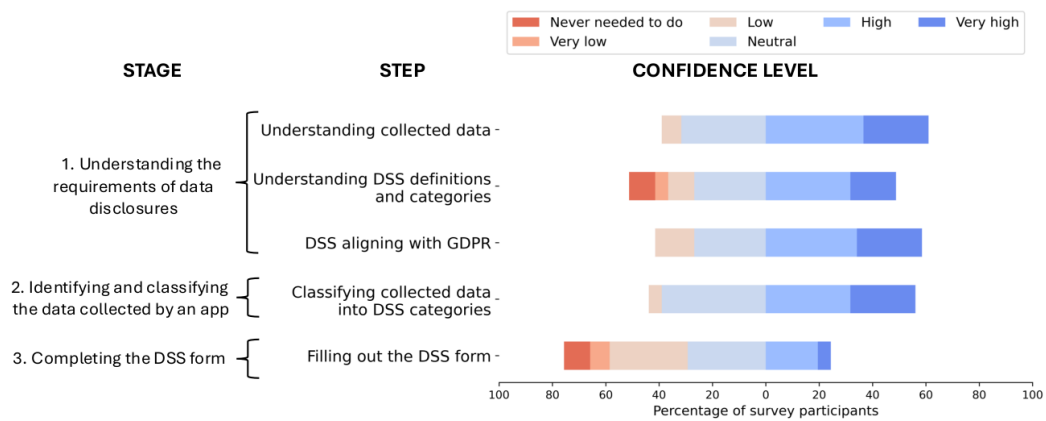


Figure 4.3.: Survey participants’ responses to the questions about confidence levels for different processes (RQ2).

RQ2: Confidence Levels

To investigate this question, we analyzed the survey responses.

We sought to identify *where* in the process of privacy-aware reporting developers encounter the most difficulty. We conceptualize this process as consisting of three stages:

1. understanding the requirements of data disclosures,
2. identifying and classifying the data collected by an app, and
3. completing the DSS form.

We asked survey participants to rate their confidence in these stages. The results are shown in Figure 4.3. While many developers reported confidence in understanding and classifying privacy-related data (stages 1 and 2), their confidence dropped noticeably when it came to accurately completing the DSS form (stage 3).

For instance, P03 noted confidence in identifying and categorizing data but highlighted the difficulties of nuanced distinctions: *“I have a good understanding of Google’s definitions and categorizations of data types within the DSS. However, some nuances, such as differentiating between data types that seem similar or ensuring compliance with the latest updates to the guidelines, can occasionally be confusing and require further clarification or consultation with relevant documentation.”* They further explained: *“Challenges may arise when distinguishing between data categories like precise versus approximate location, especially when integrating third-party services or handling edge cases. Regular reviews and consultations with documentation help mitigate these challenges.”*

Other participants expressed more uncertainty. P27, for example, admitted: “*When I use third-party libraries, I don’t have full control over what’s collected.*” Similarly, P09 observed broader inconsistencies across contexts: “*Country-wise app data collection rules are different. And there are tons of apps which don’t comply with these rules and still available on Play Store.*”

Overall, these perspectives reveal a clear pattern: developers are generally confident in recognizing the data their apps collect, yet they face challenges in translating this knowledge into DSS-compliant disclosures. Even with familiarity with Google’s data type definitions [22f], uncertainty arises when handling nuanced categories, third-party services, or varying regulatory expectations.

RQ2 summary: Developers are generally confident in identifying the data their apps collect, yet their confidence drops significantly when translating this knowledge into DSS-compliant disclosures.

RQ3: Challenges

To investigate the *specific* challenges developers face when completing the DSS form, we drew upon two complementary sources: survey answers and online developer discussions. For the survey, we analyzed responses to all questions related to RQ3 (cf. Table 4.2), which provided qualitative insights into developers’ experiences. These included questions where participants ranked predefined challenges identified from existing literature [Kha+23; KMB24], and optionally added their own challenges under an “Others” field or in other open-ended questions. We coded the survey responses to determine how many developers encountered each challenge. The resulting categorization is summarized in Figure 4.4.

We further examined online discussions from Stack Overflow, Reddit, Discord, GitHub, and Hacker News (posted after the DSS was introduced in 2022). The overall distribution of challenges identified across these online discussions is shown in Table 4.5 and Table 4.6. We categorized these discussions at two levels of granularity. At the first level, we grouped these discussions according to the high-level challenge they addressed (e.g., *understanding and complying with DSS form*). At the second level, we further subdivided each challenge into specific subtopics that captured recurring themes within the discussions (e.g., within *understanding and complying with DSS form*, discussions were split into *confusing terminologies*, *data deletion*, *encryption declaration*, and *legal compliance*).

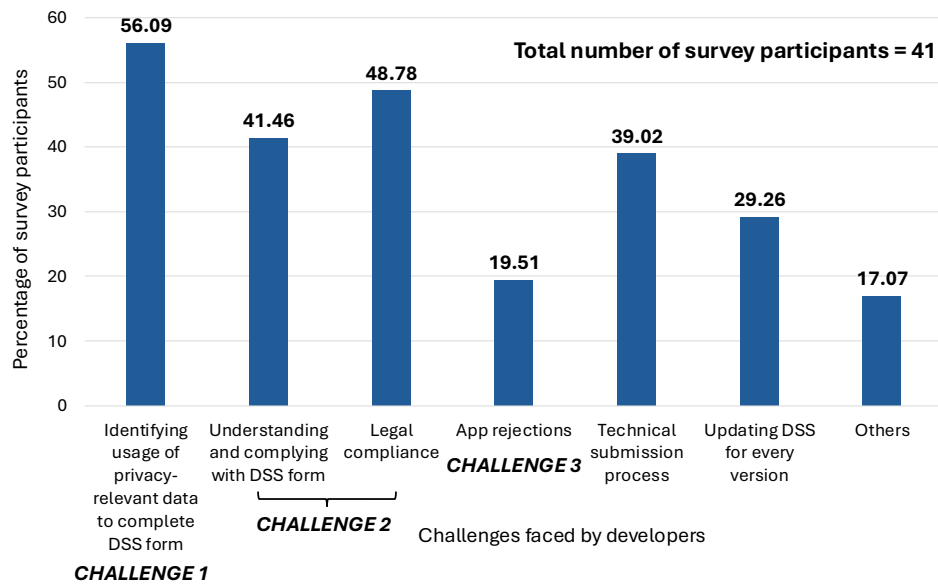


Figure 4.4.: Bar chart showing the distribution of challenges across survey participants (RQ3). Participants could choose multiple challenges through various questions.

While we applied a similar categorization process to the survey data, the online discussions provided a more comprehensive taxonomy of challenges due to their greater volume, diversity of perspectives, and depth of qualitative detail. This hierarchical analysis enabled us to identify not only the major categories of developer difficulties but also nuanced variations within each challenge.

The following provides a detailed description of the recurring challenges observed:

Identifying usage of privacy-related data to complete DSS form. Developers are required to populate the *data collected* or *data shared* sections of the DSS form (① and ② in Figure 2.5a). Many reported uncertainty regarding which types of data in their apps should be considered privacy-related and disclosed in these sections. Specifically, **56.09%** of survey respondents (cf. Figure 4.4) and **61.62%** of online discussions (106 out of 172) focused on this issue.

a) Third-Party Library (TPL) data collection. One of the most frequently reported issues was the lack of clarity and control over data collected by third-party libraries. This concern appeared in 65 online discussions (37.79%) and was mentioned by 7 survey participants (17.07%). Developers frequently sought community guidance on how to report data collected by SDKs integrated into their apps, such as analytics,

Table 4.5.: Distribution of challenges across online developer discussions (RQ3). At the first level, discussions are grouped according to the high-level challenge they addressed. Each challenge is further subdivided into specific subtopics that captured recurring themes. Each online discussion is mapped to a single subtopic. Abbreviation: TPL = third-party library.

Data Sources	Challenge 1: Identifying usage of privacy-related data to complete DSS			Challenge 2: Understanding and complying with DSS			
	a) TPL collection	b) Data collection	c) Data sharing	a) Confusing terminologies	b) Data deletion	c) Encryption declaration	d) Legal compliance
Stack Overflow	15	21	0	1	0	4	0
Reddit	28	11	3	10	9	1	0
Discord	11	8	0	4	1	0	0
GitHub	11	1	3	2	1	0	0
Hacker News	0	0	0	0	0	0	0
Total (n=172)	65	41	6	17	11	5	0
Percentage (%)	37.79	23.83	3.48	9.88	6.39	2.90	0.00

Table 4.6.: Distribution of challenges across online developer discussions (continued).

Data Sources	Challenge 3: Understanding app rejections	Other challenges
Stack Overflow	14	1
Reddit	3	0
Discord	4	0
GitHub	2	2
Hacker News	1	0
Total (n=172)	24	3
Percentage (%)	13.95	1.74

login, or advertising libraries⁴. The most frequently cited third-party libraries were Admob [22a] (21 mentions), Firebase [25h] (19), Unity [22l] (6), Google services (Maps, Billing, Play Services; 3 each), and AppLovin [22b] (3).

Survey participants emphasized the difficulty of identifying data collected by third-party code and distinguishing it from data collected directly by the app. As P39 explained, “When third-party libraries are involved, we would need to declare data collected by those libraries too. In the current situation, it appears as if the app itself collects that information. The reality is a bit different. It might be better if there is some way to clearly communicate this distinction.” P33 and P40 highlighted the challenges of determining what data is collected through external SDKs and the lack of accurate third-party documentation. Some developers, such as P35, recommended using only SDKs whose behavior could be fully verified, ideally by reviewing source code. Such issues also resulted in app rejections. For instance, P27 recalled, “I developed an app that uses Google Maps. When I uploaded a release, it was rejected for not declaring that my app uses a library (Google Maps) that collects user location.”

⁴An example (TPL data collection): https://www.reddit.com/r/androiddev/comments/s7mk24/safety_data_admob/

b) Data collection. Challenges were not limited to data collected by third-party libraries. 41 online discussions (23.83%) and 7 survey participants (17.07%) were unsure if a certain data item needs to be disclosed in the data collected part of the DSS form.

13 Stack Overflow posts⁵ discussed the following Play Store error messages: *we detected user data transmitted off device that you have not disclosed in your app's Data safety form as user data collected*. Other discussions revealed confusion over what qualifies as data collection—such as metadata embedded in images (e.g., location), crash reports, Application Not Responding (ANR) reports, SDK activity, or login forms—and whether it needs to be disclosed. Some others struggled to identify the collected data items requiring mandatory disclosure via the DSS form.

Among the survey responses, participants reported difficulties when apps acted primarily as frontends for remote services. As P39 explained, *“The manual work required is a bit challenging. I’m always a bit skeptical as to whether I’ve accurately captured the data safety details in Play store. One specific challenge that comes to my mind is when a remote server is involved in the mix. Our app primarily acts a frontend to a remote service. It is unclear whether we need to point out the data collected by the service in Play Store or not. This is not clear as the user has to sign up separately for the service and would’ve already agreed to their terms and policies. So, it is not clear whether we need to reiterate this.”*

c) Data sharing. 6 online discussions (3.48%) focused on understanding if certain collected data needs to be disclosed as shared. These discussions highlighted confusion about the data sharing practices of Google SDKs, Firebase, AdMob, and notifee [25n]. No survey respondents mentioned this issue.

Understanding and complying with DSS form. Developers expressed frustration in understanding the DSS form itself. **58.53%** of survey participants (cf. Figure 4.4) and **19.18%** of discussions marked understanding and complying with DSS as a major challenge.

a) Confusing terminologies. 16 online discussions (9.88%) and 8 survey participants (19.51%) highlighted confusion in understanding the DSS terminologies and the structure of the form.

⁵An example (data collection error): <https://stackoverflow.com/questions/71199143/>

A user from an online discussion said, *“There are so many questions, and I have no idea what to answer.”* Some developers struggled to interpret terminologies such as ephemeral data processing, and understand the differences between the definitions of data collection and sharing. Some others expressed confusion on what is considered a required data type⁶, and if accounts created by third-party SDK also need to be disclosed under data collection and sharing.

Similar sentiments appeared in the survey answers. P20 admitted, *“To be honest, I have no idea about any of the DSS data types, this is a mess. (I don’t know) how to write it and not spend years on that, especially, to meet some uncertain requirements from Google (aka gray zone rules).”* P34 noted that the wording of terms was unclear, and P27 emphasized the burden of navigating numerous categories and subcategories to avoid violations. P12 found the DSS form lengthy. P37 complained about excessive buttons and questions, explaining, *“I had to create a bot to automate the process for my apps.”* P31 described the effort of carefully parsing each question as a challenge in itself.

b) Data deletion. 11 discussions (6.39%) addressed challenges related to Google Play’s data deletion requirements (③ in Figure 2.5a). Developers described confusion around implementing deletion mechanisms in apps that did not collect personal data or maintain user accounts. Others reported difficulties generating a data deletion URL when data was only collected through third-party SDKs. No survey respondents mentioned this issue.

c) Encryption declaration. Only five online discussions (2.9%) focused on issues related to declaring data encryption in the DSS form (③ in Figure 2.5a). These developers reported receiving error messages from Google such as: *You have declared that user data is encrypted in transit in your app’s Data Safety form and we’ve detected unencrypted network traffic that may carry user data off device.* In most cases, developers attributed these errors to third-party library dependencies but were unable to identify which specific dependency triggered the violation. No such challenges were reported in the survey responses.

d) Legal compliance. 20 survey participants (48.78%) found it challenging to comply with legal regulations and Google’s policies. P13 observed *compliance with multiple regulations* to be a major challenge. P34 noted, *“There are several relevant Google*

⁶An example (confusing terminologies): https://www.reddit.com/r/androiddev/comments/rf5b8u/data_safety_required_and_optional_data_collection/

Play policies, and these policies can change, difficult to check all the policy details”. No online discussions addressed this issue, which was unsurprising given that such discussions typically focus on seeking developer feedback.

Understanding app rejections. Several challenges identified in our analysis were directly linked to app rejections from the Google Play Store. While rejections related to specific issues—such as identifying data usage and complying with the DSS—are discussed in their respective sections, here we focus on other app rejection experiences reported by 24 online discussions (13.95%) and 8 survey participants (19.51%).

Across online discussions, the most frequently cited reason for app rejections was a mismatch between the information provided in the DSS and the app’s privacy policy⁷. Even developers who used automated tools such as Privado [22j] reported repeated rejections due to difficulties in accurately reporting data practices.

Among the survey responses, P03 said, *“My app was rejected due to issues with the DSS because the submitted information was either incomplete or inaccurately described. This included miscategorizing data types, not fully aligning with Google Play’s privacy guidelines, or discrepancies between the disclosed data practices and the actual app functionality. Additionally, technical errors or non-compliance with regulations like GDPR might have contributed to the rejection.”* P24 recalled, *“(I was) getting threatened with having my app removed for not providing a data removal procedure, even though there is no data to be removed.”* P34 added, *“A feature that we added a long time ago to our app (in 2016) and that we forgot to mention in the DSS (caused the rejection).”*

Other challenges. Beyond these challenges, 16 survey participants (39.02%) identified issues with technical submission processes to be a major challenge, while 12 participants (29.26%) emphasized difficulties in updating the DSS for each new app version. In addition, 1.74% of online discussions and 17.07% of survey participants discussed other challenges such as lack of tool support and missing “security review” within the DSS for their apps.

RQ3 summary: Developers face many challenges when completing the DSS forms, including issues in identifying privacy-related data to complete the form, limited understanding of the form, and concerns about app rejection due to discrepancies with Google’s privacy requirements.

⁷An example (app rejection): <https://stackoverflow.com/questions/64929059/>

4.2.3 Discussion

In this section, we discuss our findings across four themes: (i) current practices of completing the DSS form, (ii) challenges in reporting data collection, (iii) the need to clarify the DSS form’s requirements, and (iv) the role and limitations of existing tools.

Current Practices. Developers mostly rely on online guidance to complete the DSS form, either manually classifying data or omitting classification altogether (**RQ1 summary**). In practice, this often means selecting all data categories as collected, none at all, or guessing which ones apply. This risks misreporting, leading to inaccurate disclosures and giving users a false sense of privacy.

Several developers reported *avoiding data collection altogether* to bypass the DSS process. As P20 explained, “*I try to avoid doing that [completing the DSS form] as much as possible. Usually just by not collecting any data. It makes the user experience worse but affordable.*” This reflects a pragmatic yet concerning trade-off, where functionality is reduced to escape compliance overhead.

While developers generally feel confident in identifying the data their apps collect (**RQ2 summary**), their confidence drops when translating this knowledge into DSS-compliant disclosures. This gap highlights that the challenge lies not in data awareness, but in interpreting Google’s disclosure requirements.

In some cases, it remains unclear whether app developers are even the ones filling out the form. As P29 noted, “*The release onus lies on the compliance team,*” while another developer declined survey participation, noting, “*Our compliance team does all law-paper stuff manually for all cases and platforms.*” Delegation to compliance teams indicates a disconnect between the DSS’s intent—holding developers accountable—and its real-world execution.

To mitigate this gap, we recommend joint review processes involving both developers and compliance teams to align technical understanding with formal DSS reporting.

Reporting Data Collection. A key challenge highlighted in our study was reporting *collected data* through the DSS form. Although the developer survey included some questions specifically about data collection (**RQ1**), our analysis of online discussions that examined developer challenges more broadly (**RQ3**) revealed that data collection remained the *most prominent issue*: difficulties in reporting it, and the resulting errors and app rejections, were discussed in 106 out of 172 cases (61.62%).

A key factor behind this challenge is developers' limited insight into the data gathered by *third-party SDKs*. Although some SDK providers offer guidance pages instructing app developers what data types their SDKs use and thus must be declared to the DSS, a recent study [IKS24] has detected inconsistencies between the guidance pages and the actual data collection of SDKs. Similarly, P33 noticed, *"Data collected by third-party SDKs is not listed or explained clearly in their documentation."* This lack of reliable, actionable guidance leaves developers without adequate support.

We recommend tools that detect SDK data collection and provide actionable disclosure suggestions, supported by standardized, machine-readable metadata and clearer Google DSS guidelines.

Clarifying the DSS Form. A major challenge lies in the ambiguity of DSS terminology and reporting requirements. Many developers expressed uncertainty about *what* information to disclose and *why*. Our study identifies several areas needing clearer guidance: (i) vague definitions of ephemeral data processing, (ii) ambiguity in data type categories (cf. Table 2.2) (e.g., classifying video recordings without gallery access), (iii) unclear account creation criteria involving third-party SDKs, (iv) confusion over exemptions and required data types, and (v) difficulty distinguishing data collection from data sharing. We recommend introducing interactive tools that explain each disclosure step with contextual examples and consistent definitions. Clearer terminology and examples would reduce errors and improve confidence.

Existing Tools and their Limitations. Although tools to support DSS completion already exist [CH22; Li+24; 22j], adoption remains limited. Only 18 of our survey participants (43.9%) reported using such tools. Of these, six still categorized collected data manually, and five omitted categorization altogether. Among the 13 who used Google's Checks framework [CH22], five found DSS completion easy, while five described it as challenging. P32, who struggled the most despite using Checks, said, *"Understanding issues about the rejection and solving the problem [is the most challenging]"*. P39, a Privado [22j] user, said, *"The manual work required is a bit challenging. I'm always a bit skeptical as to whether I've accurately captured the data safety details in Play Store."* P40, who used Checks alongside an in-house tool, said, *"Accurately identifying and categorizing all types of user data collected by the app (e.g., location, personal information, device data) can be challenging, especially in apps with third-party SDKs."* A user in one online discussion on Reddit admitted to having tried Privado in their second attempt to make sure their DSS reported all data types their app collects, but still got an "undeclared data collection" rejection message from Google.

Survey responses showed developers’ reluctance to use existing tools. For instance, P40 said, *“Many developers lack tools to automatically track and audit data usage, making it challenging to identify all collected or shared data.”* P20 said, *“I use ChatGPT for that (completing the DSS form) since many things are known only to jurists, who cost a lot, especially for private developers.”*

Community adoption appears similarly low: only 15 discussions (8.72%) referenced third-party tools such as Privado [22j], with three posts authored by Privado’s creators themselves—suggesting limited community uptake of such solutions.

These findings suggest that current DSS-support tools may fall short in terms of accuracy, usability, or accessibility. Future work should systematically evaluate these tools to understand why they are underutilized and how they can be improved to better support app developers.

Improving the DSS ecosystem therefore requires a multifaceted approach: clearer, standardized guidelines from Google; better alignment between SDK documentation and actual behaviors; and practical, developer-centered tools that make disclosure both accurate and accessible. Ultimately, bridging these gaps would not only improve the reliability of DSS disclosures but also strengthen user trust in the privacy information presented on the Play Store.

4.2.4 Limitations and Threats to Validity

Our empirical study centers exclusively on Google’s DSS, and hence its results cannot be generalized to Apple’s labels without further research. Conversely, findings from prior work on Apple’s labels [Li+22] are not directly transferable to our context. Our results can motivate and guide researchers to replicate the study for Apple Store.

The developer survey is susceptible to self-selection and recall biases, which may influence the representativeness and accuracy of responses. The sample size (41 developers) also limits generalizability. To mitigate these risks, we conducted multiple rounds of recruitment and directly contacted 196 developers to ensure diversity across app domains and organization types. Nonetheless, participation was constrained by the nature of the task—several developers declined due to reliance on compliance or app publishing teams for completing the DSS form. To strengthen external validity and compensate for the modest survey size, we supplemented the survey findings with data from online developer discussions.

Our analysis of developer feedback in online communities focuses on developer discussions from Stack Overflow, Reddit, Discord, GitHub, and Hacker News, which may not reflect perspectives from all communities. However, prior research [Höl+24;

Iqb+21; Ant+23; TVS20] supports online community’s value for qualitative insights. To broaden our coverage, we also examined other popular developer forums (Dzone [25g], Sitepoint [25r], Code Project [25b], and XDA forums [25u]) but found no relevant posts. Since the DSS was introduced in 2022 [22e], the volume of related discussions remains limited. To address this, we ensured that our dataset included sufficient engagement (642 unique developers), and we complemented it with survey data to enable triangulation [RH09]. Our data collection concluded in September 2025 and includes the most recent discussions available at the time of analysis. Nonetheless, as DSS adoption continues to evolve, future research should examine emerging discussions to capture potential changes in developer experiences.

To ensure that our analysis accurately represented developers’ experiences with the DSS, we employed an iterative open coding process [CS90] following thematic analysis principles [CB14]. The author of this thesis performed inductive coding to derive emergent themes. To improve coding consistency and reliability, a second researcher independently reviewed the annotations, and disagreements were resolved through discussion, ensuring coder alignment. This systematic refinement process, along with the public availability of the final codebook, enhances the transparency and construct validity of our findings by ensuring that the themes faithfully reflect the underlying data.

Despite these limitations, the convergence between survey findings and online discussion analysis provides confidence in the validity of our results, suggesting that the combined evidence base is sufficient to draw meaningful insights.

4.3 Related Work

We now discuss various aspects of existing related research.

DSS Accuracy. Multiple recent studies have examined the accuracy of DSS submissions [23b; Kha+23; Rod+23]. A Mozilla study [23b] evaluated 40 popular Android apps and found discrepancies between their DSS and privacy policies. Khandelwal et al. [Kha+23] explored how developers engage with the DSS by analyzing snapshots of submitted forms and contacting over 3,500 developers. More recently, Girish et al. [Gir+25] compared the runtime behavior of beacon-enabled apps with their declared safety labels, revealing that many apps collect device identifiers without proper disclosure. While these studies collectively highlight persistent challenges in the disclosure process, they do not specifically investigate *how* developers report

data collection—a stage that has proven particularly error-prone [Gir+25; KMB24]. Our work extends this line of inquiry by not only exploring the challenges developers face in reporting data collection, but also analyzing their confidence levels.

Analyzing Online Discussions. Prior research [Ant+23; TVS20; Li+21a; SSM25] has examined online developer communities such as Stack Overflow and Reddit to study various aspects of software development practices. To the best of our knowledge, our work is the first to use discussions from these online communities to explore the specific challenges Android developers face when completing the DSS form.

DSS-Support Tools. Prior studies evaluating DSS accuracy [Kha+23; 23b] were conducted at a time when DSS-assistive tools were either unavailable or too immature to study in practice, leaving open questions about their potential impact on developers’ disclosure practices. Since then, Google has introduced Checks [CH22], a paid service that assists app developers in completing the DSS, alongside open-source alternatives such as Matcha [Li+24] and Privado.ai [22i]. Matcha analyzes app code and suggests appropriate DSS labels, relying on developer annotations to finalize the disclosures. Privado is a command-line tool that accepts open-source projects as input, and then generates their DSS form. Despite these advancements, little is known about how widely such tools are adopted, whether they effectively address challenges in reporting data collection, or what limitations remain. By examining the adoption of these tools, and analyzing the broader challenges developers face in data reporting, our study provides timely insights into a problem space that has evolved considerably since earlier work.

4.4 Summary

In this chapter, we conducted a case study using PRICE to observe how data collection is reported via the Data Safety Sections of 70 apps, and observed *over-* and *under-reporting* of collected data. Using the results from two Messaging and Social Media apps (Signal and Instagram), we discussed how app developers under-report and over-report data collection, respectively, and identified inaccurately reported data categories. Our findings have shown that app developers struggle to accurately report data collection, either due to Google’s abstract definition of collected data or insufficient existing tool support.

Next, we conducted an *empirical study* to understand Android developers’ experience with the DSS form. Through a survey of 41 Android app developers, we found that developers often manually classify the privacy-related data their apps collect into

the DSS data categories defined by Google—or, in some cases, omit classification entirely—and rely heavily on existing online resources when completing the DSS forms. While developers generally feel confident in identifying the data their apps collect, their confidence drops significantly when translating this knowledge into DSS-compliant disclosures. To broaden our understanding of these challenges, we analyzed 172 online developer discussions, revealing recurring issues such as issues in identifying privacy-related data to complete the form, limited understanding of the form, and concerns about app rejection due to discrepancies with Google’s privacy requirements.

Overall, our findings highlight the importance of more usable tooling and clear guidance to support Android developers in meeting privacy-aware reporting requirements.

Exploring Automation of Privacy Assessments

This chapter presents the third major contribution of this thesis: an exploration of how static program analysis can support the automation of privacy assessments (*Contribution 3*). We address the knowledge gap between the different stakeholders involved in privacy assessments, particularly between technical stakeholders who understand application code and legal or privacy stakeholders who assess compliance requirements. As a solution, we present Assessor View, a web tool based on static program analysis that aims at easing communication between stakeholders, and helping them understand the source code to conduct accurate privacy assessments.

We first describe the design and implementation of Assessor View. Next, we discuss stakeholder evaluations conducted on Assessor View: a user study with 12 participants to explore if Assessor View can aid *Android app developers* in understanding Android data protection, and an interview-based user study conducted with 16 *privacy experts (including Data Protection Officers and legal experts)* to understand the challenges faced by them, and to explore the extent to which Assessor View can aid privacy experts in conducting privacy assessments.

5.1 Assessor View

Assessor View comprises three key components: the user interface (Module [A](#) in Figure 5.1), the Privacy Slice Visualizer (Module [B](#) in Figure 5.1), and the DPV Transformer (Module [C](#) in Figure 5.1).

Initially, the *user interface* [A](#) accepts an Android app as input, and forwards it to the *Privacy Slice Visualizer* [B](#) for further analysis. The Privacy Slice Visualizer statically slices the source code from privacy-related data sources, and visualizes the results. Subsequently, the *DPV Transformer* [C](#) accepts these program slices as input and translates them into graphical Data Privacy Vocabulary (DPV) representations familiar to the different stakeholders involved in a privacy assessment, enhancing their comprehension of the code. Throughout this chapter, we use the term *pri-*

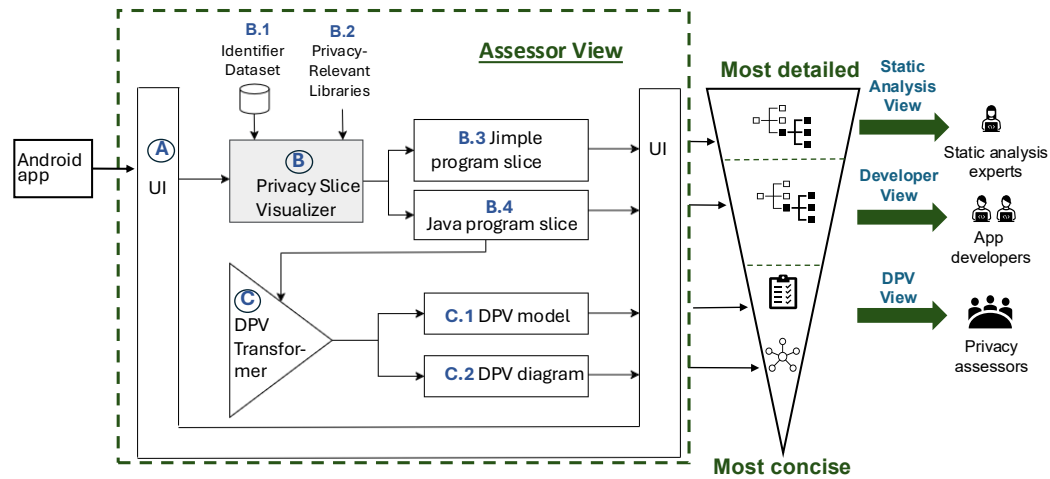


Figure 5.1.: Workflow of Assessor View. Module **B** (shaded with gray) builds on Jicer [PW21], an existing slicer for Android apps.

vacy assessors to collectively refer to DPOs, privacy experts, and legal experts. While some DPOs have legal training, others come from compliance or technical backgrounds; therefore, we distinguish DPOs from legal experts but include both under the broader category of privacy assessors.

A User Interface. The *User Interface* accepts any APK as input (① in Figure 5.2) and feeds it to the Privacy Slice Visualizer. For convenience, users can also click *Try Demo App* to analyze a sample APK provided by the back-end (② in Figure 5.2). This option is included solely to make the system easily accessible to users who may not have an APK readily available. It proved particularly useful during the user studies (cf. Section 5.2.1 and Section 5.2.2), as it eliminated the need for a prior process to provide users with a sample APK.

Once Assessor View has been executed, the UI displays Assessor View’s results on the dashboard (cf. Figure 5.3). The dashboard gives an overview of all risky sources detected in the APK (cf. Figure 5.3a), and the slices that originate from these sources (cf. Figure 5.3b).

B Privacy Slice Visualizer

Privacy Slice Visualizer performs static analysis on APKs and generates dependence graphs. It requires two elements, namely an *Identifier Dataset* (B.1) to match system API calls with privacy labels, and a collection of *privacy-relevant libraries* package names to identify privacy-relevant methods (B.2). The Privacy Slice Visualizer employs both B.1 and B.2 to first construct an Application Dependence Graph (ADG) of

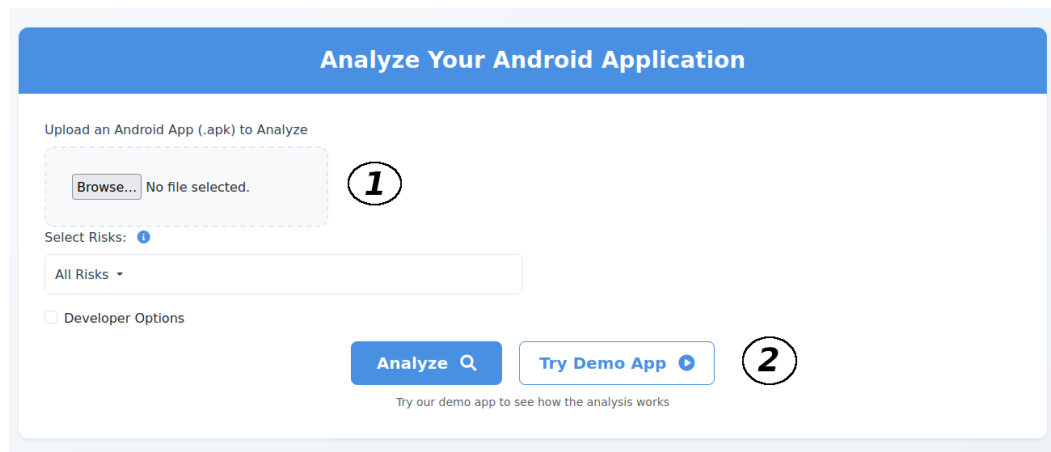


Figure 5.2.: Upload APK page of Assessor View, showing ① input APK, and ② the option to analyze a sample APK provided by the back-end.

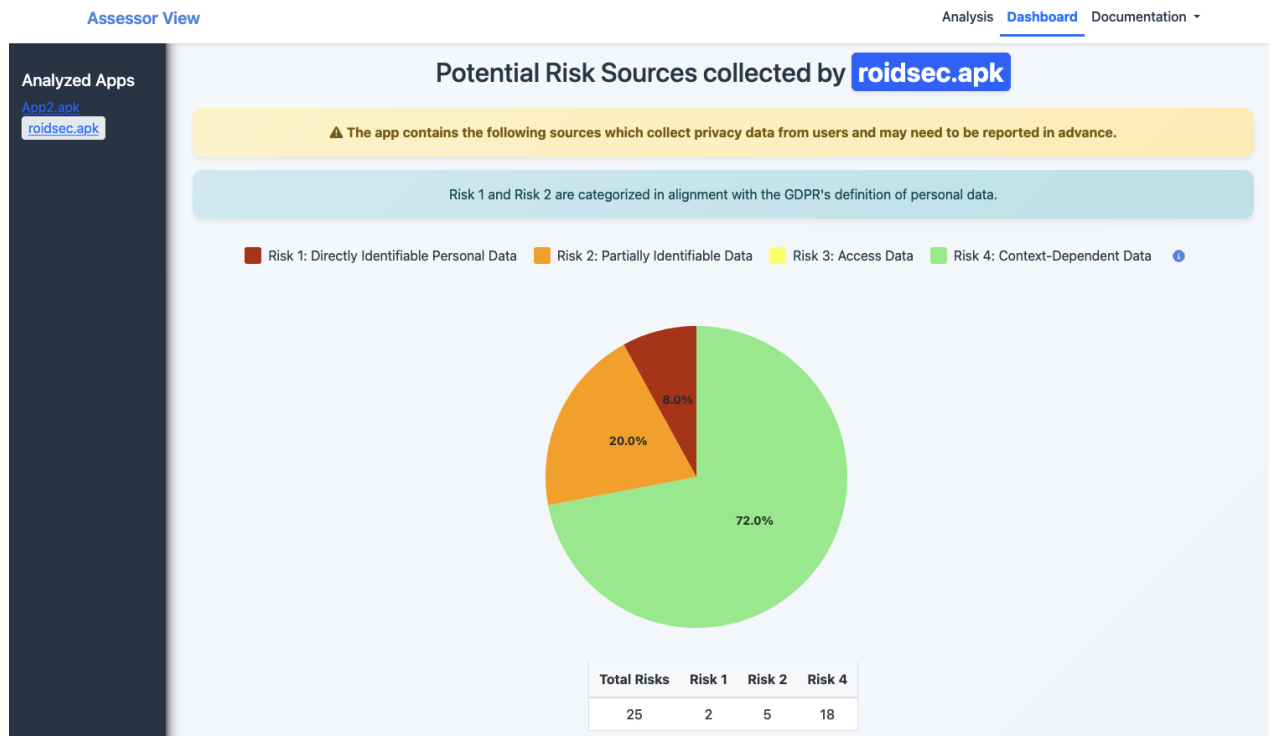
the given app, capturing all control and data dependencies. It then cross-references each system API call in the code against the manually annotated *Identifier Dataset* (B.1).

It extends Jicer [PW21], a static program slicer that operates on Jimple, an intermediate representation of Java code [Val+10]. While program slicing has been extensively studied across platforms [Xu+05], Jicer was chosen because it is the only open-source static slicer that addresses Android-specific challenges, such as the absence of a main method, extensive use of callbacks, and inter-component communication (cf. Section 2.4). The Privacy Slice Visualizer seeks to answer two primary questions that developers might have:

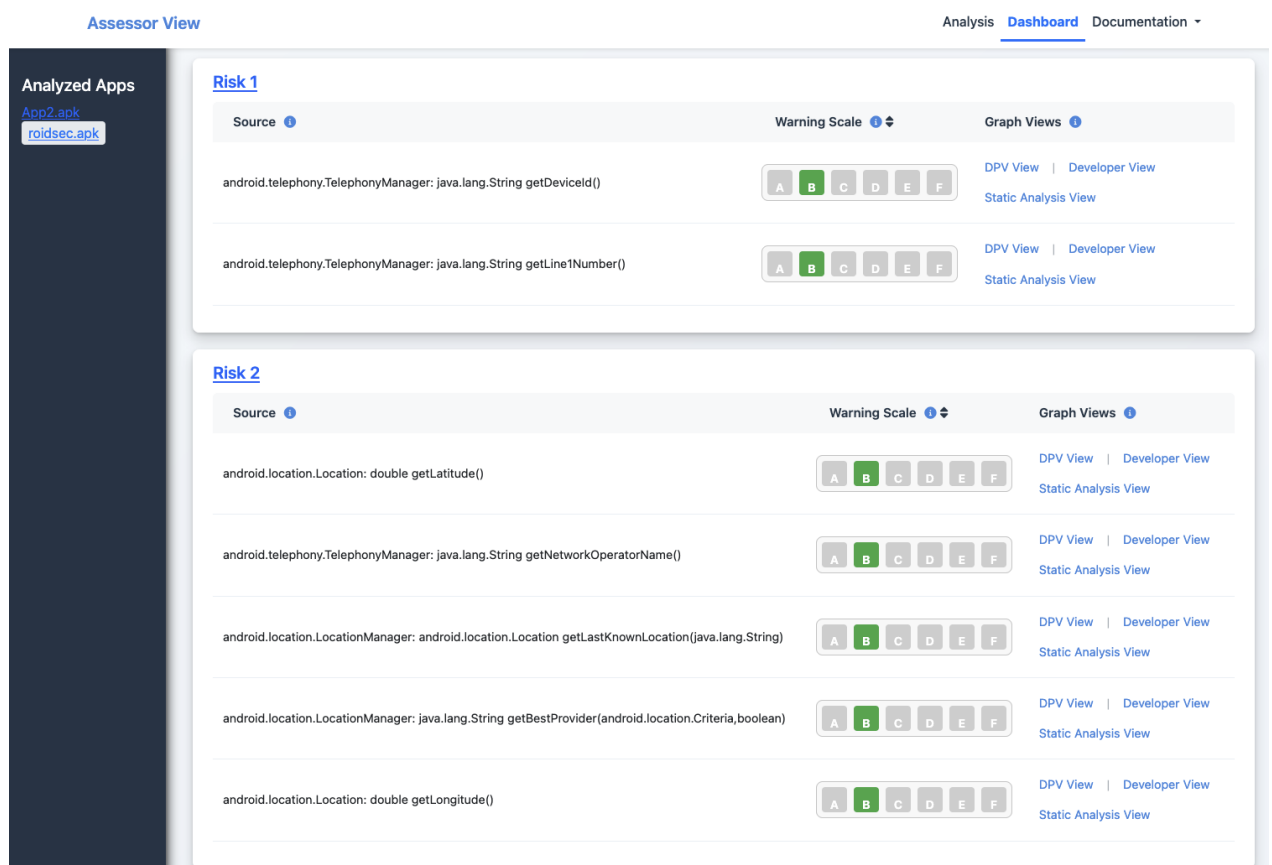
1. Which data collected by the app should be considered and reported as high-risk?
2. How does this data flow through the source code, and does it flow into data processing methods (eg. analytics, advertisement, authorization methods)?

B.1 Identifier Dataset. To answer the first question, the Privacy Slice Visualizer uses our manually annotated *Identifier Dataset* (cf. Section 3.2) which labels data sources based on a risk-based definition of privacy-related data.

A privacy-related data source refers to any API call or data access point in the source code that collects personal data—that is, information that can directly or indirectly identify an individual or device. This includes both explicit identifiers (e.g., device ID, email address) and contextual data (e.g., location, network information) that may lead to identification when combined with other data. Personal data that can identify an individual or a device without the use of any additional information is



(a) Visualizing risk sources detected in the APK.



(b) List of risky slices detected in the APK.

Figure 5.3.: Assessor View dashboard for Roidsec, a real-world app from the TaintBench suite [Luo+21].

categorized as **risk 1**, while data requiring additional information for identification is classified as **risk 2**. This dataset provides a risk-based definition of privacy-related data, allowing fine-grained labeling of APIs as data sources:

- *Risk 1*: Data that can identify a user or device without additional information.
- *Risk 2*: Data that requires supplementary information to enable identification.
- *Risk 3*: Data that can grant access to an individual's device but cannot identify an individual's device.
- *Risk 4*: Data that may identify an individual or a device depending on its content and the context in which it is used.

The risk-based definition is explained in detail in Section 3.1. The data categories in the dataset are aligned with Google Play Store's Data Safety Section (DSS), as developers are required to disclose these same data types when publishing apps on Google Play Console [25j]. This alignment ensures that the tool's output directly supports developers' compliance and reporting obligations, making the analysis results immediately actionable for completing the DSS. The process followed to construct this dataset is described in detail in Section 3.2.

B.2 Privacy-Relevant Libraries. The Privacy Slice Visualizer statically matches the system API calls against a list of packages of privacy-relevant third-party libraries and APIs. Note that the Privacy Slice Visualizer is designed as a proactive privacy by design [Cav09] measure, integrating into the app developer's workflow to analyze privacy-relevant third party libraries and API calls early in the development process. It operates before name mangling or obfuscation [Wer+18; NSS25], which are typically applied later when packaging the app for deployment.

In 2024, we consulted the AppBrain statistics [14b] and related literature [Cap+23; BBD16] to identify widely used libraries for pseudonymization and other privacy-relevant domains. Concretely, we manually configured Privacy Slice Visualizer to detect calls to privacy-relevant APIs, including system APIs and APIs belonging to 66 most commonly used third-party libraries across eight domains: analytics, advertisements, authentication, data serialization, network, input/output, email, and image processing.

According to GDPR, robust pseudonymization involves creating pseudonyms that cannot be easily re-identified and reproduced by third parties [Kon19]. Hashing, considered a weak technique for pseudonymization, can be reversed with a limited dictionary [Dem+18]. Grading pseudonymization functions based on these criteria can help app developers identify areas requiring additional data protection measures.

After identifying privacy-related sources, the Privacy Slice Visualizer performs forward slicing to retain all program statements affected by these data flows. Once the slicing is complete, it transforms the resulting program slices to a graph illustrating control and data dependencies within the source code. These program slices are represented in Jimple (B.3) and Java (B.4).

The UI A converts Jimple program slices to a **Static Analysis View** suitable for static analysis experts, and Java source code level program slices to a **Developer View** suitable for Android app developers.

Prior usability studies on these slices indicated that, for many stakeholders, raw code-level slices are too detailed and therefore difficult to interpret. In response to this need, we provide a simplified, privacy-oriented representation through the DPV Transformer.

© **DPV Transformer.** To address the knowledge gap between different stakeholders involved in privacy assessments, Assessor View uses the *DPV Transformer* to convert Java program slices into a privacy-oriented view with multiple abstraction levels. This module maps code-level properties observed in the Java slices to corresponding privacy concepts defined in the DPV.

The DPV Transformer is currently Java-dependent, as it operates on statically generated Java program slices (B.4). It performs a direct, rule-based translation of Java statements and API calls into DPV concepts such as data categories and processing operations. While the approach could, in principle, be extended to other languages, doing so would require adapting the parsing and mapping rules to their respective syntax and semantics. The details of this mapping are discussed in Section 5.1.2.

DPV Transformer accepts the Java program slices as input, and renders DPV information using two high-level DPV specifications, the DPV model (C.1), and DPV diagram (C.2).

The final step involves analyzing data processing activities within a program slice to identify potential GDPR compliance issues based on predefined risks. To provide warnings and suggestions for GDPR compliance, the DPV Transformer establishes these guidelines using clear and recognizable patterns. Currently, the transformer addresses five specific GDPR rules (cf. Table 5.1), which the user interface displays to the user (cf. Figure 5.4). Each box in the UI visualization represents a unique compliance risk and is color-coded semantically: light green for potential adherence warnings (rule 4 in Table 5.1), light yellow for warnings (rule 1 in Table 5.1), and pale red for high-severity GDPR violation risks (rules 2, 3, and 5 in Table 5.1). Section 5.1.2 further details how these compliance checks are implemented.

Table 5.1.: GDPR rules and warnings addressed by the DPV Transformer.

Rule	Program Slice Property	Warning	Legal Recommendation
1	Data collection.	This part of the app code collects data. Ensure transparency by reporting this data collection through privacy policy and Google Play Store's Data Safety Section.	Check reporting documents to adhere to GDPR Article §13 [18g].
2	Data collection but no processing operations.	This part of the app code collects data but may not be processing it. Ensure that data collection is necessary.	Potential violation of GDPR Article §5 [18m].
3	Processing of data in the absence of technical measures.	This part of the app code collects and processes data, in the absence of technical measures (pseudonymization).	Potential violation of GDPR Article §25 [18h].
4	Processing of data after implementing technical measures.	This part of the app code collects and processes data, but only after implementing technical measures (pseudonymization).	Potential adherence to GDPR Article §25 [18h].
5	Processing of data before implementing technical measures.	This part of the app code collects and processes data, before the implementation of technical measures (pseudonymization).	Potential violation of GDPR Article §25 [18h].

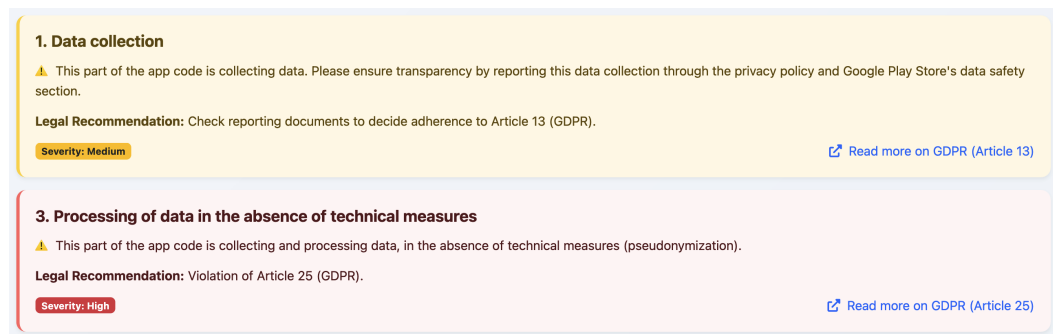


Figure 5.4.: GDPR Warnings for Roidsec. These warnings correspond to rules 1 and 3 in Table 5.1.

5.1.1 An Example

Throughout this section, we discuss a particular program slice from Roidsec¹, a real-world app from the TaintBench suite (cf. Figure 5.5).

We begin with the **Static Analysis View**, the most detailed representation produced by the Privacy Slice Visualizer. This view shows the program slice in Jimple intermediate representation as extracted from the slicer, preserving low-level Jimple-specific nodes (eg., Nodes 2 and 4 in Figure 5.5a).

¹<https://github.com/TaintBench/roidsec>

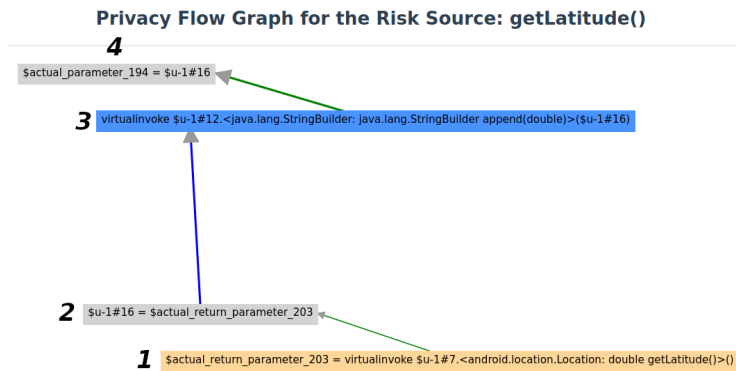
Next, the Privacy Slice Visualizer transforms the Jimple-based representation into a higher-level graph containing semantically correct, Java-like statements instead of raw Jimple code. This transformation enables developers to analyze privacy-relevant behavior without requiring Jimple expertise. The resulting output is the **Developer View** (cf. Figure 5.5b), where nodes represent source code statements and edges represent control and data dependencies.

In this slice, *latitude* (labeled as location data) is collected and combined with a string variable *var_2*. Since the DPV treats the *combine operation* as data processing, Assessor View recognizes that data collection and processing have occurred but notes that the source code lacks pseudonymization.

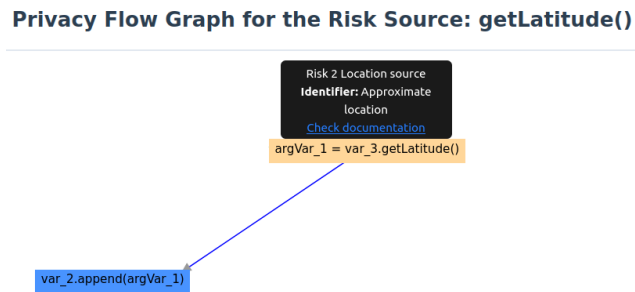
For a higher-level overview, the **DPV View** represents the Java slice as DPV specifications, presenting the app in terms of the Data Privacy Vocabulary (DPV) [Pan+24]. The DPV provides a detailed, machine-readable taxonomy for privacy-related information, based on GDPR terminology (cf. Section 2.5). The author of this thesis is a member of the DPV maintenance group [18c] and will collaborate with the group to further refine and extend this view.

Figure 5.5c shows a concrete instance of this DPV model for Roidsec. It uses DPV terms such as personal data categories, and processing categories, with relationships established through predicates such as *hasPersonalData* and *hasProcessing*. In Line 1, the placeholder app name *Droid* is used to represent any application, and the keyword *Process* defines the program slice as a new use case. Line 2 suggests that the app collects location, more specifically the GPS coordinate. The terms *pd:Tracking*, *pd:Location*, and *pd:GPSCoordinate* show that the personal data falls under the GPS coordinate category, which is a descendant of the DPV class *pd:Location*, itself a descendant of *pd:Tracking*. Line 3 shows that the app first collects the data and then combines it with another variable. To assist users in correctly interpreting the DPV model, a human-readable explanation is provided alongside the model: *This section of the application processes personal data of type GPS Coordinate. It undergoes processing operations in the following order: Collect, Combine.* (cf. Figure 5.5c)

Assessor View also converts the DPV model to a concise DPV diagram, summarizing the app at a high level (cf. Figure 5.5d). The DPV diagram represents the highest level of abstraction and intentionally omits the order of processing operations. This high-level view displays potential GDPR violations, GDPR compliance warnings, and suggestions to the privacy assessors (cf. Figure 5.4). The **DPV View** will flag the absence of technical measures such as pseudonymization, encouraging DPOs to question app developers about the source code. The lack of pseudonymization will be mapped directly to a **potential violation** of GDPR Article §25 [18h], providing



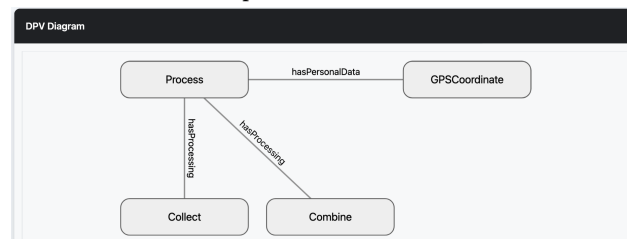
(a) **Static Analysis View**. Nodes 2 and 4 are Jimple specific nodes that are transformed by Privacy Slice Visualizer.



(b) **Developer View**.

DPV Model	DPV Model - Explained
1. ap: Droid dpv: Process ; 2. dpv: hasPersonalData <i>pd</i>: Tracking, <i>pd</i>: Location, <i>pd</i>: GPSCoordinate 3. dpv: hasProcessing 1. <i>dpv</i>:Collect ; 2. <i>dpv</i>:Combine ;	This section of the application processes personal data of type G P S Coordinate . It undergoes processing operations in the following order: Collect , Combine .

(c) DPV model and its human-readable explanation (**DPV View**).



(d) DPV diagram (**DPV View**).

Figure 5.5.: Different views for Roidsec.

actionable insights for assessors. During privacy assessments, assessors can easily switch between different views to ask developers specific questions and understand the technical measures implemented to protect data.

5.1.2 Implementation

We now discuss the implementation of Assessor View (cf. Figure 5.1).

Users begin by uploading an APK in Assessor View, and its *user interface* (Module [A](#)) processes user requests and executes the *Privacy Slice Visualizer* (Module [B](#)) with selected command-line options. Subsequently, the *DPV Transformer* (Module [C](#)) accepts these program slices as input and translates them into DPV representations for privacy assessors.

The user interface uses D3.js [24a], a JavaScript library for visualizing data. The visualized graph (cf. Figure 5.6) allows users to check documentation of the data source and offers other documentation explaining the risk-based classification of data.

Privacy Slice Visualizer

Privacy Slice Visualizer ([B](#)) is the back-end of Assessor View, responsible for performing static analysis on APKs by generating dependence graphs. It is a command-line tool implemented in Java. It is built on top of Jicer [PW21], a static program slicer that works with Jimple, an intermediate representation of Java code. Jicer is tailored to address Android-specific challenges, such as the absence of a main method, extensive use of callbacks, and inter-component communication (cf. Section 2.4). Privacy Slice Visualizer uses Jicer’s functionality to statically construct an Application Dependence Graph (ADG) from the APK, preserving control and data dependencies in the source code [HRB88]. It introduces its own algorithm to match method signatures from the ADG with the *Identifier Dataset* ([B.1](#)), labeling privacy-related data sources in the code. Once labeled, Privacy Slice Visualizer uses Jicer’s slicing component to perform forward slicing on the ADG, starting from the identified privacy-related data sources. This process yields forward slices representing the code components reachable from all labeled privacy-related data sources. Finally, the Privacy Slice Visualizer adds its own functionality to automatically label pseudonymization methods and other privacy-relevant methods within the sliced ADG using a privacy-relevant libraries dataset ([B.2](#)). It stores the sliced ADGs as text files, facilitating further analysis.

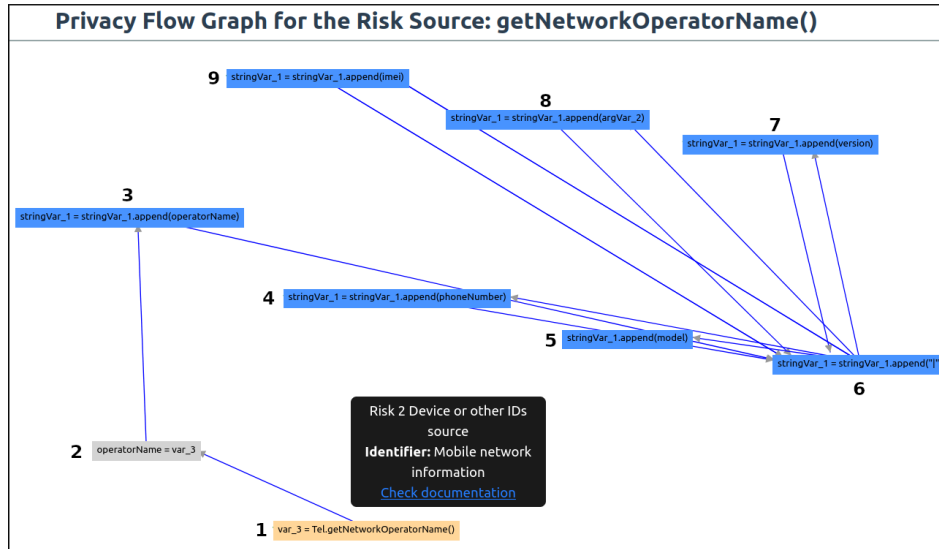


Figure 5.6.: Developer View of a Roidsec slice. Nodes: source code statements, edges: control and data dependencies. Pop-ups reveal details for each node (pop-up for node 1 shown in the figure). Node 1 collects the network operator name (risk 2), which is copied (node 2), and appended to a string (node 3). Nodes 4–9 append other variables. The dashboard assigns a low-risk (B) warning since only string manipulations occur, suggesting potential for data minimization (GDPR Article §4 [18]).

For rendering the **Developer View**, Assessor View triggers Privacy Slice Visualizer to execute custom transformations on the sliced ADGs. These transformations remove intermediate Jimple-specific nodes, reduce graph size and complexity, and convert Jimple nodes into more readable, Java-like representations to improve comprehension for developers. The transformed graphs present semantically correct Java-like statements (cf. Figure 5.6) instead of raw Jimple statements, enabling developers to work effectively without needing prior Jimple knowledge.

Currently, Privacy Slice Visualizer transforms key Jimple statements into Java-like representations, including *invoke*, *assignment*, *if*, *return*, *binary expressions*, *invoke expressions*, and *array variables*. These statements are commonly involved in data flows and their processing, and they tend to be more complex to interpret in Jimple compared to Java. Moreover, recent research [Su+25] reveals that *function calls*, *expression statements*, *if* and *decl statements* implement privacy behaviors in Android apps. We are actively working on expanding the mapping between Jimple and Java to improve the **Developer View**.

For brevity, we illustrate a simplified program slice example in this thesis (cf. Figure 5.6); more complex cases can be found in the accompanying artifacts.

Table 5.2.: A small subset of the mapping from source code statements in **Developer View** to DPV processing operations in **DPV View**.

Source code statement	Mapped DPV concepts
API methods from the Identifier Dataset (B.1)	dpv:HasPersonalData → <pd:identifier> dpv:HasProcessing dpv:Collect
Pseudonymization method	dpv:hasTechnicalMeasure <encryption OR others>
Assignment statement	dpv:HasProcessing dpv:Copy
Input output retrieval method	dpv:HasProcessing dpv:Retrieve
String manipulation method (eg., append(), concat())	dpv:HasProcessing dpv:Combine
Input output access method	dpv:HasProcessing dpv:Access
Database traversal method	dpv:HasProcessing dpv:Query
String replacement method (eg., replace())	dpv:HasProcessing dpv:Modify

DPV Transformer

The DPV Transformer serves as the bridge between the technical program slices and the higher-level privacy vocabulary used by assessors. It accepts Java program slices (B.4) as input and translates individual Java statements into corresponding DPV concepts. This translation is achieved through a mapping algorithm that aligns each program statement from the **Developer View** with a relevant DPV concept.

At present, this mapping process involves a one-time manual curation step. Each Java source method in the Identifier Dataset (B.1) is assigned an appropriate DPV term and stored in a reference database. During transformation, the DPV Transformer consults this database to automatically convert recognized source statements into DPV concepts. While this initial mapping was created manually, it is fully customizable and extensible, allowing new mappings to be added or refined as the DPV evolves or as new APIs are introduced.

The **Developer View** for each program slice provides structured details about the kinds of programming statements present, such as data manipulations, assignments, or I/O operations. The DPV Transformer uses this information to infer the type of processing activity being performed and map it to a corresponding DPV Processing concept (e.g., Collect, Store, Combine). A small subset of this current mapping is illustrated in Table 5.2.

An example of this mapping for Roidsec (cf. Figure 5.5) is presented in Figure 5.7. This mapping is also shown to the users of Assessor View so they can inspect every node in detail if required. The resulting output is a DPV model (C.1) that captures both the types of data and the sequence of processing operations. This model is then transformed into a DPV diagram (C.2).

The DPV diagram represents the highest level of abstraction and intentionally omits some details, including the order of processing operations. Additionally, if multiple labels can be assigned to the collected personal data, the DPV diagram retains

Mapping between Developer and DPV View			Hide
Developer View Node	Privacy Relevance	Mapped DPV Concepts	
argVar_1 = var_3.getLatitude()	Risk 2 Location source	dpv:Collect	
var_2.append(argVar_1)	String manipulation method	dpv:Combine	

</> Take me to the Developer View

Figure 5.7.: An example mapping between **Developer View** and **DPV View** for the Roidsec example from Figure 5.5.

only the most specific one. For instance, in the example from Figure 5.5c, the DPV diagram would preserve only a *hasPersonalData* edge between Process and GPSCoordinate (cf. Figure 5.5d).

To analyze GDPR compliance, DPV Transformer applies a set of predefined rules that link observed data-handling patterns in the code to GDPR obligations. Each program slice extracted by the Privacy Slice Visualizer is represented as a sequence of processing events—such as collect, process, or pseudonymize—which are mapped to corresponding DPV concepts (cf. Table 5.2).

The rule engine then evaluates these event sequences to identify conditions reflecting potential compliance or violation scenarios. For instance, to detect *processing of data in the absence of technical measures*, the tool checks whether a slice contains both data collection and processing events but lacks any pseudonymization operation. When this condition is met, Assessor View issues a high-severity warning linked to GDPR Article 25 (3 in Figure 5.4).

Each rule similarly encodes a combination of DPV terms, logical conditions, and corresponding legal references. While the current implementation relies on manually defined rules, the modular design allows the addition or modification of rules as new DPV concepts or regulatory interpretations emerge.

5.1.3 Early Experiments

At the initial assessment of Assessor View, we tested Privacy Slice Visualizer on 36 applications: 5 apps from DroidBench [18e], 25 apps from TaintBench [Luo+21], and 6 real-world Android apps available on Google Play Store. While DroidBench contains hand-crafted, small micro benchmark apps, the TaintBench suite consists of real-world malware apps, which are similar with respect to size and features to apps downloadable from Google Play Store. The real-world applications were selected from the dataset used to analyze domain-specific data collection in Section 3.4.2.

Figure 5.8.: Customization options for developers on the *Upload APK* page (cf. Figure 5.2), showing ① inclusion of control dependencies in the graph, ② maximum time limit for the analysis, and ③ maximum nodes allowed in a slice.

However, we observed that program slices were getting too large for manual evaluation. For this reason, we did not extend this experiment to a larger set of real-world apps from our dataset.

We investigated adapting further changes in the program slicing and compared the sizes of slices generated by default and the slices generated when excluding control dependencies while building the ADG. We observed that excluding control dependencies significantly reduced slice sizes. We thus added a *customization option* in the user interface, that allows toggling inclusion of control dependencies in the graph (① in Figure 5.8).

The first implementation of Assessor View composed only of a simple UI, the Privacy Slice Visualizer and the **Static Analysis View**. For an initial assessment of the viability of our approach we conducted two pilot studies (involving 10 and 6 participants, respectively) to evaluate the usability of Assessor View. While participants found the visualized program slices to be more efficient than manual code inspection, those unfamiliar with Jimple struggled to interpret the graphs generated by Assessor View. Additionally, visualizations with more than 50 nodes became challenging to interpret, due to increased cognitive load, as proven by Yoghourdjian et al. [Yog+21]. Since Jimple breaks down each Java statement into multiple statements, the **Static Analysis View** resulted in larger slices.

To address this, we implemented the **Developer View** that relies on the back-end to simplify and transform the data. The Privacy Slice Visualizer’s transformers omit Jimple-specific nodes, reducing the size and complexity of the application dependence graph (ADG) while preserving essential privacy-relevant information. Devel-

opers can now choose which view, [Developer](#) or [Static Analysis](#), they use. Moreover, the UI provides three key *customization options* to manage graph complexity and reduce visual overload:

1. Toggle inclusion of control dependencies in the graph (① in Figure 5.8).
2. Set a maximum analysis time limit for generating slices (② in Figure 5.8).
3. Specify a maximum number of nodes allowed in a slice (③ in Figure 5.8).

Correctness Checks on Assessor View

To ensure the correctness of Assessor View’s results, we first conducted sanity checks by manually verifying its output. The experiments were performed on an Ubuntu 20.04.2 machine with an Intel(R) Core i7-10850H processor (6 cores). In total, we evaluated Assessor View on 51 benchmark applications.

We first confirmed the correctness of the slicing algorithm on which the DPV mapping is based. To this end, we applied Assessor View to 18 DroidBench test cases [18e] and manually compared the generated program slices against the documented program behavior. To ensure coverage, we selected at least one test case from each category of the microbenchmark suite. No discrepancies were observed. We then verified that the detected DPV concepts (personal data and processing operations) were correctly mapped from the [Developer View](#) to the [DPV View](#). Out of 293 detected DPV concepts, 281 (95.9%) were verified through manual inspection. The remaining cases involved return statements and other complex statements, which are conservatively matched to *dpv:Processing* by design, resulting in intentional over-approximations. On average, Assessor View processed each test case in **34.38 seconds**.

To further assess runtime and verify the correctness of detected risk sources, we evaluated the tool on 33 applications from TaintBench [Luo+21]. We chose all apps from the TaintBench suite (33 out of 39) that had code size less than 100K lines. This LOC threshold ensured that the detected sources remained comparable against the benchmark’s ground truth—something not feasible with larger real-world apps. The sanity check revealed that all the sources detected by Assessor View matched with the sources present in the TaintBench ground truth. On average, Assessor View processed each app in **39.10 seconds**.

5.1.4 Limitations

Assessor View has several technical limitations. First, it uses the *Identifier Dataset* to identify privacy-related data sources. Although the dataset includes widely used third-party libraries from multiple categories (cf. Table 3.3), it may not cover all libraries used in Android applications. However, we ensure a comprehensive evaluation by including the most used third-party libraries from different categories. In addition, the degree of identifiability assigned to different data types may vary with different domains and compliance regimes. Since our approach focuses on GDPR, we can use the *Identifier Dataset* directly.

The Privacy Slice Visualizer matches the method signatures in the ADG with the labeled methods present in the *Identifier Dataset*. Since we use keyword matching to detect the presence of data sources including those from some third-party libraries, the analysis cannot handle code obfuscation in cases where the obfuscation also renames calls to these third-party APIs. However, the Privacy Slice Visualizer is designed to operate as a proactive privacy measure before name mangling or obfuscation measures are applied.

Another limitation concerns the scalability and interpretability of generated program slices. Although Assessor View can generate slices for real-world apps, our preliminary experiments showed that these slices—often exceeding 100 nodes—quickly become too large for meaningful manual interpretation. This complexity highlights the need for manageable high-level views that summarize the source code, making it a strong motivation for our approach. However, scalability remains an open challenge when automatically visualizing such large slices across multiple views. Consequently, in this work we focus on smaller, controlled benchmarks: the hand-crafted micro-apps from DroidBench [18e] and the apps from TaintBench [Luo+21], which are comparable in size and structure to typical Google Play applications.

The mapping between the Developer View and DPV concepts was performed manually and is therefore subject to human interpretation. We mitigated this threat to construct validity by engaging in detailed discussions among the researchers and grounding our decisions in the official DPV documentation and definitions. We also sought feedback from the DPV maintenance group, whose suggestions helped refine the mapping. The implementation will be updated accordingly to include their recommendations in future revisions of the codebase.

Assessor View currently supports DPV 2.0, and its mapping is designed to be modular but not fully self-updating. Any structural or conceptual changes in future DPV versions—such as updates to processing operations or personal data categories—will require manual review and adaptation of the Developer View to DPV mapping.

When asked about any plans to introduce stable versions of DPV, a member of the DPV maintenance group said, *“In applying DPV to resources, you want the latest legal implications and interpretations, and that is only possible if you advance the vocabulary. So we plan to release a new version approximately every 6 months. Any fixes or such will get adopted into that version.”* Continuous maintenance is therefore essential to ensure Assessor View remains aligned with the most recent legal and technical interpretations embedded in DPV.

Another limitation is the lack of a benchmark for systematically evaluating GDPR-specific rule detection in Android APKs. While we verified the correctness of the underlying slicing algorithm using DroidBench and TaintBench, there is currently no established benchmark for measuring the precision or recall of GDPR rule detection. Consequently, quantitative accuracy measures, such as false positive and false negative rates for GDPR-specific rule detection, remain an open direction for future work.

Finally, ongoing efforts are required to optimize the static program slicing algorithm in the Privacy Slice Visualizer and enhance the precision of the resulting program slices. While static analysis is useful for creating an initial prototype, future research could build on this foundation by exploring hybrid or dynamic approaches to further streamline privacy assessments.

5.2 Stakeholder Evaluations using Assessor View

Assessor View was evaluated in three sequential phases (cf. Figure 5.9). This section presents the first two phases of this evaluation. Section 5.2.1 reports the findings from the formative evaluation conducted with app developers using the initial prototype. The feedback obtained in this phase served as the basis for refining and extending the design, including the addition of DPV View for non-technical stakeholders. Building on these revisions, Section 5.2.2 presents the second evaluation phase, in which the updated prototype was assessed by privacy professionals including Data Protection Officers, to validate its usability and practical relevance within real-world assessment contexts. The final refinements and feature extensions of Assessor View, as recommended by privacy professionals, are presented in Section 6.2.

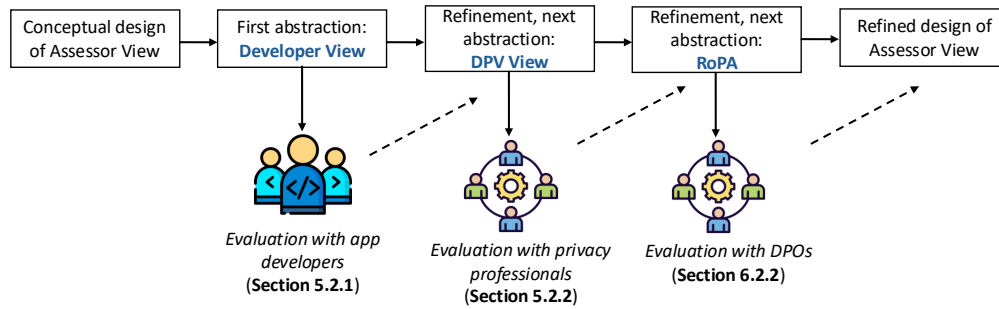


Figure 5.9.: Iterative user-centered design process of Assessor View. Developers evaluated the first implemented abstraction, the Developer View, which served as the basis for the later higher-level DPV View (cf. Section 5.2.1). After refinement of the system and interaction design, the extended prototype was evaluated with privacy professionals (cf. Section 5.2.2). The final refinement was evaluated again by DPOs (cf. Section 6.2.2).

5.2.1 Evaluation with Developers

We first wanted to understand if Assessor View and more specifically, Developer View is suitable for developers to understand data protection in their Android apps. Thus we conducted a user study to answer the following research questions:

RQ1. *To what extent does Assessor View help developers understand data protection in Android apps?*

This question evaluates Assessor View, capturing the combined effect of program slicing and visualization on developers’ ability to understand privacy-relevant aspects of source code. By addressing this question, we assess how tooling that integrates slicing and visualization can support developers during the development and analysis of Android apps.

RQ2. *To what extent does the Developer View better support developers in understanding the flow of privacy-related data within an app?*

RQ2 focuses on one central component of Assessor View, the Developer View, which simplifies Jimple statements to improve program comprehension. By isolating this design choice, we examine how a Java-oriented visualization contributes to developers’ understanding of privacy-related data flows and helps explain the effects observed in RQ1.

In our evaluation, the baseline reflects the current practice: app developers performing these tasks manually, without tool support. This involves manually inspecting source code to identify privacy-related data and assess processing activities—an approach that, as prior research [KMB24; Kha+23] suggests, is time-consuming and may be error-prone.

Study Methodology

To answer the research questions, we conducted a within-subjects user study [23a] with 12 participants. We initially attempted to recruit app developers who had participated in a survey we conducted in Section 4.2, but none responded within the available timeframe. As a result, and given that students are considered reasonable proxies for software developers in usability studies [Ber04; SMJ15; SAW08], we recruited Computer Science students instead. The 12 participants of this user study are disjoint from the 16 participants of the pilot studies conducted as early experiments (cf. Section 5.1.3). Interested participants were first sent to a landing page informing them about the study’s purpose. Then, they were shown our consent form where we informed them about how we would handle their data, their right to terminate the study at any time, and where they could give consent to participate in the study. Finally, they could schedule select time slots for the user study.

Study Procedure. At the beginning of the study, participants were given all the information about the study, and were asked to sign the *informed consent* form. They then answered questions on a Likert scale from 1 (Beginner) to 5 (Expert) about their experience (cf. Table 5.3).

Next, participants received a 5-minute overview of Assessor View. Since we wanted them to try out the tool themselves, we skipped a detailed demo. Participants were then asked to access Assessor View through a web server, and complete three tasks:

Task 1: Analyze an app from the micro-benchmark DroidBench [18e] by identifying the data collected, determining GDPR protection requirements, and observing privacy-relevant properties in the resulting program slice. While the dashboard provided partial support, fully assessing privacy-relevant properties required further inspection of the data flow in the visualized program slices.

Task 2: Evaluate both Static Analysis and Developer Views in terms of usability, simplicity, and relevance to app-specific tasks.

Task 3: Perform the same analysis as in Task 1 on a more complex app from the TaintBench suite [Luo+21].

Table 5.3.: Summary of participant experience. Filled circles indicate expertise from 1 (Beginner) to 5 (Expert).

ID	Android app dev	Java dev	Jimple	Static analysis tools	Graph visualization	Data privacy	GDPR
P01	●●○○○	●●●○○	●○○○○	●○○○○	●●●○○	●●●○○	●●●○○
P02	●●●●○	●●●○○	●○○○○	●●○○○	●●○○○	●●●○○	●○○○○
P03	●●●○○	●●●○○	●○○○○	●●○○○	●●○○○	●●●○○	●●●○○
P04	●○○○○	●●●○○	●●●○○	●●○○○	●●●○○	●●●○○	●●●○○
P06	●○○○○	●●●○○	●●●○○	●●○○○	●●○○○	●●○○○	●●○○○
P07	●○○○○	●●●○○	●●○○○	●○○○○	●●○○○	●●○○○	●○○○○
P08	●○○○○	●●●○○	●○○○○	●●○○○	●●○○○	●○○○○	●○○○○
P09	●○○○○	●●○○○	●●●○○	●●○○○	●●○○○	●●●○○	●○○○○
P10	●●●○○	●●●○○	●●○○○	●○○○○	●●●○○	●●●○○	●●○○○
P11	●○○○○	●●●○○	●●○○○	●●○○○	●●●○○	●●●○○	●●●○○
P12	●○○○○	●●●○○	●○○○○	●○○○○	●●●○○	●●●○○	●●●○○

Tasks 1 and 3 were designed to assess if Assessor View assists developers in understanding data protection in Android apps (*RQ1*). For these tasks, DroidBench and TaintBench apps were chosen because their source code and ground truth were available. In addition to the documented ground truth, we manually examined the app source code to evaluate the accuracy of participant responses. The selected apps also provided examples of manageable size, making them suitable for a controlled, time-constrained user study.

Task 2 was designed to understand how effectively the experimental Developer View aids in understanding program slices in comparison to the Static Analysis View (*RQ2*). To evaluate the Developer View, participants were given a DroidBench app, and asked to explore and understand both views.

After these tasks, participants answered several 5-point Likert-type questions from the System Usability Scale (SUS) [Gri+13], a questionnaire designed to measure the effectiveness and efficiency of a system, and rated Assessor View using a Net Promoter Score (NPS) [Rei04], that measures their likelihood of recommending the tool to a friend.

Participant Experience. At the beginning of the study, participants self-reported familiarity with Android app development (median 1/5), Java (median 3/5), Jimple (median 2/5), static analysis tools (median 2/5), graph visualization (median 3/5), data privacy (median 3/5), and GDPR (median 3/5) (cf. Table 5.3). Participants have a median of approximately 1.5 years of experience in Java development, with maximum experience of 5 years.

Ethics and Data Protection. The user study design was approved by the Ethics committee of our institution. Our consent form, privacy policy, and data handling practices were approved by the Data Protection Office of our institution.

Study Results

In this section, we present the results of the user study conducted with 12 participants, who we refer to as P01–P12. We discarded P05’s data, since they declared themselves to be a beginner and novice in all skills, and required additional assistance and time during the study, far beyond what was needed by other participants. We summarize the results below:

RQ1. To what extent does Assessor View help developers understand data protection in Android apps?

Participants first worked with a DroidBench app, which had one privacy-relevant program slice with 10 nodes. Despite limited knowledge of GDPR, 10 participants successfully identified the privacy risks in the app and accurately determined which data required GDPR protection. Apart from P04 who over-reported the risky sources, all the other participants were able to analyze the slices, interpret the graphs, and understand the data flow, correctly identifying privacy-relevant properties.

The TaintBench app given for Task 3 had seven program slices, with a maximum size of 20 nodes. Of the 11 participants, 10 accurately identified all the privacy-relevant information in the given app. P03 correctly identified 50% of the privacy-relevant details, but failed to report risk 4 slices, likely due to customizing Assessor View to only report risk 1 data sources. Interestingly, P04 (who struggled to understand privacy risks and data flow in the first task) could successfully identify and report the privacy-relevant details in this app, suggesting a slight learning curve.

Tasks 1 and 3 included questions about the dashboard. All 11 participants agreed that the overview and the hints provided by the dashboard helped them better understand the tool (cf. Figure 5.10). P02 suggested more detailed description of all data sources within the dashboard.

Four participants (P03, P07, P10, and P12) explicitly mentioned the warning scale as one of the most useful features of the dashboard. P03 noted, *“I like how all risks are categorized with source and warning scale”*. P10 said, *“Warning scale is the best way to convey overall risks”*. P04 found the bar and pie charts easy to understand. P09 said, *“Assessor View gave me a good understanding about privacy-related data*

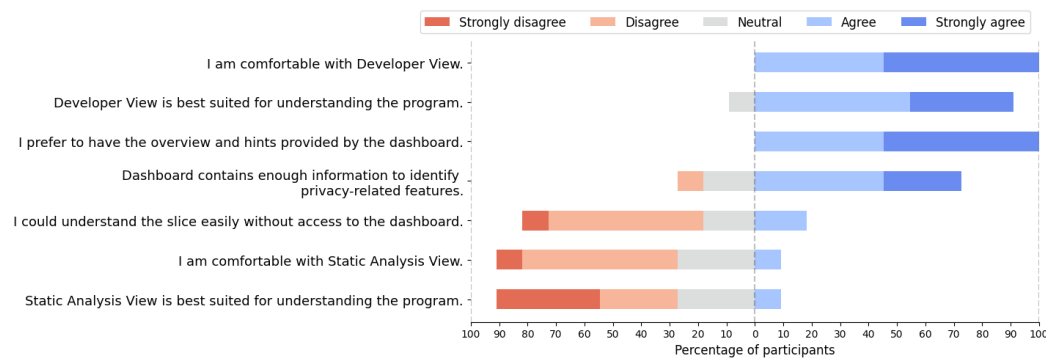


Figure 5.10.: Participants’ responses to the tool-based questions about dashboard and the views.

flows. Warning scale was helpful. I prioritized looking at slices rated D before I moved on to safer slices”. P12 confessed that Assessor View made them aware of potential privacy risks.

Using the System Usability Scale (SUS), Assessor View obtained a mean score of 63.96, which falls below the industry benchmark of 68. Notably, users with no prior Java development experience expressed dissatisfaction with its usability. This lower score may be attributed to the fact that this version of Assessor View is the result of an initial iteration in the user-centered design process. Since Assessor View is designed for privacy analysis and visualization, the gap in experience with Android development and static analysis tools (cf. Table 5.3) could explain some of the usability challenges reported by less experienced users.

However, participants with one to five years of Java development experience rated Assessor View with an average SUS score of 68, indicating a satisfactory level of usability. These findings suggest that Assessor View provides an acceptable usability experience for Java and Android developers.

The Net Promoter Score (NPS) for Assessor View was calculated as 18.18, reflecting a positive perception among participants.

RQ1 summary: Assessor View effectively assists developers with prior development experience in identifying and observing privacy-relevant properties in Android apps.

RQ2. To what extent does the Developer View better support developers in understanding the flow of privacy-related data within an app?

Most of the participants (N=10) agreed that the Developer View was **easier** and **faster to understand** compared to the Static Analysis View: *“I have more experience with Java so I could quickly understand the Developer View”* (P07), *“Unlike Static Analysis View, Developer view shows the data of risk sources in a simple way instead of giving too many details.”* (P02). While P09 also found the Developer View quicker to grasp, they appreciated the Static Analysis View for its detailed control flow information. However, they acknowledged that their familiarity with Jimple made it easier to follow, and those less experienced with Jimple might struggle with its unfamiliar syntax and temporary variables. Notably, all 11 participants agreed or strongly agreed that they were **more comfortable** with Developer View (cf. Figure 5.10).

To compare user preferences for both views, participants were asked, *“How likely is it you would recommend Developer View over Static Analysis View to a friend?”* (and vice versa). Their responses were used to calculate separate Net Promoter Scores (NPS) for each view. The NPS for Developer View was 45.45, reflecting strong user preference over the Static Analysis View. In contrast, the Static Analysis View received a negative score of -72.73.

Participants found the Static Analysis View **complex** (P06: *“The Static Analysis View has too many nodes and edges”*), **hard to understand**, and **not user-friendly for beginners** (P02: *“It is not understandable for any non-programmer or normal user”*). P10 noted, *“I don’t know the Jimple syntax.”* P11 complained, *“The Static Analysis View contains so many other constructs and naming techniques that require explicit familiarity”*.

On the other hand, the Developer View received good feedback. For instance, P03 commented, *“The Developer View aligns with the source code I’m familiar with in Android Studio,”* showcasing how the representation resonates with developers accustomed to Java environments. P07 stated, *“The Developer View captures the necessary flow without overwhelming the user with too much detail.”* P01 claimed the Developer View was *coherent with source code*. P11 said, *“It’s much more concise and helps me focus more on the actual code that I’m familiar with.”* P12 thought that Developer View was *human readable, and less cluttered*.

RQ2 summary: Developers are comfortable with Developer View, since it is more concise and aligns with the source code.

Overall feedback. The final set of questions had some open questions asking for feedback.

Participants generally regarded Assessor View as a *good tool for developers* (P01) which was *reasonably easy to use* (P01), with some describing it as a *promising start* (P03) and noting its *great usability* (P02). Ten out of eleven participants found the documentation helpful, and P02 appreciated the *inclusion of relevant research papers linked within the documentation*. P03 highlighted the benefit of being able to access all previous results. P08 appreciated being able to track the whole workflow with helpful legends.

Participants also offered valuable suggestions for improving Assessor View. P04 recommended providing more information about Jimple and suggested in-depth modifications to the user interface. P06 pointed out that graphical views may not be very effective for handling large codebases. P03 proposed including more specific hints to clarify the meaning of the graph views.

Feature Extensions

Based on the findings of this user study, we introduced DPV View and GDPR warnings in the next prototype of Assessor View. Furthermore, we made usability enhancements to the dashboard and added more documentation tailored towards DPOs and legal experts.

Limitations and Threats to Validity

To investigate possible usability implications of Assessor View for developers, we conducted two pilot studies (cf. Section 5.1.3) and one user study (cf. Section 5.2.1). A threat to external validity arises from the participant population. Since it is challenging to recruit developers for a user study and students are considered reasonable proxies for software developers [Ber04; SMJ15; SAW08], our user study used students as participants. Nevertheless, students may differ from professional developers in terms of experience, familiarity with privacy assessments, and exposure to real-world Android development practices. As a result, the findings may not fully generalize to professional developers.

We conducted the user study in a controlled environment (12 participants, 1 hour study duration) rather than in a development setting. In practice, developers would have more time to investigate more advanced apps. Therefore, the results should be interpreted as evidence of Assessor View's usefulness in a controlled study setting, rather than as a complete reflection of how it would be used in real-world development workflows. It would be interesting to conduct a larger quantitative evaluation in real-life conditions.

Another threat concerns the choice of apps used in the study. We selected DroidBench and TaintBench apps because their source code and documented ground truth were available. This allowed us to compare participant responses against known data flows and, in addition, to manually inspect the app source code to assess the accuracy of their answers. However, the use of DroidBench apps limits external validity. DroidBench consists of small, hand-crafted benchmark apps that are designed to exercise specific data-flow scenarios and therefore do not fully capture the size, complexity, and architectural diversity of real-world Android applications.

This choice was also influenced by the current visualization scalability of Assessor View. Although Assessor View can generate slices for real-world apps, our preliminary experiments showed that such slices can quickly grow to more than 100 nodes, making them difficult to interpret manually within a time-constrained user study. This observation reinforces the need for manageable high-level views that summarize relevant source code behaviour, which is one of the motivations for Assessor View. At the same time, automatically visualizing large slices across multiple views remains an open challenge.

Consequently, the results of this study should be interpreted as evidence for the usefulness of Assessor View on controlled benchmark tasks of manageable size, while further work is needed to evaluate its scalability and effectiveness on larger real-world apps.

5.2.2 Evaluation with Privacy Professionals

In this section, we discuss the study we conducted with Assessor View after introducing DPV View. Our aim with this study was to understand Assessor View's possible role in conducting privacy assessments, with a particular emphasis on non-technical stakeholders such as DPOs and legal experts. In this study, we aim to answer the following research questions:

RQ1. *How are privacy assessments conducted in a real-world setting?*

This question identifies the key stakeholders responsible for these evaluations, their specific roles, and whether they rely on any tools or frameworks for support. Understanding these aspects will provide insight into the practical challenges and best practices within privacy assessment workflows.

RQ2. *To what extent does the source code analysis-based Assessor View support privacy assessors in conducting DPIAs?*

This question examines to what extent Assessor View can act as a bridge between technical and non-technical stakeholders, enhancing communication and collaboration. The aim is to assess its effectiveness in reducing knowledge gaps and ensuring smoother privacy assessments through better alignment of technical and legal perspectives.

Study Methodology

Given the exploratory nature of our research, we employed a semi-structured interview approach to gather insights while allowing flexibility for participants to share their perspectives.

Participant Recruitment. As interview subjects, we were seeking to recruit DPOs or other legal and privacy experts. The author of this thesis directly contacted 175 privacy experts on LinkedIn. We received 38 responses (21.71% response rate), out of which 15 agreed to participate in the study. We also recruited 4 participants by sending an email to the DPV maintenance group [18c].

Those interested in participating in our study were sent to a landing page informing them about the study's purpose. Then, they were shown our consent form where we informed them about how we would handle their data, their right to terminate the interview at any time, and where they could give consent to audio-recording of the interview. Finally, they could schedule their interview.

In total, we recruited 19 privacy assessors, but three participants had to cancel due to unforeseen circumstances. Therefore, 16 participants took part. We refer to our participants as P01–P16 (cf. Section 5.2.2).

Study Procedure. We used Zoom for all interviews. All interviews were audio-recorded via Zoom's functionality. Five interviews (P01-P05) were conducted by the author of this thesis together with a second researcher, while the remaining interviews were conducted solely by the author. All interviews were conducted remotely between February and June 2025. We expected the interviews to take 45 to 60 minutes, and scheduled one-hour appointments with all interviewees. Interviews lasted between 26 to 85 minutes; the median duration was 47.5 minutes.

At the beginning of each interview, we introduced ourselves and provided details about the interview process. We explained that participation was voluntary, and we could skip any questions with no penalty (interview briefing in Appendix C.1). Interviewees could ask questions, and we took additional verbal consent for in-

terview recording. Given the exploratory nature of our research, we employed a semi-structured interview approach to gather insights while allowing flexibility for participants to share their perspectives. The interview had 4 sections:

1. **Pre-questionnaire:** We first asked participants to self-report their knowledge about data visualization, GDPR, GDPR compliance, DPIA, and DPV. The pre-questionnaire also asked them about their experience, their highest degree, their additional privacy certifications (if any), and their role within their organization.
2. **Current state of the art:** In this section, we asked participants questions about their experience with DPIA, and other GDPR compliance assessments. This step was to gain understanding on how privacy assessments are conducted within an organization, who is involved in them, and to what extent they are currently automated (*RQ1*).
3. **Tool-based questionnaire:** We introduced Assessor View using a three-minute demo video². We then asked participants to try out Assessor View, and complete two tasks to assess whether given source code of a demo application from TaintBench [Luo+21] complies with GDPR guidelines. This part of the interview allowed participants to explore the various views of Assessor View, and report on their experiences.
4. **Tool usability and effectiveness for DPIA:** Finally, we asked for some qualitative feedback about Assessor View, and tried to understand its relevance for automating DPIA. Here we also asked the participants some specific questions about the design of the DPV model, and DPV diagram. After these tasks, participants answered several 5-point Likert-type questions from the System Usability Scale (SUS) [Gri+13], a questionnaire designed to measure the effectiveness and efficiency of a system, and rated Assessor View using a Net Promoter Score (NPS) [Rei04], that measures their likelihood of recommending the tool to a colleague. This final part of the interview study was aimed at understanding to what extent Assessor View, in its current state, can automate the process of DPIA (*RQ2*).

The interview questionnaire is available in Appendix C. The interview protocol was refined following a pilot study with a colleague, ensuring clarity. A summary of the final interview protocol is presented in Table 5.4.

²Available at <https://doi.org/10.5281/zenodo.15085611>

Table 5.4.: Summary of interview protocol. The first column is the topic and the number of questions for that topic.

Topic (#)	Example questions
Privacy assessments/ DPIAs (5)	Can you walk us through a privacy assessment process? Who are the stakeholders? Which parts of the assessment are manual? What is your role in the assessment process? How do privacy assessors navigate understanding source code to successfully complete such an assessment?
DPV (2)	Is DPV or any other vocabulary actively used in the assessment process? Any plans of long-term support version of DPV?
GDPR compliance (3)	Have you used any tools to verify whether a software complies with GDPR? Which documents do data controllers need to complete to adhere to GDPR? Do your clients consult you to help complete these documents? Is this a manual process?
Privacy training (2)	Are there any training programs specific to GDPR that are mandatory for the stakeholders involved in a privacy assessment process? Do these training programs cover automation?

The last two sections of the study—the tool-based questionnaire, and tool usability evaluation—employed a mixed-methods approach, combining a cognitive walk-through with a usability-focused questionnaire. During this phase, participants were encouraged to share their experiences freely, and as a result, we did not follow a strictly predefined protocol for this part of the study.

Participant Experience. In the pre-questionnaire, participants answered questions on a Likert scale from 1 (Beginner) to 5 (Expert) about their experience (cf. Table 5.5). Participants have an average of approximately **10.78 years** of experience in privacy and/or law. Participants self-reported familiarity with Data visualization (median 3/5), GDPR (median 5/5), GDPR compliance (median 5/5), Data Protection Impact Assessment (median 5/5), and Data Privacy Vocabulary (median 4.5/5).

6 participants have a doctoral degree. P03, P07, and P14 have a doctoral degree in Computer Science. P10 and P13 have a doctoral degree in Law, and are actively working in data protection and compliance. P06 has a doctoral degree in Information Law.

P01, P04, P05, P08, P09, P12, and P13 (N=7) are DPOs. P01, P04, P05, P06, P08, and P09 (N=6) have additional privacy certifications from the International Association for Privacy Professionals (IAPP) [18p]. P01, P04, P09, and P13 (N=4) have German DPO certifications. P02, P03, P14 are Computer Science researchers from the Data Privacy Vocabulary maintenance group [18c], and P10 is a legal researcher from the DPV group. P11 is an auditor.

Table 5.5.: Summary of participant experience. * denotes members of the DPV maintenance group [18c]. Filled circles represent expertise levels on a Likert scale from 1 (Beginner) to 5 (Expert). P12 and P15 only participated in the non-tool based study (Sections 1 and 2 of the study).

ID	Job title	Country	Exp in privacy /law (yrs)	Familiarity				
				Data visualization	GDPR	GDPR compliance	DPIA	DPV
P01	Jr. Data Protection Counsel Legal	Germany	3.5	●●●○○	●●●●●	●●●●○	●●●○○	●●●●●
P02 *	Researcher	Germany	7	●●○○○	●●●●○	●●●●○	●●●○○	●●●●○
P03 *	Assistant Professor	Ireland	11	●●●○○	●●●●●	●●●●●	●●●●●	●●●●●
P04	Privacy Counsel	Germany	8	●●●○○	●●●●○	●●●●○	●●●○○	●○○○○
P05	Managing Director, External DPO	UK	15	●●●○○	●●●●●	●●●●●	●●●●●	●●●●○
P06	Manager	India	12	●●●●●	●●●●●	●●●●●	●●●●●	●●●●●
P07	Researcher	Germany	15	●●●○○	●●●●○	●●●○○	●●●○○	●●○○○
P08	DPO	Cyprus	7	●●●○○	●●●●●	●●●●●	●●●●●	●●●●●
P09	Corporate DPO	Germany	9	●●●●○	●●●●●	●●●●●	●●●○○	●●○○○
P10 *	Associate Researcher	Greece	14	●●●○○	●●●●●	●●●●●	●●●●●	●●●●●
P11	Regulatory Compliance Analyst	Nigeria	3	●●●○○	●●●○○	●●●○○	●●●●●	●●●●●
P12	DPO	Saudi Arabia	5	●●●●●	●●●●○	●●●●○	●●●●●	●●●●●
P13	DPO	Germany	12	●○○○○	●●●●●	●●●●●	●●●●●	●●●●○
P14 *	Associate Professor	Spain	20	●●●○○	●●●●○	●●●●○	●●●○○	●●●○○
P15	Legal and Policy Counsel	Netherlands	4	●●●○○	●●●●●	●●●●●	●●●●●	●○○○○
P16	Legal Researcher	Belgium	7	●●○○○	●●●●●	●●●●●	●●●●●	●●●●●

Participants have a versatile experience in attending and conducting DPIAs. P01, P04, P05, P08, P09, P12, and P13 (N=7) have been DPOs in many DPIAs. P03 and P07 have experience in leading the writing of DPIA, drafting documents and conducting the necessary research. P10, P15, and P16 provide legal insights in a DPIA. P02 and P14 are privacy researchers who have studied the process of DPIAs without being actively involved in its implementation.

Data Analysis. We recorded the audio of all interviews, transcribed them using NoScribe [25m] and reviewed the transcripts for potential mistakes. We used an iterative open coding approach [CS90] to perform thematic analysis [CB14] of all interview transcripts. The author of this thesis carried out inductive coding to derive emergent themes, which were then compiled into a codebook. A second researcher independently reviewed the codebook, and disagreements were discussed and resolved through refinement of the final coding scheme. During this process, the researchers reached full agreement on all sub-themes but refined the classification by renaming 10 of the 95 codes. The final codebook is available in Appendix C.3.

Ethics and Data Protection. The study design was approved by the Ethics committee of our institution. Our consent form, privacy policy, and data handling practices were approved by the Data Protection Office of our institution. NoScribe [25m] was selected to transcribe the interviews following our DPO's recommendation. Some DPO participants also reviewed the study materials before participation to confirm compliance with data protection principles.

Study Results

In this section, we present the results of the analysis of the 16 semi-structured interviews.

RQ1. How are privacy assessments conducted in a real-world setting?

We now discuss the processes conducted as part of a DPIA, the stakeholders involved, tool support for DPIA, the challenges faced in conducting a DPIA, and the processes and support required for GDPR compliance.

Processes in a DPIA. The interview study revealed that a DPIA is primarily an **information-gathering process** (P05). It is often conducted as an online call, with a mixture of forms and questionnaires where the team implementing a system provides details on the system's purpose, the necessity of its approach, the technical and organizational secure measures implemented within the system, and a discusses possible risks i.e. what would happen if the system went rogue, or breached and data would be lost or made public (P04). While P01 described the process as taking

stock, making inventory, and documenting it within the Record of Processing Activities (RoPA) [18q], P03 summarized it as **GDPR-specific investigations of systems**. According to P12, the primary goal of the DPIA is to map where the personal data goes, how it is used, and what risk is attached at each stage. P05 emphasized on the importance of talking to their clients and exploring alternative approaches that could reduce risks to the personal data.

Participants also highlighted the different perspectives regarding the DPIA process. P01 explained how legal teams verify the security measures implemented in a system: *“We (the legal team) ask for confirmation from cybersecurity team, if they are happy with these measures that are implemented. If they say yes, we are happy with it. The legal team doesn’t go and check the technical implementation.”* P05 stressed on the ethical dimension of a DPIA: *“Something I add into a DPIA (which is not a part of the standard process) is making them think if they would be happy if this was their data that all this processing was happening to. In my mind, if that answer is “no”, well then you need to ask yourself why anyone else would be happy (with this happening to their data).”* P02 highlighted the importance of source code analysis in DPIAs: *“(On why source code analysis is important in a DPIA) For example if there’s a logging mechanism which is not necessary which logs personal data, then that is something that needs to be taken into consideration. If I do a bank account transfer, the bank would log when I transferred how much money to which other company (in addition to the account) then this is a second location where personal data is effectively stored. Then these internals of the software (where is this stored etc) are highly relevant. Otherwise IT administrator of the bank can see what I did. I don’t want that.”*

P09 explained their job in a DPIA: *“I can translate between technical and legal requirements. The core reason why I have a job is because that is what I’m good at that. I can talk with the developer and understand what they say and then I can talk with legal people or business people and understand what they say. So this is basically my core competency to bridge this gap of understanding.”*

Stakeholders in a DPIA. Participants identified four main categories of stakeholders in a DPIA:

1. **Business functions:** These include entities responsible for collecting and processing personal data, such as data controllers, data processors, application owners, technical teams (IT, cybersecurity), management teams (human resources, marketing), and project sponsors.
2. **Privacy functions:** The Data Protection Officer (DPO) plays a counselling role, with the privacy team in the operational role. P01 (a DPO) said, *“I prepare the questionnaires, go after the stakeholders, walk them through the questionnaire,*

interview them to understand what activities they are doing and to collect the data flow from them from the moment they collect it. For which purpose they are using data, based on which legal ground, where are they hosted, if there are technical measures in the document, and other important topics of that conversations. I then document it on the Record of Processing Activities.”

3. **Data subjects:** DPIAs should involve representatives of the individuals directly affected by the system. As P10 observed, *“As the European data protection supervisor puts it, we do not protect the data. We protect the people behind the data. So our primary consideration in data protection law are those people and their vulnerabilities.”*
4. **Third parties:** Business partners or third parties involved with the system under assessment. DPIA allows for consultations with different experts. In rare cases, DPOs may involve the data protection supervisory authority at later stages (P03).

Tool support for a DPIA. All participants with DPIA experience reported that most organizations rely on **manual processes** for conducting DPIAs. Furthermore, the degree of automation varies based on the level of digitisation of the company. Some mid-level digital organizations use software solutions to automate inventory of processing activities and stocktaking activities. More digitally advanced companies may apply process mining to analyze event logs.

P02 explored the use of a tool [18b] by the French Data Protection Authority [18a] that guides organizations in conducting DPIAs, resulting in a report that you can use to demonstrate compliance with GDPR and the risks, how you can mitigate risks, and to properly inform data subjects. P04 noted that some organizations automate meeting transcript generation and summarization. P09 and P12 used data governance tools such as Caralegal [25a] and Collibra [25c] to structure their DPIA. P06 explained, *“Some of these clients do have the money to invest in tools, like OneTrust, etc. where all DPIA questionnaires are just uploaded onto the tool and then is shared with the application owner.”* P08 said, *“So, a lot of it is manual, but because of the type of environment within which we work. We have, like, standard forms that can just be triggered. But then someone still has to do the manual data entry and someone still has to do the manual risk review so it’s not like it’s powered by AI or something.”*

P16 explained where manual assessment can be the most useful: *“As a strong lawyer and privacy oriented person, I would say the more manual the process, the better to a certain extent, especially in this phase of assessing the risks. While I see the automatic tools being more important for maybe having the catalog and mapping of data processing activities, but while I also understand that there are certain limitations*

as budget and resources, I believe that at least this final assessment of are we accepting these risks should be done manually.” Thus, automation remains uncommon in DPIA workflows.

Vocabularies used in a DPIA. The use of Data Privacy Vocabulary (DPV) or other standard vocabularies for structuring DPIA content is **rare**. In some cases, highly digital organizations develop their own internal vocabularies, shared across departments (e.g., IT, privacy, information security, and data governance) (P04). P08 explained, “We are familiar with it (DPV), but I think that we don’t really use it specifically for our DPIA process.” P16 further explained, “These common vocabularies, standardization practices are completely relevant and used exactly because then it’s easier to make this communication among different areas. So, for example, technical developers are more keen on understanding certain standards and vocabularies. So using these is really important. But it’s also important to understand the limits of certain initiatives. Since they are not part of the regulation per se, they are not necessarily officially recognized. There are certain limits because they will not necessarily translate everything that is needed for a DPIA. And at the same time, you cannot just rely on that as to translate everything because the authorities will not necessarily see that as the final result or the final tool to be used. So there are limits, even though they are very used and very, very useful.”

Challenges faced in a DPIA. Participants identified **knowledge gaps** and **lack of communication** between the different stakeholders as the biggest challenge in conducting DPIAs: P01 observed, “As my professional experience is concerned, that (privacy assessors understanding source code to complete a DPIA) is the biggest void in the market. There is no communication between developers and lawyers, they are in two completely different worlds.” Similarly, P03 emphasized the difficulty of collaboration due to differing expertise: “Legal team drafting DPIA (liability or contractual rights or data subject rights: legal topics) would have to discuss stuff with a developer. A developer is not going to give them legal answers. There has to be some common ground, which doesn’t exist.” P04 explained how the difference in conceptual models between legal and technical teams further complicates collaboration: “We (legal experts and technical teams) have a difference in vocabulary and I’d say mental model and concepts, which makes it sometimes hard. This leads to frustrations on both sides, by the way.”

Beyond communication barriers, conflicts can arise due to the **trade-off between privacy and innovation** of a software. P06 observed, “It’s not just legal versus technical as well, because you also need to understand the business interests. So you need to understand how the business functions, why they’re using this application for, you know, exactly what purpose and what are they trying to achieve even fiscally, monetar-

ily. Because you have to then, because it's not your job to just go and say no to everyone, right? You have to understand why they're trying to achieve a certain goal and then what might be the best way to actually protect data while achieving that goal." P16 added, "Sometimes technical developers are more inclined into going to innovation and first testing and training a certain tool and then thinking about compliance while it is really important to have privacy by design and by default. So even before we know the whole potential of a certain software, it is important to already have these privacy solutions embedded solutions. So this clash of interests is always a challenge."

Participants also pointed to **delays in considering privacy principles**, with privacy teams often being involved too late in the process. Ideally, DPIAs should be conducted before processing activities begin, but this is rarely the case in practice. P01 noted, "Best case (in conducting a DPIA) is before starting processing activities, but in practical world that does not happen because people get privacy involved later. According to privacy by design, developers should think of privacy principles. They should keep requirements of the law (such as data minimization, pseudonymization) in mind when they develop a new software. But it's not a rule. And technical knowledge of the lawyers is very limited." P15 added, "Sometimes we are involved a bit from the outset. Other times we are really a bit of an afterthought. There is no point in lying: most GDPR is an afterthought." P05 stressed the importance of involving DPOs earlier in the development process: "If DPOs have important conversations (with the IT team) on day one, it saves a lot of time and conflict. Reasons for such conflicts vary. In some cases, it is the ignorance and lack of knowledge (of what the law says they can and can't do). In my experience some know that this is wrong. They hope (in a DPIA), the DPO would be light on them and they would get away with doing anything they want."

Another key challenge is the **lack of source code analysis-based tools** for assessing source code from a privacy perspective. P04 noted the absence of automated code scanning tools for GDPR compliance, despite their prevalence in security assessments: "I have never seen automated code scanning from privacy point of view (so far). From security point of view, yes, which takes some privacy issues but it is more focused on security, not on GDPR compliance part apart from Article §32 [18j] (security of processing)." P09 observed, "So I remember looking at tools like that would look through code and then give you certain information but nothing specifically tailored for DPIA. I remember some early prototypes, like 2020 maybe, but nothing that ever got into production that say like consistent use."

P01 pointed out that privacy experts and DPOs are not actively engaged in guiding privacy-conscious development: "Majority of recommendations of DPOs or privacy experts stay superficially on the principles of GDPR in Article §5 [18m]: data mini-

mization, limitation of storage, and things like that. At least I don't know anyone that goes inside of it in the development, in trying to influence and recommend Privacy Enhancing Technologies.”

Finally, P03 noted that many stakeholders lack awareness of the information required for a DPIA. As a result, privacy professionals must actively seek out relevant details: *“Usually the challenge is that most people don't know all the information needed for a DPIA, so you have to hunt for people who have the information, and figure out a way to convert their answers into what exactly you want in a DPIA.”* P16 added, *“I see a challenge in defining roles in, for example, a software developing company. They can then act as processors or controllers of data. Exactly defining these roles is quite challenging.”*

GDPR-Specific Documentation. Participants identified **18 documents** that data controllers may need to complete to adhere to GDPR (cf. Appendix C.3). Fourteen participants identified **Record of Processing Activities (RoPA)** [18q] as the most important document. RoPA, a legally mandated inventory detailing how personal data is processed, is often used as starting point for DPIAs. As P10 explained: *“Completing the RoPA is a data mapping exercise which allows stakeholders to envisage potential threats.”*

Clients, typically data controllers, seek legal help to complete other GDPR-related documents: privacy notice, standard agreements and contracts, assessment of partner activities, rule of conduct, DPIA report, informed consent, report of data breach, data access request, and data protection by design assessment.

Most participants help their clients by manually completing and reviewing these documents. P04 noted that the approach depends on the level of sophistication in the company: *“If we're talking to a start up that basically has little financial resources then we work with Word and Excel templates that we provide to them that have comments on how something should be filled out. If we have a more financially sophisticated organization that is also capable to do more, then we use so-called legal tech tools that are decision trees in a way where for instance for legitimate interest assessment under Article §6.(f) you will be asked a couple of questions.. and in the end it'll give you the balancing test or the outcome of the balancing test created if you wish in a way by the court of justice. We have these types of document generators (that don't check source code) that help clients to fill out this documentation without me as a legal advisor being there all the time. We train them so they can use it.”* P09 explained, *“But the last five years, I would say the focus was mainly getting people to enter the data rather than getting it out of systems directly.”*

Moreover, 10 out of 16 participants mentioned using standard templates to complete GDPR-specific documentation. P07 noted: *“(Completing such documents) is mostly based on spreadsheets and word tables and word documents, because that’s what most people are comfortable in using.”* P10 elaborated, *“Most of the templates are available. So there are templates by the European Commission for processing agreements or international transfers. There are templates for deep data protection impact assessments. What you need is to put the expertise on how to fill it in to know what every section requires.”*

Compliance Support. Participants claimed that GDPR compliance is a **manual process**. P03, P04, P08, and P12 mentioned using cookie compliance tools to verify website dialogue boxes but had not used any automated code-scanning tools for GDPR compliance. P07 admitted to using privacy policy scanners. P04 noted: *“I played around with a Privado [22i] product in my spare time. Code scanner to identify what type of processing is done. But I’ve not seen this in organizations, or in their Continuous Integration and Continuous Deployment (CI/CD) pipelines. So I have little experience with it.”* P12, who has used security and code analysis tools such as SonarQube [25s], OneTrust [25o], Salesforce Privacy Center [25q], said, *“The tool is helping through the process, but I have in my position to check everything manually. I am the point of contact. I explain every section in the document and help them to complete it.”*

Training Programs. P01, P05, and P10 pointed out the lack of training programs designed specifically for GDPR compliance.

In contrast, many participants observed different kinds of training programs and certifications that are encouraged. P04 observed: *“All organizations I’ve seen have this base level compliance for all employees (who use a computer) and then dedicated training programs for different roles depending on the intensity of handling personal data.”* P09 said, *“For the business units, we usually have onboarding for the entire RoPA process, which is part of the DPIA.”* P06 added, *“We recommend mandatory awareness sessions simply because if they don’t know what they’re doing, they will just treat it as a box-ticking exercise. So we recommend it, but obviously as consultants, we don’t mandate awareness exercises.”* P16 explained, *“There are no trainings that are mandatory. Any training offered is seen as a good practice and can be used to further show that all employees were part of a training, but none is mandatory. So there is no specific course that should be done, but any course can be used as a proof of compliance of making the whole company aware of privacy issues.”* P12 emphasized different formal certifications (CIPP/E, CIPM) that DPOs can take to enhance their knowledge.

RQ1 summary:

- DPIA is a manual, information-gathering process requiring coordination among privacy, legal, and technical experts.
- Key challenges include knowledge and communication gaps between experts, the privacy–innovation trade-off, delayed involvement of privacy professionals, and the lack of source code analysis-based tools.
- Data controllers rely on privacy assessors for manual GDPR compliance support.
- RoPA is the most important documentation required for GDPR compliance.

RQ2. To what extent does the source code analysis-based Assessor View support privacy assessors in conducting DPIAs?

To answer RQ2, the final part of the interview involved a hands-on session with Assessor View. P12 could not access the tool at their location, and P15 declined participation in this session, and hence we have data from 14 participants.

DPV View and GDPR Warnings. After using the tool, almost all participants (N=13) agreed that the **DPV View helps in identifying potential GDPR violations** or areas requiring further investigation (cf. Figure 5.11). P09 remained neutral: *“I probably would end up having to look at the documentation of the specific function to understand the scope of the risk”*.

80% of all participants agreed that **GDPR compliance warnings and suggestions are useful in guiding their analysis**. P05 added, *“The compliance warnings and suggestions are good and would help in guiding analysis. I will caveat that by saying that of course without having sight of the application being reviewed itself, it is not possible to judge how accurately the issues have been detected.”* P04 further remarked, *“They already hint at potential problems that would need further analysis and to be linked back with what was initially planned.”*

80% of all participants agreed that the **warnings effectively highlight potential GDPR compliance issues**. P06 commented, *“The warnings do a good job of flagging issues that non-privacy folks may not have thought of.”* While 70% of all participants felt that the warnings simplify the assessment of GDPR principles such as data minimization, purpose limitation, or security measures, P01 disagreed because they believe that *the thorough analyses of the compliance of the processing should be connected to an end purpose, which is difficult to be detected from the source code*.

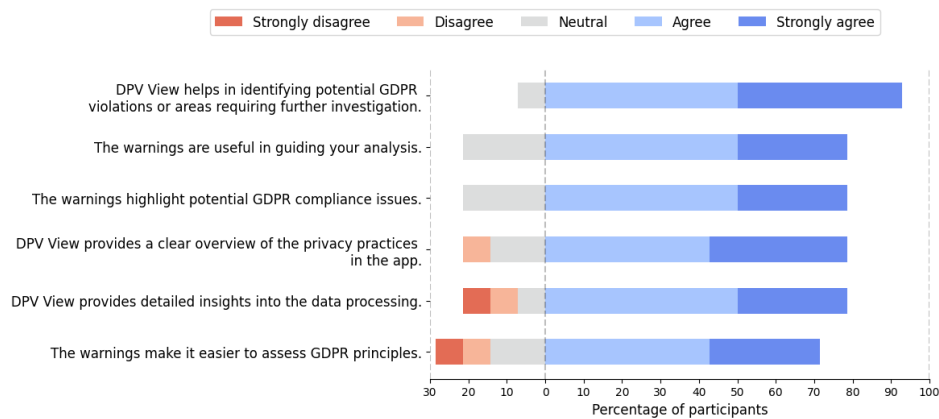


Figure 5.11.: Participants’ responses to the tool-based questions about DPV View and GDPR compliance warnings.

80% of all participants agreed that the **DPV View provides a clear overview of privacy practices in the program slice**. P06, however, disagreed: “*The DPV View needs to work on explaining itself more. The flow itself may be useful, but the tags are not.*” P04 commented, “*The DPV View for the demo app tells me that there is a combination of data, but I lack information on what data it was combined with.*”

80% of all participants pointed out that the **DPV View provides detailed insights into specific data processing activities and flows**. However, P04 remained neutral, noting that the connections among different program slices are missing, making it challenging to fully understand the broader data flow within the application.

Usability and User Satisfaction. Participants answered several 5-point Likert-type questions from the System Usability Scale (SUS) [Gri+13], a questionnaire designed to measure the effectiveness and efficiency of a system. Figure 5.12 presents their responses.

Notably, over 80% of participants found Assessor View **easy to use**, and said they would like to use it for their needs. However, 20% felt that people would struggle to learn the tool quickly and, as a result, 14% lacked confidence when using it. Additionally, over 40% indicated that they would need support from a technical expert to use the tool effectively—which is understandable, given that the tool is designed for both technical and non-technical users. Moreover, 14% identified a significant learning curve associated with using Assessor View.

The SUS score for Assessor View was 75.41, marking a clear improvement over the previous version’s score of 63.96 (cf. Section 5.2.1). It also surpassed the industry benchmark of 68, indicating overall acceptable usability. Net Promoter Scores (NPS) indicated that the tool is better suited for privacy experts (28.57) and DPOs (21.43)

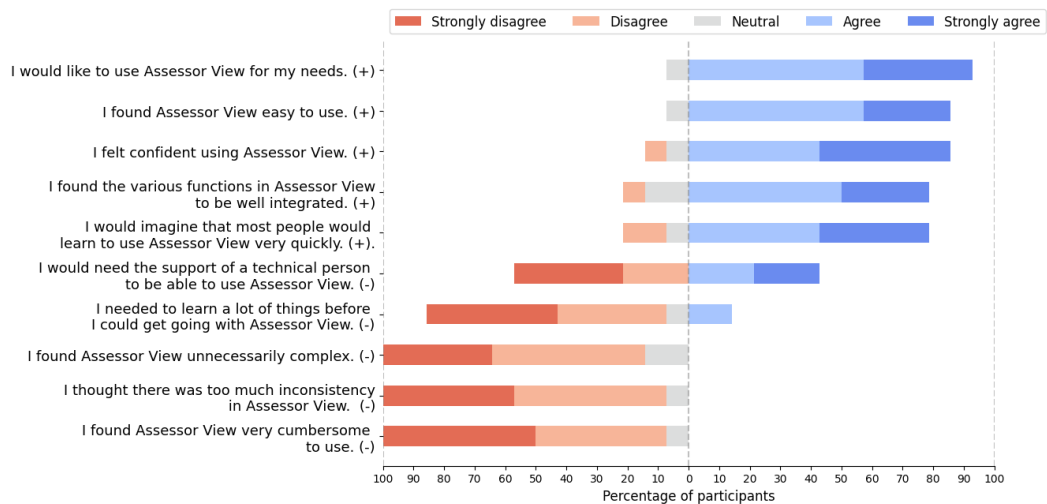


Figure 5.12.: Participants’ responses to the usability questions. For (+) statements, *Strongly Agree* represents a more favorable response. For (-) statements, *Strongly Disagree* represents a more favorable response.

than for legal professionals (-14.29) (individual ratings in Figure 5.13). This suggests that participants may perceive Assessor View as more immediately relevant to privacy assessment and DPO-oriented tasks than to purely legal work. However, these findings should be interpreted with caution because the expertise categories of participants overlap and SUS and NPS values are sensitive to individual responses. Therefore, we did not draw strong subgroup-level conclusions from the SUS or NPS scores.

Potential Use for Assessments. All participants agreed that Assessor View can be used for conducting DPIAs. P01 said, “*It will help to get answers from the technical team and developers. But as I mentioned, connecting relevant slices to a final purpose would help a lot.*” They added, “*It would be great for tech teams to self-assess their tools, and use the output to defend their decisions in a DPIA.*” P05 commented, “*I think the tool is good. I think it could form part of an assessment very well, actually.*”

Participants highlighted Assessor View’s use during the **pre-DPIA screening phase**. P08 said, “*I think it might be very helpful at the information gathering part of the privacy assessment. Where we are trying to understand the processing activity and the context. I think it might be very, very helpful. And maybe where the company is in the process of building a software. They can also use this to reveal and have a better understanding of their codebase.*” P16 noted, “*The tool has an enormous potential to facilitate the data mapping as it is based on the source of development, which mitigates the risks of not addressing certain activities that could not be mentioned in a manual process.*” They added, “*It makes it even easier for a DPO or a legal expert to evaluate*

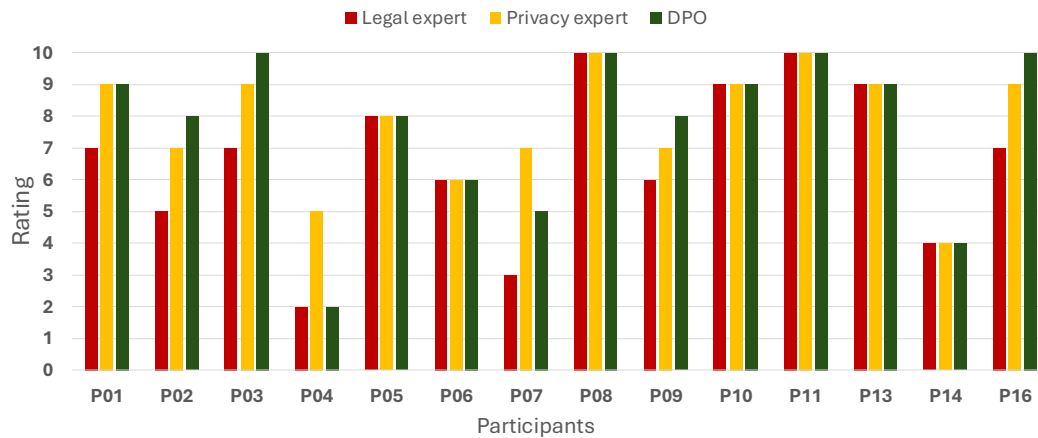


Figure 5.13.: Assessor View's rating for legal experts, privacy experts, and DPOs. (0: not recommended, 10: highly recommended)

a software, which sometimes can be very challenging exactly because of this issue of translation between the developer and the person conducting the DPIA. So it's really, really relevant to make sure that the entry points and the exit points are mapped. So exactly to understand the data flow within an app."

Participants emphasized on its importance for **bridging communication gap** between stakeholders. P11 said, "Definitely it will ease communication because with this, it was like a lot more simple for me to see the issues in relation to the app. It kind of dumbed it down for me because I'm not technical." P09 further noted, "I think it could actually help a little bit both sides to understand each other better. It basically opens the use for two groups, one who understands the things, who can drill down on it; and the other one to get a broad overview of what's going on." P06 added: "It's a good place to start because the biggest issue I see in privacy is because DPIAs generally have to be triggered by the application owners. A lot of them, by default treat it as a box-ticking exercise. So they just mark everything as low risk so that it never gets flagged for a DPO." P04 said, "Assessor View would not enable legal people to talk more in a technical way, but the other way around (technical people can talk more legally). With what we have now, it will analyze a piece of technology and hint at potential issues that you would have to have a look at. It will draw attention, and that could be helpful." P02 remarked, "The DPO is not that involved in the development, and isn't the legal expert. The DPO can ask the developer: What do you think about this warning? Can you apply pseudonymization? Does it justify leaving boundaries? Or... is storing something on Cloud necessary? I think this is a good tool for DPOs."

At the same time, participants emphasized the **importance of retaining manual review for final risk evaluation**. P07 observed, *“There are other parts, especially, you know, regarding the question of risk assessment and access, like talking to stakeholders, understanding their individual concerns, which I think is important that I don’t think can be, should be automated.”* P06 remained neutral about tool usage, explaining, *“They (tool-based warnings) may be useful, but only if they are a precursor to a full-blown investigation.”* P16 added, *“As a strong lawyer and privacy-oriented person, I would say the more manual the process, the better to a certain extent, especially in this phase of assessing the risks. While I see the automatic tools being more important for maybe having the catalog and mapping of data processing activities, but while I also understand that there are certain limitations as budget and resources, I believe that at least this final assessment of are we accepting these risks should be done manually.”*

This perspective reinforces the idea that automation should **complement** rather than **replace** expert judgment, providing structured insights and traceable mappings of data processing activities while leaving ultimate decisions about risk acceptance to privacy assessors.

Additional Insights and Suggestions. P01 emphasized that processing personal data is only lawful if linked to a legitimate business purpose or legal ground, suggesting that GDPR compliance should be assessed in context rather than in isolation. For example, storing and processing personal data without pseudonymization could be GDPR compliant if justified by a legitimate business need. P02 supported this view, adding, *“This makes me skeptical if it is sufficient to look at the software without considering the context.”* P04 recommended adding hints to show where analyzed functions are located in the source code to facilitate collaboration between privacy professionals and developers. P05 noted that while the tool identifies risks, it lacks actionable guidance on resolving them. They proposed adding hints such as *“this data should be encrypted at this point”* and *“users should be encouraged to use multifactor authentication to enhance security”*. P04 recommended enhancing data flow transparency by providing information on the type of collected data, its movement within the system, and whether it is combined with other data. P07 suggested making the dashboard more interactive: *“It would be great if you could follow your thoughts, maybe leave notes somewhere and say, okay, I need to follow up on this.”* P08 recommended adding pseudonymization related hints to the relevant GDPR warnings.

Participants suggested the following tool extensions: P01 recommended documenting dashboard visualizations to present as evidence of GDPR compliance to supervisory authorities. P02 noted, *“Dataflow diagram is the most relevant extension of this tool. You are showing me a graph which connects nodes within the software, but*

specially the case where data leaves the boundary of your software, things become interesting and relevant, and this is something I would like to see.” P03 and P05 suggested converting Assessor View’s output to RoPA. P03 and P04 proposed displaying the tool’s output as a list or report for easier discussion in DPIAs. P04 observed, “A list would be more natural for a DPO. The way it’s presented now is more natural to a developer.”

RQ2 summary:

- Participants found Assessor View well suited to assist DPOs and privacy experts in conducting DPIAs and pre-DPIA screenings.
- They suggested automatically converting DPV View to RoPA, or reports that DPOs can use to conduct DPIAs.

Discussion

The interview study identified a diverse range of stakeholders involved in a DPIA, with expertise spanning privacy, technology, and law. This highlights the need for effective communication and a platform such as Assessor View that caters to multiple target groups.

Further discussions revealed that DPIAs are still largely conducted manually, with minimal use of standard vocabularies. While this presents an opportunity for automation, the lack of active use of DPV in DPIAs poses a challenge for making Assessor View relevant in this context. Interestingly, although P01, P04, and P05 were unfamiliar with DPV, they were acquainted with other ontologies, which helped them understand the DPV View. Notably, P04, who identified as a beginner in DPV (cf. Table 5.5), remarked, “DPV... *oh, that looks familiar to me*” upon reading the DPV documentation in Assessor View. This suggests that while DPV is not yet widely used in DPIAs, privacy assessors may find the DPV specifications more accessible than source code-based representations. P07 supported the use of DPV for bridging knowledge gap: “*I wouldn’t rely on it (DPV) to be the perfect dictionary, but it’s a very good start, and I don’t know of any better (standard vocabulary) at the moment.*”

The biggest challenges in conducting a DPIA include knowledge and communication gaps between legal and technical experts, absence of automated privacy-focused tools, and delayed involvement of privacy professionals. The study also revealed that data controllers often rely on legal advice to comply with GDPR, with privacy assessors providing manual support.

Moving forward, our findings suggest several design directions for automation in privacy assessments:

- UI-based tools that allow partial document generation (e.g., pre-filling sections of the DPIA or RoPA from technical analyses) to reduce manual effort while maintaining expert oversight.
- Interactive dashboards that guide privacy assessors through compliance tasks, translating technical outputs into regulatory language suitable for DPOs and consultants.
- Automated prompts for business owners or developers when new data processing activities may require initiating or updating a DPIA.
- Integration with existing compliance ecosystems, since current tools such as OneTrust [25o] do not process source code, and static analyzers such as SonarQube [25s] overlook privacy-specific dimensions such as data minimization, purpose limitation [18m], and Record of Processing Activities (RoPA).

Assessor View demonstrates how such automation can be used as a reflective probe-supporting structured discussions, improving mutual understanding, and helping privacy professionals translate code-level insights into compliance documentation. As several participants (P03, P04, P05) suggested, extending Assessor View to automatically generate reports such as the RoPA would be a natural next step.

Overall, these insights illustrate how automation can complement human expertise, providing structured information to support decision-making rather than replacing expert judgment.

Takeaway. Assessor View represents an initial step toward facilitating communication between legal and technical stakeholders, and automating privacy assessments. It is currently well suited for privacy experts and DPOs.

Limitations and Threats to Validity

As with any qualitative study, our findings are influenced by the sample of participants and their perspectives. To improve representativeness, we recruited a diverse group of privacy assessors through multiple rounds of outreach, including direct contact with 175 experts from 43 countries. While we aimed for a diverse participant pool, some potential participants outside the European Union had limited familiarity with GDPR and redirected us to their EU-based teams, who did not respond. Despite these efforts, we acknowledge that experts with strong privacy concerns

may be underrepresented, as they are typically less likely to participate in interview-based studies [Sch+07]. Similarly, we encountered a low response rate among DPOs and members of the DPV maintenance group, which may limit the breadth of viewpoints captured. However, we successfully recruited participants from 11 countries, including 4 participants from outside the European Union, which provided a geographically diverse set of perspectives.

Another threat to external validity concerns the applications used during the study. We selected the TaintBench suite because it provides real-world Android applications with documented privacy-related data flows, making it suitable for discussing privacy assessment tasks with experts. However, although TaintBench apps are more realistic than hand-crafted benchmark apps, they still represent a limited set of applications and may not capture the full diversity of Android apps. Therefore, the findings should be interpreted as reflecting expert feedback on Assessor View in the context of TaintBench-based assessment tasks, rather than as evidence of its effectiveness across all real-world Android applications.

To ensure that our analysis accurately represented participants' experiences with privacy assessments, we employed an iterative open coding process [CS90] following thematic analysis principles [CB14]. The author of this thesis performed inductive coding to derive emergent themes. To improve coding consistency and reliability, a second researcher independently reviewed the codebook, and disagreements were resolved through discussion, ensuring coder alignment. This systematic refinement process, along with the public availability of the final codebook (cf. Appendix C.3), enhances the transparency and construct validity of our findings by ensuring that the themes faithfully reflect the underlying data.

Although the main goal of the study was to capture experts' experiences with Assessor View, we also collected quantitative feedback using the System Usability Scale (SUS) [Gri+13] and Net Promoter Score (NPS) [Rei04]. Since the study is primarily qualitative, these scores should be interpreted as complementary indicators rather than as statistically generalizable measures of usability. Nevertheless, the sample size is reasonable for a qualitative study with well-defined goals. Prior work indicates that as few as six interviews can be sufficient when participants are relatively homogeneous and the study objectives are qualitative in nature [GBJ06]. A larger-scale quantitative evaluation with more participants and more varied assessment tasks remains an interesting direction for future work.

5.3 Related Work

We discuss prior research in two major areas relevant to our work: (i) program slicing and visualization, and (ii) data protection assessment.

Program Slicing and Visualization. Since we focus on analyzing Android apps, we concentrate on slicing techniques tailored to the Android ecosystem. While program slicing has a rich history across platforms [Xu+05], Jicer [PW21] is the only available static slicer for Android apps. It uses Soot [Val+10]’s DotGraph API [24b] to generate static graphs in Scalable Vector Graphics (SVG) format. However, these graphs do not scale well and lack interactivity. Several dynamic slicing approaches for Android have also been proposed [Azi+19; ALR21].

In 2007, Sridharan et al. [SFB07] proposed WALA [26j]-based thin slicing, which excludes control dependencies and focuses only on statements that compute and copy the value returned by the slicing criterion—a strategy relevant to our compliance goals. Prior work has visualized program slices either graphically [19a; BE94] or textually [Kri04], but these visualizations were primarily designed for debugging and program understanding. Our work builds on these foundations but shifts the focus from traditional debugging or general program understanding toward compliance-oriented program comprehension and analysis. Specifically, we use static program slicing that preserves both control and data dependencies, and we develop a visualization interface tailored to identifying privacy risks in accordance with GDPR obligations.

Data Protection Assessment. A growing body of work investigates how static analysis techniques can support privacy risk assessment. A survey conducted by Franke et al. [Fra+24a] showed that developers frequently cite the lack of automated support as a major obstacle to GDPR compliance. Yet, a **research gap** remains in understanding how collaboration between different stakeholders can be effectively supported. Our study addresses this gap by interviewing privacy assessors to identify practices and challenges that may inform better collaborative tools.

Pandit et al. [Pan+19] identified the absence of standard vocabularies to describe personal data, data handling purposes, and categories of processing. They introduced the Data Privacy Vocabulary (DPV), which describes various components and relationships between them [Pan+24]. However, there remains a **research gap** in mapping software components to such vocabularies, underscoring the persistent knowledge and representation challenges in privacy assessments. Our study

addresses this gap by introducing Assessor View, which automatically maps code elements to DPV-based representations to support shared understanding among assessors.

Feiyang Tang et al.'s analysis [TØ22] was among the first to assist app developers and DPOs in evaluating privacy compliance and simplifying the DPIA process. They introduced an automatic software analysis technique that presents the results as a graph of privacy flows and operations. While their approach aligns with our objectives, we additionally use the Data Privacy Vocabulary (DPV) [Pan+24] to make the analysis results more understandable to privacy assessors.

Similarly, other taint analysis-based tools [TS23; Zha+23; Mao+24] use FlowDroid [Arz+14] to analyze data flows, primarily to detect privacy vulnerabilities, or verify third-party library compliance. While effective at identifying direct leaks, traditional taint analysis is limited in scope [KB24; FS18]. It operates by analyzing reachability between predefined sources (where sensitive data originates) and sinks (where data may be exposed). Without known *predefined* sinks, taint analysis cannot proceed.

Our approach overcomes this limitation. Rather than requiring predefined sinks, we take known sources as input and identify potential sinks as output. While, in theory, taint analysis could be adapted for GDPR compliance by treating every program point as a potential sink, this is not scalable in practice. In contrast, our source-centric, slicing-based analysis supports broader coverage and enables a more nuanced, risk-based assessment of personal data handling. It supports a more actionable evaluation of GDPR obligations.

There has been some exploratory research in the area of automated GDPR compliance. Ferrara et al. [FS18] discussed main components needed to develop GDPR reports, and static analysis techniques in the form of source to sink analysis, to support GDPR compliance- but not specifically for DPIAs. They also observed that information flow analyses only report the program point where data is leaked, not the complete flow, arguing that these were not sufficient for GDPR compliance. Martin et al. [MK18] recommended privacy by design [Cav09] and proposed tools for GDPR compliance, and Li et al. [LWE19] explored automated GDPR compliance in continuous integration workflows. However, these approaches primarily target developers, without addressing the broader collaboration challenges faced by privacy assessors and legal experts—establishing a **research gap** in supporting multiple stakeholder roles during privacy assessments. Our study addresses this gap by using Assessor View to explore how automation can support collaboration across these diverse roles.

5.4 Summary

In this chapter, we introduced Assessor View, a web tool based on static program analysis that analyzes Android apps and presents its findings through stakeholder-specific views tailored to the needs of different actors involved in the privacy assessment process.

Through a user study with 12 participants, we demonstrated that Assessor View effectively supports *developers* with prior development experience in identifying and reasoning about privacy-relevant properties of Android apps. We then employed a next version of Assessor View as a research probe in an interview-based study with 16 participants to explore the challenges *privacy assessors* face during privacy assessments and to examine their perspectives on the role of automation as a form of support.

Our findings reveal several key challenges in current privacy assessment practices, including gaps in domain knowledge, limited communication between legal and technical experts, the lack of source code analysis-based privacy analysis tools, and the late involvement of privacy professionals in the development lifecycle. The results further show that the GDPR-related warnings and guidance provided by Assessor View are particularly valuable to Data Protection Officers and privacy experts, and that the tool's design is well aligned with the needs of these stakeholders.

Together, these studies provide a comprehensive view of how different user groups interact with and benefit from Assessor View. Overall, our results suggest that Assessor View represents an important step toward improving collaboration between legal and technical experts and advancing the automation of privacy assessments.

Tool Support for Privacy-Aware Reporting

This chapter presents the final contribution of this thesis: the design and implementation of tool support that directly assists stakeholders in privacy-aware reporting (*Contribution 4*). Building on the analytical capabilities of PRICE and Assessor View, this contribution addresses their lack of direct support for generating legally required documentation from software artifacts. We therefore extend both tools with reporting-oriented functionality that assists stakeholders in producing and reviewing privacy-related documentation.

First, we present AutoPRICE (**A**utomatic **P**rivacy-**R**elated **I**ntput **C**lassification **E**ngine), a web tool built on top of PRICE that assists developers by automatically populating the DSS form. To evaluate the usability of AutoPRICE, we conduct a preliminary user study. Based on its findings, we introduce additional features that more directly support developers during DSS form completion and help users better understand and visualize the data collected by an application.

Next, we extend Assessor View with new features aimed at supporting privacy experts, in particular Data Protection Officers (DPOs). The enhanced Assessor View partially automates the generation of Record of Processing Activities (RoPA). We evaluate this extension through semi-structured interviews with six DPOs to examine how the the RoPA functionality supports their workflows and reporting practices.

6.1 AutoPRICE for DSS Completion

AutoPRICE (**A**utomatic **P**rivacy-**R**elated **I**ntput **C**lassification **E**ngine) is a web tool that identifies the data collected by an Android app, and automatically guides Android developers in completing the DSS form.

We introduced PRICE in Section 3.3. PRICE was a command-line tool and was not designed to directly support developers in meeting reporting requirements. AutoPRICE enhances PRICE with a user-friendly interface and extends coverage by incorporating permission-to-data mappings required for DSS compliance. AutoPRICE

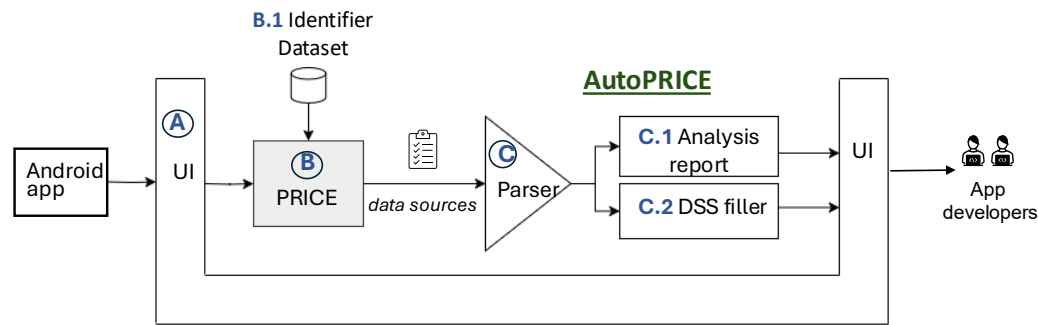


Figure 6.1.: Our approach built on top of PRICE (Module **B**) (cf. Section 3.3).

was designed specifically to address the key challenges identified in our empirical study (cf. Section 4.2.2), namely developers’ difficulty in understanding what data is collected by their apps, and how to accurately complete the DSS form.

It comprises three components: the **User Interface** (Module **A** in Figure 6.1), **PRICE** (Module **B** in Figure 6.1), and the **Parser** (Module **C** in Figure 6.1). Initially, the user interface accepts an Android APK as input, and forwards it to PRICE for further analysis. PRICE is an existing component that statically analyzes the app to detect and label privacy-related data sources in the sources code (cf. Section 3.3). Subsequently, the Parser accepts these data sources as input and translates them into graphical representations familiar to developers.

A User Interface. The *User Interface* accepts an APK as input and feeds it to PRICE. Once AutoPRICE has been executed, the UI displays its results on the dashboard (cf. Figure 6.2).

The dashboard gives a high level overview of all risky sources detected in the source code, manifest permissions, and the UI screens of an app—directly addressing developers’ lack of visibility into data collection mechanisms (cf. Section 4.2.2). It also allows users to compare the data collected by their app with that of other popular apps in the same domain, leveraging PRICE’s results from 700 previously analyzed domain-specific apps (cf. Section 3.4.2).

The dashboard also links to a pre-filled DSS form (cf. Figure 6.3), providing structured guidance on data categories and types identified during the analysis. Each data type entry includes a *Show Details* button that, when clicked, expands to reveal more information about why that data type should be declared. This includes the sources (e.g., methods, permissions, keywords) from which the data was collected and specific details such as method names or permissions used (③ in Figure 6.3). This feature aims to help developers understand *what* data their app collects, *how*

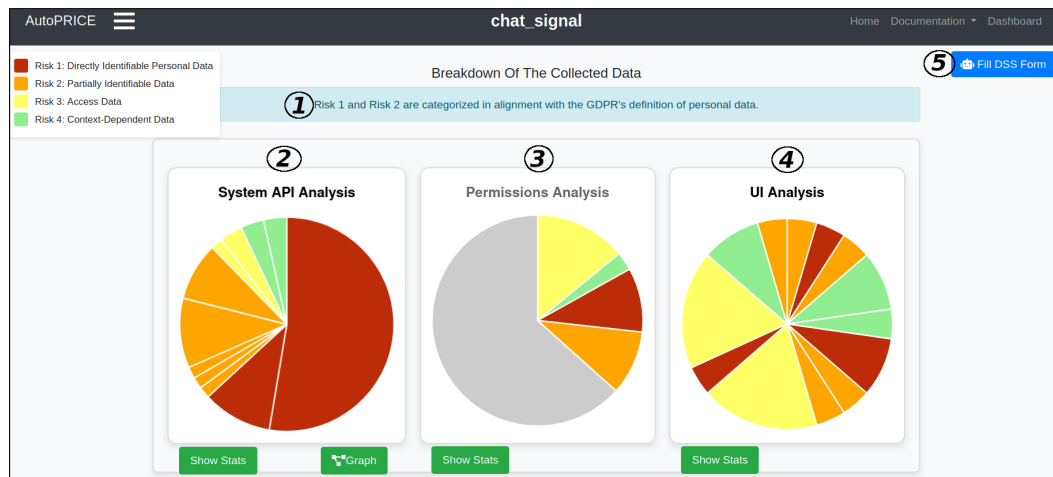


Figure 6.2.: AutoPRICE dashboard for Signal [22k], showing ① legends to identify privacy-related data and GDPR's personal data, ② data collected via system APIs, ③ data collected via permissions, ④ data collected via app UI, and ⑤ button to automatically populate the *Data collected* part of the DSS form.

it collects this data, and *why* it needs to be reported—targeting the DSS comprehension gap (cf. Figure 4.4 in Section 4.2.2). Note that app developers still need to manually fill out the purpose of data collection (3. in Figure 2.5b).

④ **PRICE.** *PRICE*, first introduced in Section 3.3, is a command-line tool that uses a manually annotated *Identifier Dataset* (B.1) to label data collected by an app as privacy-related data sources.

This dataset uses a risk-based definition of privacy-related data. It includes privacy-related keywords for UI analysis, and system API calls from Android, Java, and *popular third-party libraries* such as AppLovin [22b], AppsFlyer [26b], Firebase [25h], and Mopub [25i]. In this dataset, personal data that can identify an individual without the use of any additional information is categorized as **risk 1**, while data requiring additional information for identification is classified as **risk 2**. This analyzer is particularly well-suited to DSS support because its data types and categories match those in the DSS form, and it captures data collected through third-party libraries—a key challenge identified in Section 4.2.2. Furthermore, it incorporates elements of the GDPR's definition of personal data (① in Figure 6.2), which enhances its utility for developers targeting users in the European Union—an important consideration for the 56% of surveyed participants who publish their apps in the EU (cf. Section 4.2.2).

Figure 6.3.: A part of the pre-filled DSS form for Signal [22k], showing ① data category, ② data type, and ③ explanation on how the data type has been collected.

🔍 **Parser.** The *Parser* reads and interprets PRICE’s output. It generates an analysis report (C.1), which is displayed to the user through the dashboard (cf. Figure 6.2). The Parser also triggers the DSS filler component (C.2), which results in the pre-filled DSS form (cf. Figure 6.2).

6.1.1 Early Experiment

In this section, we discuss the preliminary user study we conducted to address the following research question: *How can AutoPRICE support app developers in understanding and disclosing the data collected by their apps?*

To answer the research question, we conducted a within-subjects user study [23a] involving 8 participants. We initially attempted to recruit app developers who had participated in our earlier survey (cf. Section 4.2.2). In line with the GDPR’s data minimization principle [181], we made the provision of contact details optional, and only 7 out of 41 participants shared them in the survey. Unfortunately, none of these developers responded within the available timeframe. As a result, and given that students are considered reasonable proxies for software developers in usability studies [Ber04; SMJ15; SAW08], we recruited Computer Science students instead. Prior to the main study, we carried out a pilot study with 4 participants to identify and resolve potential technical issues. The final user study involved a different set

of 8 participants, all of whom were Computer Science students and distinct from the 41 survey respondents mentioned in Section 4.2.2. We refer to our user study participants as P01–P08.

Study Procedure. At the beginning of the user study, participants were given all the information about the study, and were asked to sign the *informed consent* form. Then, participants rated their experience on a Likert scale from 1 (Beginner) to 5 (Expert) (cf. Table 6.1).

Participants were divided into two groups:

- P01–P04 received a short, structured introduction to AutoPRICE’s key features and workflow.
- P05–P08 did not receive any demonstration, simulating a first-contact scenario in which users approach the tool solely through its interface and on-screen guidance.

Participants then accessed AutoPRICE through a web server. Each participant was assigned one of the following popular apps: Signal [22k], Telegram [26i], and Instagram [26g], and was asked to complete two tasks:

Task 1: Analyze the data collected by the app, and observe how this data compares with data collected by other apps in the same domain. This task required close interaction with the dashboard and its features.

Task 2: Use AutoPRICE to automatically complete the *data collected* part of the DSS form. Participants were then asked to manually locate the app’s Data Safety Section (DSS) on the Google Play Store, compare the two outputs, and report their findings.

These tasks were designed to evaluate whether AutoPRICE effectively supports developers in understanding how their apps collect personal data and whether that information is disclosed appropriately (RQ).

After these tasks, participants answered several 5-point Likert-type questions from the System Usability Scale (SUS) [Gri+13], a questionnaire designed to measure the effectiveness of a system, and rated AutoPRICE using a Net Promoter Score (NPS) [Rei04], that measures their likelihood of recommending the tool to a friend. The final section of the study gathered open-ended feedback about participants’ experiences with AutoPRICE. The full questionnaire is available in Appendix D.

Table 6.1.: Summary of participant experience. Filled circles indicate expertise from 1 (Beginner) to 5 (Expert).

ID	Android app development	Java development	Data privacy	GDPR
P01	●●○○○	●●●●○	●●●●○	●●●○○
P02	●○○○○	●●●○○	●●●○○	●●○○○
P03	●●○○○	●●●○○	●●●○○	●●●○○
P04	●○○○○	●●○○○	●●○○○	●○○○○
P05	●○○○○	●●●○○	●●○○○	●●○○○
P06	●○○○○	●○○○○	●●○○○	●●○○○
P07	●○○○○	●●●●○	●●●○○	●○○○○
P08	●○○○○	●●○○○	●○○○○	●○○○○

Participant Experience. Participants rated their experience on a Likert scale from 1 (Beginner) to 5 (Expert) (cf. Table 6.1). They reported familiarity with Android app development (median 1/5), Java development (median 3/5), Data privacy (median 2.5/5), and GDPR (median 2/5).

Ethics and Data Protection. The user study design was approved by the Ethics committee of our institution. Our consent form and privacy policy were approved by the Data Protection Office of our institution.

Study Results

We next share the results of the user study in the context of the research question.

Understanding data collection. In Task 1, all participants could identify the data collected by their app. They were then asked to classify the app’s data collection as *safe* (collects very little data), *moderate* (collects an appropriate amount of data), or *unsafe* (collects too much data). Their responses provided insight into participants’ individual perceptions of acceptable data practices. Among those who analyzed Instagram, three out of four participants rated its data collection as *unsafe*, while one rated it as *moderate*. Of the three participants who worked with Signal, two deemed it *unsafe* and one considered it *moderate*. P04 examined Telegram and categorized its data collection as *moderate*.

When asked about their confidence in using AutoPRICE, only P06 expressed low confidence, citing confusion as a beginner. While 4 participants reported feeling confident, 3 participants felt somewhat confident. P07 commented, “*AutoPRICE gives detailed info on which APIs or UI are accessing what kind of personal information. I can now see that Instagram has access to risky data which is scary if my account get hacked.*”

We noticed that participants faced difficulties in comparing the data collection practices of their assigned app to those of similar apps. This suggests that the comparative overview feature needs improvement in both visualization and presentation. P07, who analyzed Instagram said, *“Looks like most of the social media apps collect directly identifiable and partially identifiable data (location, personal info, and device data)”*.

Automatic DSS Generation. In Task 2, participants were asked to automatically generate the *data collected* part of the DSS form for a given app using AutoPRICE, and then report on their findings. All participants responded positively when asked if they agree with AutoPRICE’s DSS results. P01 commented, *“Very encouraging results.”* P03 remarked, *“Well descriptive and mapped.”* P06: *“The findings are very elaborate and extensive. The only issue it is not understandable by a common man.”* P08 said, *“Accurate enough, I agree with the DSS results.”*

Participants were next asked to locate the Data Safety Section (DSS) from the Google Play Store for the same app and compare it with the DSS generated by AutoPRICE. All participants observed notable differences between the two. While four participants initially struggled to locate the Play Store’s DSS, they were able to compare it effectively once found. Two participants were able to locate and compare it with ease. P05 observed, *“Google Play Store DSS lists only phone number (for Signal). AutoPRICE shows all the direct identifiable personal data and partial identifiable personal data information.”* P07 noted, *“Instagram is overreporting some of the permissions they are taking from the user’s device. AutoPRICE showed that Instagram is using PAN info, which was not listed in Google DSS.”*

Interestingly, prior exposure to AutoPRICE did not guarantee an easier experience. P01, who received an overview of AutoPRICE before the study, found it difficult to locate and compare the Play Store’s DSS with AutoPRICE’s DSS. In contrast, P06, who received no demonstration, could locate the Play Store’s DSS easily but had difficulty comparing it with AutoPRICE’s DSS.

Participant feedback emphasized the strengths and limitations of AutoPRICE and Google Play Store’s DSS: *“Google Play Store seems to lack many details.”* (P01) *“Graph visualization, categories of collected data, and easy-to-navigate documentation [are helpful in AutoPRICE].”* (P02) *“AutoPRICE’s findings are more in-depth and visible on the main screen. The DSS on Google Play Store was somewhat difficult to find.”* (P08)

Usability. Participants answered 5-point Likert-type questions from the System Usability Scale (SUS) [Gri+13]. We use this quantitative metric as an indicator. 80% of participants found AutoPRICE easy to use, and said they would like to use it for

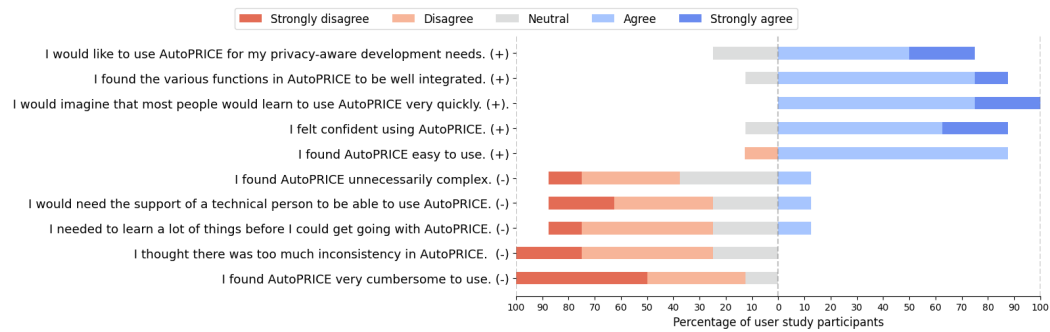


Figure 6.4.: User study participants’ responses to the usability questions. For (+) statements, *Strongly Agree* represents a more favorable response. For (-) statements, *Strongly Disagree* represents a more favorable response.

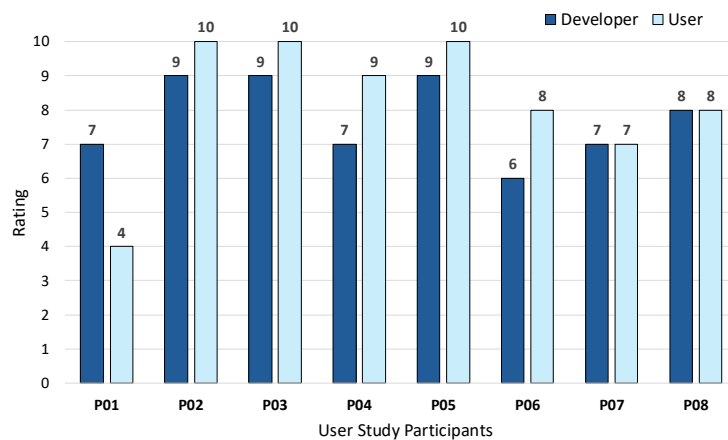


Figure 6.5.: AutoPRICE’s rating for developers and users. (0: not recommended, 10: highly recommended)

their needs. Based on their responses, the calculated SUS score for AutoPRICE was 72.5, exceeding the industry benchmark of 68 and suggesting acceptable usability. Despite their limited experience with Android (cf. Table 6.1), participants reported positive experiences with AutoPRICE, suggesting that it is accessible and usable even for novice developers with low Android expertise. Figure 6.4 presents their responses.

The Net Promoter Score (NPS) [Rei04] for AutoPRICE was calculated as 25 for app developers, and 37.5 for app users, reflecting a positive perception among participants (individual ratings in Figure 6.5). P01 explained why they would not recommend AutoPRICE to users: “Normal users hardly bother with these things, only certain individuals are concerned about where their data ends up.” P03 observed, “Android developers integrating any open source SDK could use AutoPRICE for permission anal-

ysis.” P05, who would recommend AutoPRICE to both developers and users, said, “DSS has important information which app developers should take into consideration before publishing an app. Developer can take AutoPRICE’s features and incorporate them into Play Store’s DSS. Every user could verify new app in AutoPRICE before installing it.” P07 said, “To be honest, most users do not care to even look at privacy-related information. AutoPRICE provides quite detailed info, which is more helpful for developers while they develop their apps.” P08 said they would recommend AutoPRICE to app developers for the *risk classification of the data*, and to users because it is *easy to understand*.

Although the average SUS scores remained comparatively high for all participants irrespective of whether they received a brief overview of AutoPRICE, the NPS results are different. While participants who completed their tasks without any overview (P05–P08) took a neutral stance on AutoPRICE (NPS = 0), participants in the other group were more inclined to endorse AutoPRICE to both developers (NPS = 20) and users (NPS = 50). This indicates that a brief orientation can enhance participants’ inclination to recommend AutoPRICE.

General feedback. The final section of the user study had some open questions asking for feedback. All participants agreed that AutoPRICE gave them a better understanding of app data collection. P07 commented, “*I would prefer the visual view that AutoPRICE provides over textual content on Google Play Store.*” Participants generally regarded AutoPRICE as an *understandable tool* (P06) that was *easy to use* (P03), and gave *more transparency about the actual hidden data collected by apps* (P01). P02 commented, “*Now I know how much data is collected by the big apps in Play Store.*”

Participants offered valuable suggestions for improving AutoPRICE. P01, P03, P05, and P08 recommended improving the user interface to ease navigation and usage. While P02 appreciated the documentation provided, P03 suggested using simpler notations to benefit novice users. P01 suggested adding hints and tips for beginners. P04 recommended adding a score to grade an app based on its collected data, and suggested adding a short summary of the results on the dashboard.

Finding. AutoPRICE effectively aids developers in identifying collected data and automatically generating the *data collected* part of the DSS form.

6.1.2 Feature Extensions

The early experiment conducted in Section 6.1.1 indicated that further usability improvements are necessary to enhance comprehension of the analysis results and to provide a more detailed and transparent DSS form completion process.

To clearly distinguish between end-user transparency and the information required by developers to meet reporting obligations, we designed two separate interfaces: a Developer View (cf. Figure 6.6a) and User View (cf. Figure 6.6b).

AutoPRICE's Developer View details insights into the data collected by the application and redirects developers to automatically populate the DSS form (① in Figure 6.6a). It also allows them to inspect, on an individual basis, the data collected via third-party library system API calls and permissions. This functionality is described in more detail below. In contrast, the User View deliberately limits the level of detail: it presents only the categories of collected data and offers users the possibility to consult the DSS generated by AutoPRICE.

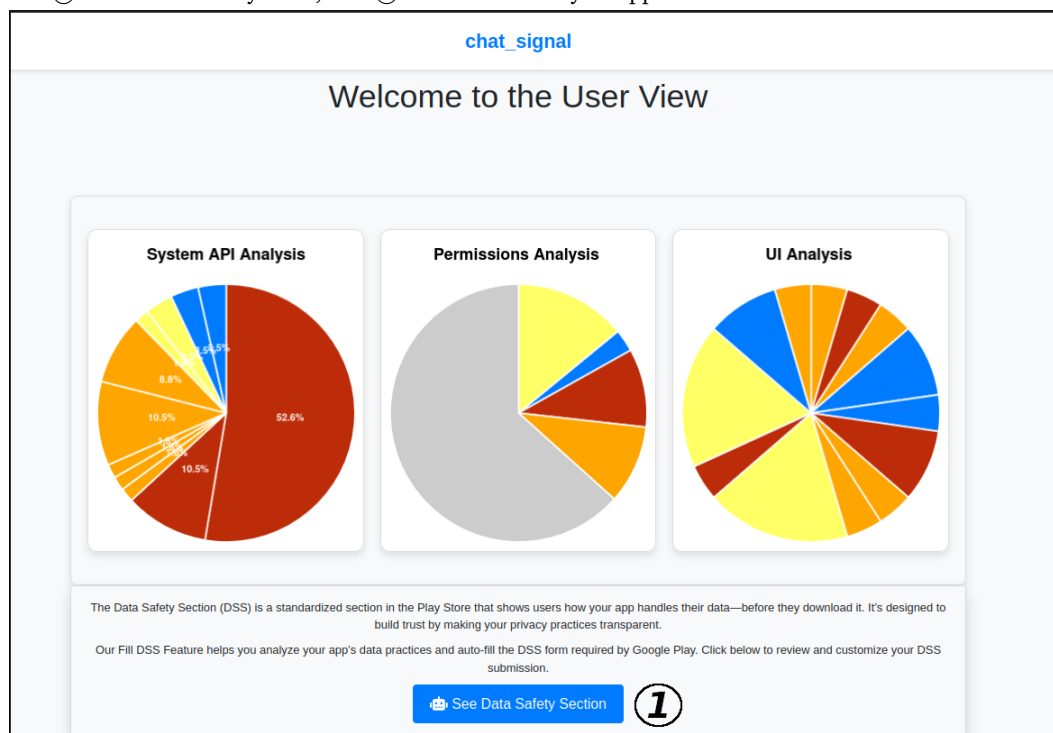
As discussed in Section 4.2.2, one of the main challenges developers face is identifying and reporting data collected through third-party libraries. To address this issue, we introduced two dedicated views that separate data collected by third-party libraries (cf. Figure 6.7a) from data collected through in-app methods (cf. Figure 6.7b). These views summarize the TPL system API methods and permissions used by the application and are accessible from the Developer dashboard (② and ③ in Figure 6.6a).

To further support the completion of the DSS form, we extended the DSS filler component. Developers can now select data processing purposes from a predefined list specified by Google (cf. Table 2.3) and indicate which categories of collected data are shared (cf. Figure 6.8). They can also select which collected data has been shared with third-party libraries, thereby supporting completion of both the *data collected* and *data shared* section of the DSS (① and ② in Figure 2.5a). In addition, we implemented an automated comparison between the DSS generated by AutoPRICE and the corresponding Play Store DSS of the application (cf. Figure 6.9), enabling developers to identify discrepancies more efficiently.

Overall, these extensions refine AutoPRICE from an analysis-output interface into a more structured decision-support tool for DSS completion. By separating developer-oriented details from user-facing transparency information, distinguishing third-party library data collection from in-app data collection, and supporting comparison with existing Play Store DSS disclosures, AutoPRICE provides developers with more actionable information for identifying and reporting privacy-related data practices.

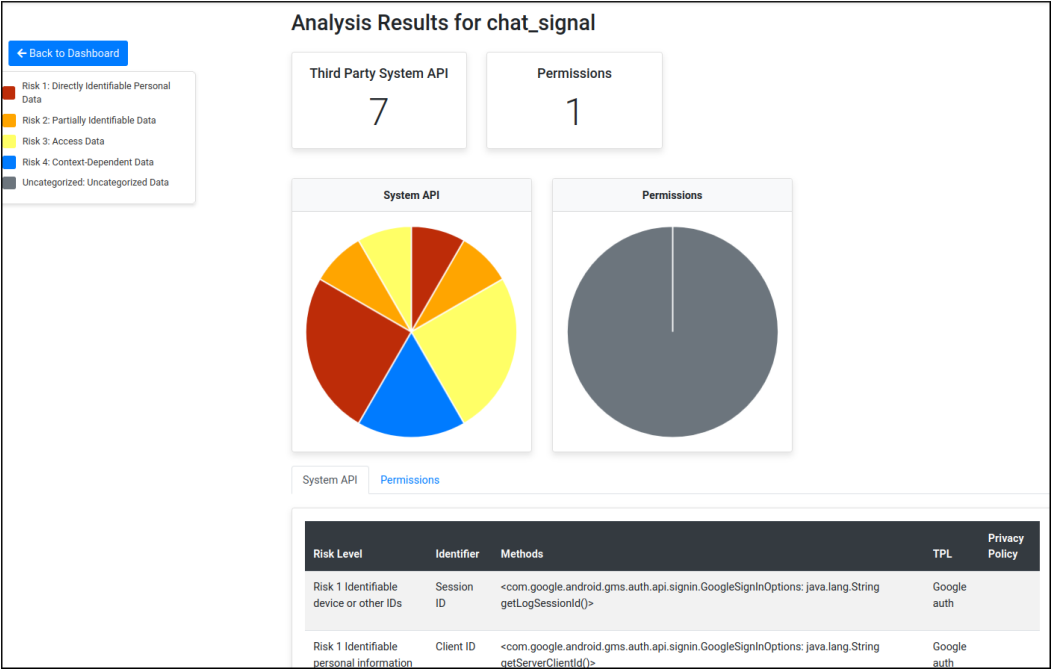


(a) Developer dashboard for Signal [22k], showing ① button to automatically populate the DSS form, ② data collected by TPLs, and ③ data collected by in-app methods.

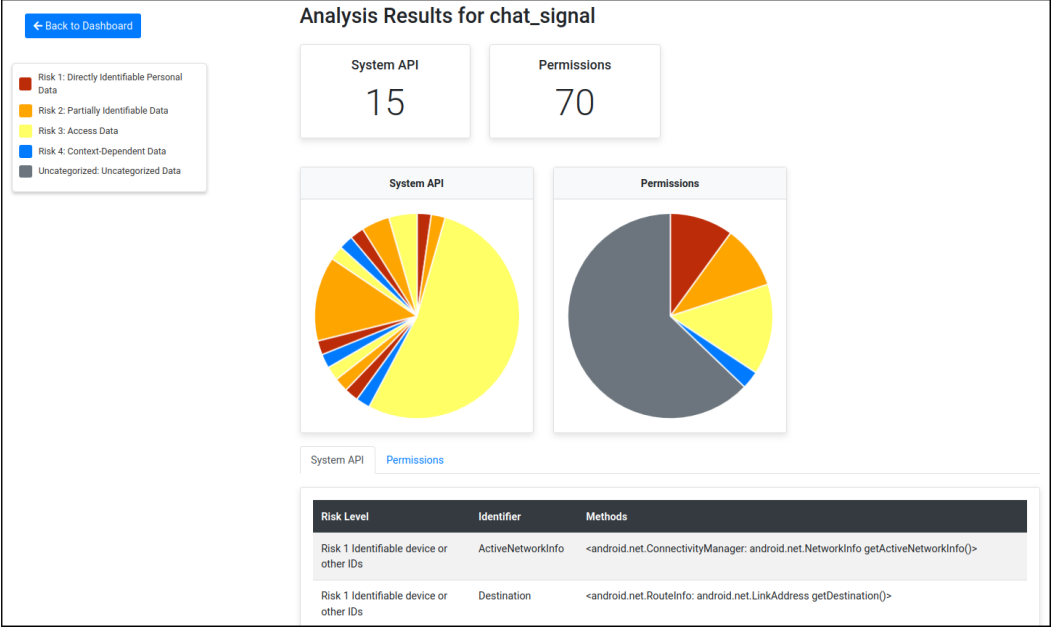


(b) User dashboard for Signal [22k], showing ① button to view the DSS.

Figure 6.6.: Different interfaces for the dashboard.



(a) Illustration of the data collected by TPLs.



(b) Illustration of the data collected by in-app methods.

Figure 6.7.: Information about *where* data is collected from: third-party libraries or in-app methods.

1 Data Collected 2 Data Shared

This table shows data collected from the APK with permissions and purposes—download the full table or select rows and click Next to continue to the Data Shared View and include them in your report.

Next
1
2
3
4

Select	Category	Data Type	Data Collected from APK			Purpose
			Methods	Permissions	Data Collected from UI	
☐	App activity	App interactions	<android.media.session.MediaController: android.media.Metadatadata getMetadata()> <android.app.Activity: android.view.View findViewById(int)> <android.content.Intent: java.lang.String getAction()>			Select purpose(s) App functionality Analytics Developer communications Advertising or marketing Select purpose(s)
☐		Other actions		android.permission.download_without_notification android.permission.access_notification_policy android.permission.receive_boot_completed android.permission.vibrate android.permission.wake_lock android.permission.foreground_service android.permission.read_sync_settings android.permission.write_sync_settings android.permission.install_shortcut com.android.launcher.permission.install_shortcut android.permission.set_wallpaper android.permission.bluetooth android.permission.use_full_screen_intent		Select purpose(s)
☐		Other action		android.permission.request_install_packages		Select purpose(s)

Figure 6.8.: Enhanced DSS filler component, showing ① data category, ② data types, ③ explanation on how the data type has been collected, and ④ editable purposes of data collection (cf. Table 2.3).

Fetch Data Safety Section

Enter the Play Store URL, here

Fetch ①

Data from AutoPrice Analysis ②

Data collected:

1 App activity

2 App info and performance

3 Audio files

4 Calendar

5 Contacts

6 Device or other IDs

7 Files and docs

8 Location

9 Messages

10 Personal info

11 Photos and videos

12 Web browsing

Data Safety Section from the playstore ③

Data collected:

Personal info

Summary

Matching data categories: 1

Underreported data categories: 11

Overreported data categories: 0

Matching Data Categories:

Personal info

Underreported:

App activity

App info and performance

Audio files

Calendar

Contacts

Device or other IDs

Files and docs

④

Figure 6.9.: Comparison between the DSS generated by AutoPRICE and the corresponding Play Store DSS of Signal, showing ① manually entered URL of the Play Store DSS, ② DSS filled by AutoPRICE, ③ DSS extracted from the given link, and ④ a summary of under-reporting, over-reporting or other observations (cropped partially due to space constraints).

These additions address the main usability issues observed in the early experiment (cf. Section 6.1.1) and provide a foundation for more systematic support of DSS compliance tasks. A large-scale interview-based study with Android developers is planned as future work to further investigate the practical applicability of these extensions in real-world development workflows.

6.1.3 Limitations and Threats to Validity

AutoPRICE extends PRICE by providing a front end for its static analysis workflow. Consequently, AutoPRICE inherits the technical limitations of the underlying analysis engine, PRICE. In particular, our *Identifier Dataset* may not include all third-party libraries. Consequently, AutoPRICE’s results can be affected by incomplete identification of data sources, particularly in cases where third-party APIs are absent from the dataset. Expanding or updating the dataset is therefore an effective mitigation strategy. Nevertheless, we aim for a comprehensive evaluation by incorporating the most widely used third-party libraries across different categories. Since AutoPRICE relies on keyword matching to detect data sources—including those from certain third-party libraries—it cannot handle obfuscation scenarios in which API calls are renamed. In such cases, adapting the dataset remains a viable mitigation approach.

To evaluate the usability of AutoPRICE, we conducted a preliminary user study. Due to difficulties in recruiting professional developers, we involved students, who are commonly considered reasonable proxies for software developers [Ber04; SMJ15; SAW08]. Although the number of participants was relatively small, the qualitative feedback, together with the SUS and NPS results, still provides meaningful insights. The study was conducted in a controlled environment, which may not fully reflect real-world usage by professional developers. In practice, developers would likely have more time to examine applications in depth. Therefore, the findings should be interpreted as preliminary evidence of usability rather than as a complete assessment of AutoPRICE in real-world development settings. To address this limitation, a large-scale interview study with Android developers is part of our future work.

6.1.4 Related Work

Prior studies evaluating the accuracy of DSS disclosures [Kha+23; 23b] were conducted when DSS-assistive tools were either unavailable or too immature for practical evaluation, leaving open questions regarding their influence on developers’ reporting practices. Since then, Google has introduced Checks [CH22], a paid ser-

vice that supports developers in completing the DSS, alongside open-source tools such as Matcha [Li+24] and Privado.ai’s data safety report builder [22i]. Matcha analyzes application source code and suggests DSS labels, relying on developer annotations to finalize disclosures, while Privado generates DSS forms from open-source projects via static analysis.

However, these tools depend on access to source code and are therefore limited to open-source projects. In contrast, AutoPRICE operates directly on compiled APKs, enabling the analysis of proprietary and published applications even when source code is unavailable. This capability allows for auditing a broader range of real-world apps and enabled us to provide feedback to a large financial institution, demonstrating AutoPRICE’s practical value beyond the scope of existing DSS-assistive tools.

6.2 Extending Assessor View for RoPA Generation

To assess the viability of Assessor View, we conducted an interview-based user study with 16 privacy experts, focusing on its potential use in privacy assessments (including DPIAs [18d]) and GDPR compliance (cf. Section 5.2.2).

In the study, most participants reported relying heavily on manual effort to prepare GDPR-related documents. Ten of the sixteen participants mentioned using standard templates and spreadsheets to complete GDPR documentation, though one noted the limitations of generic templates: *“We can’t just give them (clients) a template to begin with, right? It has to be customized to what they do.”* Fourteen participants identified Record of Processing Activities (RoPA) [18q] as the most important document required for GDPR compliance, often used as starting point for DPIAs.

Article §30 [18i] of the GDPR requires organizations to document how personal data is collected, stored, and processed, including details such as processing purposes, data categories, recipients, and retention periods. This documentation, known as the Record of Processing Activities (RoPA), serves as both a compliance record and a foundation for conducting DPIAs. As a legal researcher explained: *“Completing the RoPA is a data mapping exercise which allows stakeholders to envisage potential threats.”* Participants recommended aligning the tool’s output with the structure of a RoPA. For example, one suggested: *“(The tool can) help fill RoPA using static analysis. If you get data in a RoPA template, DPOs would be very happy.”*

A recent noyb.eu survey of 510 Data Protection Officers and privacy professionals [26d] found that over half of respondents believe that completing the RoPA consumes a significant to enormous amount of their compliance workload—more

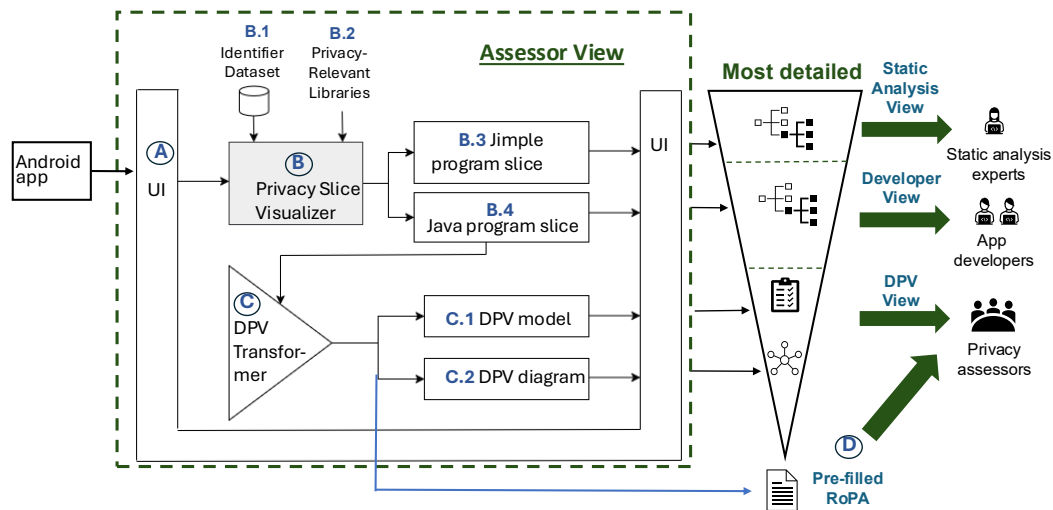


Figure 6.10.: Enhanced Assessor View architecture.

than any other GDPR requirement. DPIA documentation and RoPA is created for and by multiple stakeholders, yet it must remain consistent with the source code. Despite its central role in GDPR compliance, two key research gaps remain: (i) the need to better understand and support collaboration and communication among diverse stakeholders involved in DPIA activities; and (ii) the absence of tools that can accurately generate reporting documents directly from software artifacts.

While the initial prototype of Assessor View addressed the first research gap by providing tailored visualizations for different stakeholders, it did *not* address the second research gap—namely, the need to *accurately generate GDPR reporting documents*. This limitation and the direct feedback from participants motivated the design of the new RoPA extension, which we introduce in the following section.

6.2.1 Feature Extensions

Based on the feedback from the interview-based study conducted with 16 privacy experts (cf. Section 5.2.2), we collected improvement options and prioritized new features to extend Assessor View with RoPA support. This led to an enhanced design of Assessor View (cf. Figure 6.10).

We implemented a customizable RoPA feature (Module **D** in Figure 6.10), which automatically generates a RoPA tailored to the structure of the analyzed APK, instead of relying on generic templates. This aligns with the practices of the ten participants who indicated using templates in their work.

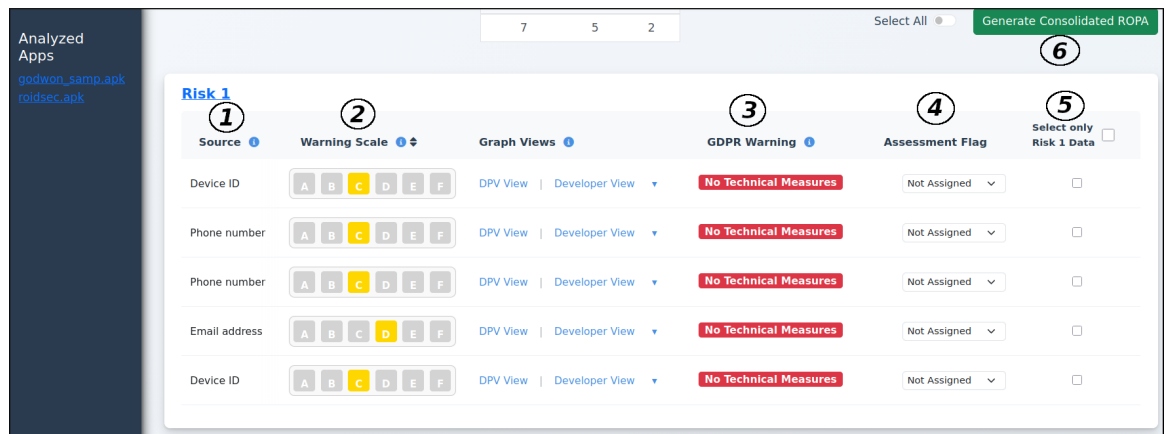


Figure 6.11.: Assessor View dashboard for the demo app [25e], showing ① data sources, ② warning scale for developers, ③ warning scale for privacy experts, ④ assessment flags for collaboration, ⑤ selection of sources to generate RoPA, and ⑥ generation of RoPA.

We also extended the dashboard to better support collaboration between stakeholders (cf. Figure 6.11). We introduced two warning scales: one for developers (② in Figure 6.11) focusing on technical issues (e.g., third-party data sharing or missing encryption), and one for privacy experts and DPOs (③ in Figure 6.11), highlighting GDPR-specific risks (e.g., lack of data minimization or missing safeguards), derived from the Developer View and DPV View, respectively. We also added assessment flags (④ in Figure 6.11) for each program slice to help stakeholders track review status (e.g., “Addressed”, “Needs review”, or “Flagged”). To support informed decision-making, we included documentation explaining the warning categories, GDPR articles, DPV concepts, and potential technical measures.

Through the dashboard, stakeholders can select the data sources they wish to document (⑤ and ⑥ in Figure 6.11). For each selected data source, the generated RoPA identifies associated processing operations, with descriptions drawn from the DPV [Pan+24]. Assessor View supports two interchangeable formats: a table (cf. Figure 6.12a), and a form (cf. Figure 6.12b). Both include editable fields where DPOs can specify the business purpose of processing (① in Figure 6.12a and Figure 6.12b), indicate whether special category data is involved (② in Figure 6.12a), and add custom remarks (③ in Figure 6.12a). DPOs can switch between views, edit content, and export the output as a printable PDF—a feature one participant highlighted as useful for communication with supervisory authorities. The form view also allows stakeholders to check off completed sections and monitor progress via a progress bar (cf. Figure 6.12b).

ROPA Table							
Personal Data	Process Activity	Description of Processing Activity	Third Party Libraries	Purpose	GDPR Articles	Special Categories of Data?	DPO Remarks
Device ID	Collect	To gather data from someone.	Not Found	Enter purpose for the processing activity here	§ 13 § 25	Is this a special category of Data?	Enter DPO remarks
	Copy	To produce an exact reproduction of the data.	Not Found	Enter purpose for the processing activity here			
	Store	To keep data for future use.	Not Found	Enter purpose for the processing activity here			
	Transfer	To move data from one place to another.	Not Found	Enter purpose for the processing activity here			

(a) A part of the RoPA table.

Personal Data Type: Device ID
25%

Processing Activities: Collect, Copy, Store, Transfer
25%

☒ Processing activity: Collect

Description: To gather data from someone.
Purpose: Enter purpose for the processing activity here
Third-Party Libraries: Not Found

☐ Processing activity: Copy

Description: To produce an exact reproduction of the data.
Purpose: 1
Third-Party Libraries: Not Found

☐ Processing activity: Store

Description: To keep data for future use.
Purpose:
Third-Party Libraries: Not Found

(b) A part of the RoPA form.

Figure 6.12.: RoPA formats for the demo app [25e], showing editable fields for ① the purpose of processing, ② special categories of data, and ③ DPO remarks (present in both formats but partially cropped in Figure 6.12b).

To evaluate these new features, we conducted a follow-up qualitative study with 6 returning participants from the original expert pool. The findings from this study are presented in the following section.

6.2.2 Evaluation: Semi-Structured Interviews

In this section, we discuss the study we conducted with Assessor View after implementing new features aimed at supporting privacy experts, especially DPOs. These include partially automated RoPA-generation, and a dashboard designed to facilitate communication between privacy experts and technical stakeholders. We evaluate these enhancements through semi-structured interviews with 6 privacy experts

from our original participant pool. Given the exploratory nature of our research, we employed a semi-structured interview approach to gather insights while allowing flexibility for participants to share their perspectives.

In this study, we aim to answer the following research question:

RQ. *To what extent does Assessor View support privacy experts in completing the Record of Processing Activities?*

This question focuses on the new features and assesses how well they can be used by DPOs.

Study Methodology

We contacted 6 privacy experts from the initial expert pool (cf. Section 5.2.2), particularly DPOs based in the European Union and United Kingdom. All of them accepted our invitation to participate. We next explain how we conducted the study.

Interview Process. We used Zoom for all interviews. All interviews were conducted by the author of this thesis in July 2025, and audio-recorded via Zoom’s functionality. We expected the interviews to take 20 to 30 minutes, and scheduled half-hour appointments with all interviewees. Interviews lasted between 18 to 35 minutes; the median duration was 26 minutes.

Interview Structure. At the start of each interview, we gave participants all the information about the study, and they signed the *informed consent* form. The interview script was organized into three sections. We began by asking about their current practices for completing the RoPA. Next, participants used Assessor View to automatically generate a RoPA for a given demo app with documented data flows. Finally, we asked questions about their experience with the tool—specifically, the usability, clarity, and reliability of the generated RoPA—and their impressions of the two RoPA formats (table and form) provided by Assessor View. A summary of the final interview protocol is presented in Table 6.2.

Participant Experience. Participants self-reported familiarity on a 5-point scale (1 = Beginner, 5 = Expert), reporting median scores of 5 for GDPR, GDPR compliance; 4.5 for DPIA; 4 for DPV; and 3 for data visualization (cf. Table 6.3). They have a median of 8.5 years of experience in privacy and/or law, with all holding additional privacy certifications. P02 holds a doctoral degree in Law.

Table 6.2.: Summary of interview protocol. The first column is the topic and the number of questions for that topic.

Topic (#)	Example questions
State-of-the-art (3)	How do you typically compile a RoPA in your current work? How often do you do this? Do you use any templates? Which ones?
Assessor View-generated RoPA (4)	What were your first impressions when using the RoPA feature in Assessor View? Do you think a partially pre-filled RoPA generated from source code would reduce manual workload during DPIA or pre-DPIA screening phases? Why or why not? Do you think Assessor View-generated partial RoPA output would be a useful starting point for documenting processing activities in compliance with GDPR Article 30? Would it make the process easier for beginners or external DPOs?
Usability, Feedback, and Suggestions (6)	Could you understand and interpret the RoPA output generated by the tool? Were there any parts of the output that you found unclear or ambiguous or that required additional validation? Which representation (table or form) have you used more often for completing the RoPA? Which would you prefer to communicate with developers? To what extent can a DPO rely on the RoPA output from Assessor View to meet legal obligations under Article 30? What changes or additions would make the RoPA feature more useful for you? What more would you like to see in Assessor View to better support RoPA generation?

Data Analysis. We recorded and transcribed all interviews using NoScribe [25m], then reviewed transcripts for errors. The author of this thesis used iterative open coding [CS90] to develop a codebook, and asked a second researcher to double-check the codes.

Ethics and Data Protection. The study design was approved by the Ethics committee of our institution. Our consent form, privacy policy, and data handling practices were approved by the Data Protection Office of our institution.

Study Results

We next share the results of the interviews. We refer to our participants as P01-P06.

Existing support for RoPA. All participants reported using standard templates for RoPA, including *checklists* (P02, P04) and *questionnaires* (P03). While many (P01, P02, P04, and P06) used *stakeholder interviews* to gather necessary information, P01 and P04 also relied on a *questionnaire-style tool* to guide the RoPA process.

Participants identified several challenges in completing a RoPA, such as *balancing abstraction layers* (P01), *time consuming manual process* (P02), and *difficulties in collecting necessary information* (P04).

Table 6.3.: Summary of participant experience. Filled circles indicate expertise from 1 (Beginner) to 5 (Expert).

ID	Job title	Country	Exp in privacy/ law (yrs)	Familiarity				
				Data Visuali- zation	GDPR	GDPR Compli- ance	DPIA	DPV
P01	Corporate DPO	Germany	9	●●●●○	●●●●●	●●●●●	●●●●○	●●○○○
P02	DPO	Germany	12	●○○○○	●●●●●	●●●●●	●●●●●	●●●●○
P03	DPO	Cyprus	7	●●●○○	●●●●●	●●●●●	●●●●●	●●●●●
P04	Data Protection Advisor	Germany	3.5	●●●○○	●●●●●	●●●●○	●●●○○	●●●●●
P05	External DPO	UK	15	●●●○○	●●●●●	●●●●●	●●●●●	●●●●○
P06	Privacy Counsel	Germany	8	●●●○○	●●●●○	●●●●○	●●●●○	●○○○○

Assessor View-generated RoPA. All participants considered the generated RoPA a *useful starting point* for documenting activities. Several (P01, P03, and P05) found it *easy to understand*, and many (P03, P04, P05, and P06) felt it could *reduce manual workload* during DPIAs or pre-DPIA screening. P01 said, “*In theory, it should start at this level. In practice, this is not possible because this granularity is missing. This would be the first representation I’ve seen of a concept that actually does the proper theory on designing processing activities.*” P03 noted, “*Maybe it could provide a good understanding of how and where data has been used, as opposed to just having to trust whatever the developers tell you.*” P04 and P06 emphasized its value in preparing for stakeholder interviews: “*Because you already have a lot of pre-information, we can make better questions for the colleagues*” (P04). P05 emphasized its usefulness for external DPOs without source code access, adding, “*The bigger plus to me would be that it reduces the chance of something being missed*”.

RoPA Formats. All but one participant (P04) preferred the table format (cf. Figure 6.12a) for documenting activities. P03 explained, “*It (table) provides a good view of what data you have in your codebase, and you can see everything at the same time.*” Most also favored the table for developer discussions, though P01 found the form view (cf. Figure 6.12b) more suitable for interviews: “*Form feels a lot more similar to other methodologies that you can use to talk through points.*”. P05 observed, “*I think it’s good to allow both options, because different people would work in different ways*”.

Other feedback. Participants suggested minor refinements to terminology and structure (P01, P04), including clarifying legal bases for data collection such as contracts or user consent (P02). They also proposed a future enhancement: including risk analysis to automatically determine whether a full DPIA is required (P04, P06).

Finding. The RoPA feature in Assessor View shows promise for improving interdisciplinary communication among stakeholders. It provides a good overview of the processed data, and has the potential to reduce manual workload of DPOs.

6.2.3 Limitations and Threats to Validity

Although Assessor View can generate slices for real-world apps, our preliminary experiments showed that these slices—often exceeding 100 nodes—quickly become too large for meaningful manual interpretation. This limitation also affects RoPA generation: if the underlying slices are too detailed, the generated RoPA outputs may become overly granular and difficult for privacy experts to inspect efficiently. Therefore, additional summarization techniques are needed to scale the approach to larger and more complex applications. This limitation reflects the prototype nature of the current implementation. Nevertheless, the RoPA generation feature serves as a useful exploratory tool for studying how automated analysis results can support privacy documentation.

As in the previous Assessor View study (cf. Section 5.2.2), we used TaintBench apps because they provide real-world Android applications with documented privacy-relevant data flows. This made them suitable for generating RoPA outputs and discussing the results with DPOs. However, TaintBench represents only a limited set of applications and may not capture the full complexity of Android apps in the broader ecosystem. Consequently, the findings should be interpreted as feedback on the usefulness of the RoPA generation feature in the context of TaintBench-based assessment tasks, rather than as evidence of its effectiveness across all real-world Android applications.

While our interview-based study is qualitative, we addressed generalizability concerns by recruiting a diverse participant pool for the previous study, through direct outreach to over 175 experts across 43 countries (cf. Section 5.2.2). The final participants in that study worked in privacy and law across 11 countries, including four outside the European Union. From this pool, we invited six Europe-based DPOs to participate in the RoPA-focused follow-up study. While this participant group is appropriate for evaluating GDPR-oriented RoPA generation, the sample size remains limited and may not capture the full range of organizational practices adopted by

DPOs in different contexts and compliance regimes. A larger study involving DPOs from a broader range of sectors and regulatory contexts would help further assess the generalizability and practical applicability of the RoPA generation feature.

6.2.4 Related Work

Several open-source static analysis tools [Arz+14; TS23; Zha+23] support developers in checking whether application code and privacy policies comply with data protection requirements. These tools are primarily developer-centric and offer limited support for the broader set of stakeholders involved in privacy compliance processes. Proprietary solutions such as Privado [22j] similarly automate privacy compliance checks through source code and data-flow analysis but remain focused on developer-facing outputs.

Other commercial platforms such as SonarQube [25s] concentrate on software quality and security analysis and do not support GDPR-specific requirements or the generation of privacy documentation such as Records of Processing Activities (RoPA). Conversely, compliance management platforms such as OneTrust [25o] focus on organizing regulatory documentation but do not perform technical code analysis.

In contrast to these approaches, Assessor View bridges the gap between technical analysis and compliance documentation by operating on compiled APKs and partially automating RoPA generation. Unlike proprietary auditing tools, Assessor View is designed to support privacy assessors directly and has been evaluated with DPOs, providing insights into how such technical outputs are interpreted and used in practice. To the best of our knowledge, no existing open-source tool integrates code-aware analysis with automated RoPA generation. Assessor View addresses this gap by enabling assessor-oriented, code-informed RoPA documentation.

6.3 Summary

In this chapter, we extended PRICE and Assessor View to support stakeholders more directly by generating the required privacy-related documentation: DSS form (cf. Section 2.6.1) and RoPA (cf. Section 2.6.2) respectively.

We introduced AutoPRICE, a web tool built on top of PRICE that assists developers by automatically populating DSS forms. A preliminary user study evaluating AutoPRICE's usability informed the design of new features that better support developers during the reporting process and improve the transparency and visualization of data collection practices for users.

Finally, we enhanced Assessor View to partially automate RoPA generation and to improve collaboration among different stakeholders. An interview-based evaluation with 6 DPOs suggests that the RoPA functionality provides a useful overview of processed data and constitutes a promising step toward enhancing interdisciplinary communication in the context of DPIA activities.

Conclusion and Future Directions

This thesis makes several contributions toward understanding and supporting privacy practices in Android applications, and toward improving tools for privacy assessments and reporting.

The first contribution, presented in Chapter 3, introduces *Privacy-Relevant Input Classification Engine (PRICE)*, a command-line tool that identifies privacy-related data collected by Android app user interfaces, source code, and Manifest permissions. PRICE also supports empirical analysis of data collection practices. A large-scale study of 7,500 Android apps reveals that personal and financial data are primarily collected through user interfaces, while device, location, and browsing data are mostly accessed via system APIs. Partially identifying personal data, such as age or postal codes, are among the most frequently collected types. Although system APIs consistently collect substantial amounts of privacy-related data, user interfaces display more diverse and application-specific collection patterns. A qualitative analysis of the 100 most popular apps in seven domains (*Messaging and Social Media*; *Banking and Finance*; *News and Entertainment*; *Sports and Games*; *Technology and Education*; *E-Commerce and Shopping*; *Health and Fitness*) further shows that Messaging and Social Media apps collect the most personal and authentication-related data directly from users, while Sports and Games apps collect the least via their user interfaces.

The second contribution, described in Chapter 4, investigates how Android developers report data collection practices in Google Play Store’s Data Safety Section (DSS). Using PRICE, we identify which data categories are consistently disclosed and highlight discrepancies between statically detected data collection and developer-reported information. Case studies of Signal and Instagram illustrate instances of *under-reporting* and *over-reporting*. An *empirical study* of developer practices shows that developers largely rely on manual methods to classify privacy-related data, frequently consult online resources, and often struggle to translate technical knowledge into DSS-compliant disclosures. Recurring challenges include difficulties in identifying privacy-related data, limited understanding of the DSS form, and concerns about app rejection due to mismatches with Google’s privacy requirements.

The third contribution, presented in Chapter 5, addresses the challenges faced by Data Protection Officers (DPOs), privacy experts, and legal professionals in conducting Data Protection Impact Assessments (DPIAs) and ensuring compliance. We design and implement *Assessor View*, a static analysis-based web tool that visualizes source code components using a data privacy vocabulary and offers multiple visualization modes tailored to both technical and legal stakeholders. Qualitative studies with *developers* and *privacy professionals* reveal key challenges in current privacy assessment practices, including knowledge gaps, poor communication between legal and technical teams, the lack of source code-based analysis tools, and the late involvement of privacy assessors. Hands-on sessions with *Assessor View* demonstrate that GDPR-related warnings and guidance are valuable to privacy professionals and that the tool effectively supports their assessment tasks. These findings suggest that *Assessor View* provides a promising first step toward improving collaboration and partially automating privacy assessment processes.

The fourth contribution, described in Chapter 6, extends *PRICE* and *Assessor View* to provide direct reporting support. *AutoPRICE*, a static analysis-based web tool, assists Android app developers in completing the DSS form by leveraging *PRICE* at the back end. Simultaneously, *Assessor View* is extended to help Data Protection Officers automatically generate Record of Processing Activities (RoPA), a legally mandated inventory documenting how personal data is processed. These extensions enable more accurate, efficient, and practical reporting, supporting multiple stakeholders in fulfilling GDPR documentation requirements.

Future Directions

While this thesis provides a first systematic investigation into the challenges faced by different stakeholders in privacy assessment and reporting, several directions remain for future work:

Scalable privacy analysis for real-world software systems. This thesis provides an initial systematic investigation into the challenges faced by developers, privacy experts, and auditors when assessing and reporting privacy-related data collection. A natural next step is to scale this work to larger software ecosystems and broader stakeholder groups. Future research can conduct larger empirical studies and extend the prototypes introduced in this thesis, *AutoPRICE* and *Assessor View*, to support scalable analysis pipelines and improved usability. In the longer term, these tools

could support privacy analysis not only for Android applications but also for other software platforms, helping bridge the gap between source code analysis and practical privacy compliance in modern software systems.

Improving precision and reliability of privacy analyses. This thesis builds on state-of-the-art static analysis techniques such as program slicing and taint analysis, adapted to identify privacy-related data flows in software. There is scope to further improve the precision and robustness of these analyses so that the results can more reliably support compliance documentation and privacy assessments. While legal responsibility ultimately remains with organizations, stronger analysis techniques can provide developers and privacy professionals with technically grounded insights into how data is collected and processed in practice.

Enhancing privacy by design using Integrated Development Environment (IDE) integrations. One promising direction is to shift privacy by design support earlier in the development workflow by embedding it directly into developers' IDEs. While this thesis focuses on analyzing compiled APKs, an IDE extension could draw on our curated *Identifier Dataset* to deliver real-time privacy feedback: flagging unsafe naming patterns, warning when sensitive data might be exposed, and guiding developers toward privacy-aware coding principles. By integrating Large Language Models (LLMs), such a plugin could automatically recognize existing identifiers in the local source code and even suggest which personal data elements need to be declared in the DSS or other GDPR-related documents. Such assistance would help teams catch privacy issues as they write code, ensuring that compliance and documentation keep pace with implementation. We are currently exploring this direction in early-stage collaborations.

Combining static and dynamic analysis for stronger guarantees. Static analysis provides broad coverage but may overapproximate program behavior, while dynamic approaches such as traffic analysis provide concrete observations of data transmission. Combining these perspectives offers a promising direction for improving confidence in privacy analyses. Future research can explore how static analysis results can be systematically compared with network traffic observations to better understand which privacy-related data is actually transmitted by applications. This line of work aims to quantify mismatches between potential and actual data flows and to develop methods for reconciling discrepancies.

We are currently exploring this direction in an ongoing Master’s thesis. The project investigates hybrid privacy analysis for Android apps by combining static analysis with dynamic traffic analysis. Although this work is outside the scope of this thesis, it illustrates a concrete continuation of the techniques and research questions developed here. In the long term, such hybrid approaches could support more precise privacy assessments, improve the accuracy of privacy disclosures, and contribute to measurable privacy assurance in deployed systems.

Privacy analysis for systems integrating Large Language Models (LLMs). Modern software increasingly integrates LLMs and external Artificial Intelligence (AI) services, introducing new privacy and security challenges. These integrations often create novel data flows between applications and third-party AI providers, complicating traditional analysis models. Future work can systematically study how LLMs are incorporated into Android development workflows and application architectures. This will enable fine-grained analysis of sensitive data flows through AI-enabled components. This research direction aims to develop analysis techniques that allow developers and auditors to reason about the privacy implications of AI-assisted systems, an increasingly urgent challenge as AI components become embedded in mainstream software.

Privacy-aware code generation with Large Language Models (LLMs). Beyond analyzing AI-integrated systems, future research can investigate whether large language models can generate software that is privacy-aware by design, rather than merely functionally correct. While prior research has evaluated LLM-generated code with respect to correctness and security vulnerabilities, little work has examined privacy properties or regulatory alignment. In the future, one can systematically evaluate whether LLM-generated Android code adheres to privacy best practices, including data minimization, appropriate permission usage, secure handling of sensitive inputs, and alignment with privacy disclosures. This research will combine static analysis with empirical large-scale evaluation of generated code. Understanding when and why LLMs succeed or fail at producing privacy-aware code will inform the design of next-generation AI-assisted developer tools that integrate automation while preserving strong privacy guarantees. We are currently exploring this direction in early-stage collaborations.

Replication Package

In this thesis, we have presented some tools that enable accurate and automatic privacy-aware reporting and privacy assessments. To enable other researchers to reuse, validate, and extend these contributions we provide all the implementations and the experiments conducted on them as open source artifacts.

8.1 Tool 1: Privacy-Relevant Input Classification Engine (PRICE)

The source code of PRICE presented in Section 3.3 is available in the replication package for this PhD thesis: <https://doi.org/10.5281/zenodo.19921018>. The repository contains the Identifier Dataset introduced by us and detailed instructions on how to use Privacy-Relevant Input Classification Engine (PRICE).

PRICE: Experiments

The experiments conducted on PRICE presented in Section 3.4 and Section 4.1 are available at <https://zenodo.org/records/14046829>. The repository contains all APK files used for the experiments along with the output of both experiments.

8.2 Empirical Study: Challenges Faced by Developers

All artifacts of the empirical study conducted in Section 4.2 are available at <https://doi.org/10.5281/zenodo.17416187>. These include the developer survey questions and anonymized responses, as well as artifacts related to the analysis of online developer discussions.

8.3 Tool 2: Assessor View

The source code of the final version of Assessor View, presented in Section 6.2, is available in the replication package for this PhD thesis: <https://doi.org/10.5281/zenodo.19921018>. Intermediate versions of the tool are available in the artifacts accompanying the individual papers [Khe+25; Khe+26a].

Assessor View: Developer Study

The developer study presented in Section 5.2.1 is available at <https://doi.org/10.5281/zenodo.15487436>. This artifact also contains the implementation, sanity checks performed on Assessor View, and questionnaires and anonymized answers from the developer study conducted on Assessor View.

Assessor View: Privacy Experts Study

The study with privacy experts presented in Section 5.2.2 is available at <https://doi.org/10.5281/zenodo.15085611>. This artifact also contains the implementation, sanity checks performed on Assessor View, and questionnaires and anonymized answers from the interview-based study conducted on Assessor View.

Assessor View: Final RoPA study with Privacy Experts

The semi-structured interviews conducted with Assessor View presented in Section 6.2.2 are available at <https://doi.org/10.5281/zenodo.16151588>, and the demonstration video is available at <https://youtu.be/rVwJDHLj7Pw>.

8.4 Tool 3: AutoPRICE

The source code of the final version of AutoPRICE presented in Section 6.1 is available in the replication package for this PhD thesis: <https://doi.org/10.5281/zenodo.19921018>.

*But I have promises to keep,
And miles to go before I sleep. . .*

— Robert Frost

Bibliography

- [19a] *A system for dynamic slicing and program visualization*. 2019 (cit. on p. 113).
- [ALR21] Khaled Ahmed, Mieszko Lis, and Julia Rubin. “Mandoline: Dynamic Slicing of Android Applications with Trace-Based Alias Analysis”. In: *14th IEEE Conference on Software Testing, Verification and Validation, ICST 2021, Porto de Galinhas, Brazil, April 12-16, 2021*. IEEE, 2021, pp. 105–115. DOI: 10.1109/ICST49551.2021.00022 (cit. on p. 113).
- [All+16] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. “AndroZoo: collecting millions of Android apps for the research community”. In: New York, NY, USA: Association for Computing Machinery, 2016. DOI: 10.1145/2901739.2903508 (cit. on p. 31).
- [And+17] Benjamin Andow, Akhil Acharya, Dengfeng Li, et al. “UiRef: analysis of sensitive user inputs in Android applications”. In: *Proceedings of the 10th ACM Conference on Security and Privacy in Wireless and Mobile Networks*. WiSec ’17. Boston, Massachusetts: Association for Computing Machinery, 2017, pp. 23–34. DOI: 10.1145/3098243.3098247 (cit. on pp. 29, 39).
- [Ant+23] Alessia Antelmi, Gennaro Cordasco, Daniele De Vinco, and Carmine Spagnuolo. “The Age of Snippet Programming: Toward Understanding Developer Communities in Stack Overflow and Reddit”. In: *Companion Proceedings of the ACM Web Conference 2023*. WWW ’23 Companion. Austin, TX, USA: Association for Computing Machinery, 2023, pp. 1218–1224. DOI: 10.1145/3543873.3587673 (cit. on pp. 51, 65, 66).
- [Arz+14] Steven Arzt, Siegfried Rasthofer, Christian Fritz, et al. “FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps”. In: *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*. PLDI ’14. Edinburgh, United Kingdom: Association for Computing Machinery, 2014, pp. 259–269. DOI: 10.1145/2594291.2594299 (cit. on pp. 2, 9, 24, 39, 114, 139).
- [Azi+19] Tanzirul Azim, Arash Alavi, Iulian Neamtiu, and Rajiv Gupta. “Dynamic slicing for Android”. In: *Proceedings of the 41st International Conference on Software Engineering*. ICSE ’19. Montreal, Quebec, Canada: IEEE Press, 2019, pp. 1154–1164. DOI: 10.1109/ICSE.2019.00118 (cit. on pp. 10, 113).
- [BBD16] Michael Backes, Sven Bugiel, and Erik Derr. “Reliable Third-Party Library Detection in Android and its Security Applications”. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’16. Vienna, Austria: Association for Computing Machinery, 2016, pp. 356–367. DOI: 10.1145/2976749.2978333 (cit. on p. 73).

- [BE94] T. Ball and S.G. Eick. “Visualizing program slices”. In: *Proceedings of 1994 IEEE Symposium on Visual Languages*. 1994, pp. 288–295. DOI: 10.1109/VL.1994.363606 (cit. on p. 113).
- [Ber04] P. Berander. “Using students as subjects in requirements prioritization”. In: *Proceedings. 2004 International Symposium on Empirical Software Engineering, 2004. ISESE '04*. 2004, pp. 167–176. DOI: 10.1109/ISESE.2004.1334904 (cit. on pp. 87, 92, 120, 130).
- [Cap+23] Davide Caputo, Francesco Pagano, Giovanni Bottino, Luca Verderame, and Alessio Merlo. “You Can’t Always Get What You Want: Towards User-Controlled Privacy on Android”. In: *IEEE Transactions on Dependable and Secure Computing* 20.02 (Mar. 2023), pp. 975–987. DOI: 10.1109/TDSC.2022.3146020 (cit. on p. 73).
- [CB14] Victoria Clarke and Virginia Braun. “Thematic Analysis”. In: *Encyclopedia of Critical Psychology*. Ed. by Thomas Teo. New York, NY: Springer New York, 2014, pp. 1947–1952. DOI: 10.1007/978-1-4614-5583-7_311 (cit. on pp. 50, 65, 98, 112).
- [CS90] Juliet Corbin and Anselm Strauss. In: *Zeitschrift fuer Soziologie* 19.6 (1990), pp. 418–427. DOI: doi:10.1515/zfsoz-1990-0602 (cit. on pp. 50, 65, 98, 112, 136).
- [Dem+18] Levent Demir, Amrit Kumar, Mathieu Cunche, and Cédric Lauradoux. “The Pitfalls of Hashing for Privacy”. In: *IEEE Communications Surveys & Tutorials* 20.1 (2018), pp. 551–565. DOI: 10.1109/COMST.2017.2747598 (cit. on p. 73).
- [FS18] Pietro Ferrara and Fausto Spoto. “Static Analysis for GDPR Compliance”. In: Jan. 2018 (cit. on p. 114).
- [Fra+24a] Lucas Franke, Huayu Liang, Sahar Farzanehpour, et al. *An Exploratory Mixed-Methods Study on General Data Protection Regulation (GDPR) Compliance in Open-Source Software*. 2024. arXiv: 2406.14724 [cs.SE] (cit. on p. 113).
- [Fra+24b] Lucas Franke, Huayu Liang, Sahar Farzanehpour, et al. “An Exploratory Mixed-methods Study on General Data Protection Regulation (GDPR) Compliance in Open-Source Software”. In: *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement. ESEM '24*. Barcelona, Spain: Association for Computing Machinery, 2024, pp. 325–336. DOI: 10.1145/3674805.3686692 (cit. on p. 49).
- [Gar+22] Jack Gardner, Yuanyuan Feng, Kayla Reiman, et al. “Helping Mobile Application Developers Create Accurate Privacy Labels”. In: *2022 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. 2022, pp. 212–230. DOI: 10.1109/EuroSPW55150.2022.00028 (cit. on p. 16).
- [Gir+25] Aniketh Girish, Joel Reardon, Juan Tapiador, Srdjan Matic, and Narseo Vallina-Rodriguez. *Your Signal, Their Data: An Empirical Privacy Analysis of Wireless-scanning SDKs in Android*. 2025. arXiv: 2503.15238 [cs.CR] (cit. on pp. 65, 66).

- [Gor+19] Sigmund Albert Gorski, Benjamin Andow, Adwait Nadkarni, et al. “ACMiner: Extraction and Analysis of Authorization Checks in Android’s Middleware”. In: *Proceedings of the Ninth ACM Conference on Data and Application Security and Privacy*. CODASPY ’19. Richardson, Texas, USA: Association for Computing Machinery, 2019, pp. 25–36. DOI: 10.1145/3292006.3300023 (cit. on p. 2).
- [Gri+13] Rebecca A. Grier, Aaron Bangor, Philip Kortum, and S. Camille Peres. “The System Usability Scale: Beyond Standard Usability Testing”. In: *Proceedings of the Human Factors and Ergonomics Society Annual Meeting* 57.1 (2013), pp. 187–191. DOI: 10.1177/1541931213571042. eprint: <https://doi.org/10.1177/1541931213571042> (cit. on pp. 88, 95, 106, 112, 121, 123).
- [GBJ06] Greg Guest, Arwen Bunce, and Laura Johnson. “How Many Interviews Are Enough?: An Experiment with Data Saturation and Variability”. In: *Field Methods* 18.1 (2006), pp. 59–82. DOI: 10.1177/1525822X05279903. eprint: <https://doi.org/10.1177/1525822X05279903> (cit. on p. 112).
- [HLT18] Hongmu Han, Ruixuan Li, and Junwei Tang. “Identify and Inspect Libraries in Android Applications”. In: *Wirel. Pers. Commun.* 103.1 (Nov. 2018), pp. 491–503 (cit. on p. 29).
- [Höl+24] Sandra Höltervennhoff, Noah Wöhler, Arne Möhle, et al. “A Mixed-Methods Study on User Experiences and Challenges of Recovery Codes for an End-to-End Encrypted Service”. In: *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 7267–7284 (cit. on pp. 51, 64).
- [Hor+24] Stefan Albert Horstmann, Samuel Domiks, Marco Gutfleisch, et al. ““Those things are written by lawyers, and programmers are reading that.” Mapping the Communication Gap Between Software Developers and Privacy Experts”. In: *Proc. Priv. Enhancing Technol.* 2024.1 (2024), pp. 151–170. DOI: 10.56553/POPEETS-2024-0010 (cit. on p. 1).
- [HRB88] S. Horwitz, T. Reps, and D. Binkley. “Interprocedural Slicing Using Dependence Graphs”. In: *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation*. PLDI ’88. Atlanta, Georgia, USA: Association for Computing Machinery, 1988, pp. 35–46. DOI: 10.1145/53990.53994 (cit. on pp. 11, 78).
- [Hua+15] Jianjun Huang, Zhichun Li, Xusheng Xiao, et al. “SUPOR: Precise and Scalable Sensitive User Input Detection for Android Apps”. In: *24th USENIX Security Symposium (USENIX Security 15)*. Washington, D.C.: USENIX Association, Aug. 2015, pp. 977–992 (cit. on pp. 27, 39).
- [Hua+14] Jianjun Huang, Xiangyu Zhang, Lin Tan, Peng Wang, and Bin Liang. “AsDroid: Detecting Stealthy Behaviors in Android Applications by User Interface and Program Behavior Contradiction”. In: *Proceedings of the 36th International Conference on Software Engineering*. ICSE 2014. Hyderabad, India: Association for Computing Machinery, 2014, pp. 1036–1046. DOI: 10.1145/2568225.2568301 (cit. on p. 39).

- [IKS24] Hiroki Inayoshi, Shohei Kakei, and Shoichi Saito. “Detection of Inconsistencies between Guidance Pages and Actual Data Collection of Third-party SDKs in Android Apps”. In: *Proceedings of the IEEE/ACM 11th International Conference on Mobile Software Engineering and Systems*. MOBILESoft ’24. Lisbon, Portugal: Association for Computing Machinery, 2024, pp. 43–53. DOI: 10.1145/3647632.3647991 (cit. on p. 63).
- [Iqb+21] Tahira Iqbal, Moniba Khan, Kuldar Taveter, and Norbert Seyff. “Mining Reddit as a New Source for Software Requirements”. In: *2021 IEEE 29th International Requirements Engineering Conference (RE)*. 2021, pp. 128–138. DOI: 10.1109/RE51729.2021.00019 (cit. on pp. 51, 65).
- [KB21] Kadiray Karakaya and Eric Bodden. “SootFX: A Static Code Feature Extraction Tool for Java and Android”. In: *2021 IEEE 21st International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE. 2021, pp. 181–186 (cit. on pp. 24, 27, 28).
- [Kel+09] Patrick Gage Kelley, Joanna Bresee, Lorrie Faith Cranor, and Robert W. Reeder. “A “nutrition label” for privacy”. In: *Proceedings of the 5th Symposium on Usable Privacy and Security*. SOUPS ’09. Mountain View, California, USA: Association for Computing Machinery, 2009. DOI: 10.1145/1572532.1572538 (cit. on p. 16).
- [KCS13] Patrick Gage Kelley, Lorrie Faith Cranor, and Norman Sadeh. “Privacy as part of the app decision-making process”. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. CHI ’13. Paris, France: Association for Computing Machinery, 2013, pp. 3393–3402. DOI: 10.1145/2470654.2466466 (cit. on p. 16).
- [Kha+23] Rishabh Khandelwal, Asmit Nayak, Paul Chung, and Kassem Fawaz. *Unpacking Privacy Labels: A Measurement and Developer Perspective on Google’s Data Safety Section*. 2023. arXiv: 2306.08111 [cs.CY] (cit. on pp. 17, 42, 43, 46, 47, 56, 65, 66, 87, 130).
- [Khe23] **Mugdha Khedkar**. “Static Analysis for Android GDPR Compliance Assurance”. In: *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 2023, pp. 197–199. DOI: 10.1109/ICSE-Companion58688.2023.00054 (cit. on p. xiii).
- [KB24] **Mugdha Khedkar** and Eric Bodden. “Toward an Android Static Analysis Approach for Data Protection”. In: *Proceedings of the IEEE/ACM 11th International Conference on Mobile Software Engineering and Systems*. MOBILESoft ’24. Lisbon, Portugal: Association for Computing Machinery, 2024, pp. 65–68. DOI: 10.1145/3647632.3651389 (cit. on pp. xiii, 114).
- [KKB26] **Mugdha Khedkar**, Ambuj Kumar Mondal, and Eric Bodden. “A Study of Privacy-Related Data Collected by Android Apps”. In: *Automated Software Engineering* 33.2 (2026), p. 45. DOI: 10.1007/s10515-025-00589-3 (cit. on p. xiii).

- [KMB24] **Mugdha Khedkar**, Ambuj Kumar Mondal, and Eric Bodden. “Do Android App Developers Accurately Report Collection of Privacy-Related Data?” In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops*. ASEW ’24. Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 176–186. DOI: 10.1145/3691621.3694949 (cit. on pp. xiii, 56, 66, 87).
- [Khe+26a] **Mugdha Khedkar**, Michael Schlichtig, Nihad Atakishiyev, and Eric Bodden. “Between Law and Code: Challenges and Opportunities for Automating Privacy Assessments”. In: *Automated Software Engineering* 33.56 (2026), p. 45. DOI: 10.1007/s10515-026-00601-4 (cit. on pp. xiv, 146).
- [KSB24] **Mugdha Khedkar**, Michael Schlichtig, and Eric Bodden. “Advancing Android Privacy Assessments with Automation”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops*. ASEW ’24. Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 218–222. DOI: 10.1145/3691621.3694953 (cit. on p. xiv).
- [KSB26] **Mugdha Khedkar**, Michael Schlichtig, and Eric Bodden. “Source Code-Driven GDPR Documentation: Supporting RoPA with Assessor View”. In: *2026 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 2026, pp. 935–939. DOI: 10.1109/SANER67736.2026.00109 (cit. on p. xiv).
- [Khe+25] **Mugdha Khedkar**, Michael Schlichtig, Santhosh Mohan, and Eric Bodden. *Visualizing Privacy-Relevant Data Flows in Android Applications*. 2025. arXiv: 2503.16640 [cs.CR] (cit. on p. 146).
- [Khe+26b] **Mugdha Khedkar**, Michael Schlichtig, Mohamed Soliman, and Eric Bodden. “Challenges in Android Data Disclosure: An Empirical Study.” In: *2026 IEEE/ACM 13th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. 2026. DOI: 10.1145/3795077.3795118 (cit. on p. xiii).
- [Kob+23] Maria Kober, Jordan Samhi, Steven Arzt, Tegawendé F Bissyandé, and Jacques Klein. “Sensitive and Personal Data: What Exactly Are You Talking About?” In: *2023 10th International Conference on Mobile Software Engineering and Systems 2023 (MobileSoft)*. May 2023 (cit. on pp. 21, 40).
- [Kon19] Konstantinos Limniotis, Marit Hansen. *Recommendations on shaping technology according to GDPR provisions - An overview on data pseudonymisation*. Tech. rep. European Union Agency for Cybersecurity (ENISA), 2019 (cit. on pp. 8, 21, 73).
- [Kri04] J. Krinke. “Visualization of program dependence and slices”. In: *20th IEEE International Conference on Software Maintenance, 2004. Proceedings*. 2004, pp. 168–177. DOI: 10.1109/ICSM.2004.1357801 (cit. on p. 113).
- [Kur+16] Andreas Kurtz, Hugo Gascon, Tobias Becker, Konrad Rieck, and Felix Freiling. “Fingerprinting Mobile Devices Using Personalized Configurations”. In: *Proceedings on Privacy Enhancing Technologies* (2016). DOI: 10.1515/popets-2015-0027 (cit. on p. 22).

- [LH03] Ondřej Lhoták and Laurie Hendren. “Scaling Java points-to analysis using SPARK”. In: *Proceedings of the 12th International Conference on Compiler Construction*. CC’03. Warsaw, Poland: Springer-Verlag, 2003, pp. 153–169 (cit. on p. 28).
- [Li+15] Li Li, Alexandre Bartel, Tegawendé F. Bissyandé, et al. “IccTA: Detecting Inter-Component Privacy Leaks in Android Apps”. In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 1. 2015, pp. 280–291. DOI: 10.1109/ICSE.2015.48 (cit. on pp. 2, 39).
- [Li+21a] Ruiyin Li, Peng Liang, Mohamed Soliman, and Paris Avgeriou. “Understanding Architecture Erosion: The Practitioners’ Perceptive”. In: *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*. 2021, pp. 311–322. DOI: 10.1109/ICPC52881.2021.00037 (cit. on pp. 51, 66).
- [Li+24] Tianshi Li, Lorrie Faith Cranor, Yuvraj Agarwal, and Jason I. Hong. “Matcha: An IDE Plugin for Creating Accurate Privacy Nutrition Labels”. In: *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8.1 (Mar. 2024), pp. 1–38. DOI: 10.1145/3643544 (cit. on pp. 18, 45, 63, 66, 131).
- [Li+21b] Tianshi Li, Elizabeth Louie, Laura Dabbish, and Jason I. Hong. “How Developers Talk About Personal Data and What It Means for User Privacy: A Case Study of a Developer Forum on Reddit”. In: *Proc. ACM Hum.-Comput. Interact.* 4.CSCW3 (Jan. 2021). DOI: 10.1145/3432919 (cit. on p. 51).
- [Li+22] Tianshi Li, Kayla Reiman, Yuvraj Agarwal, Lorrie Faith Cranor, and Jason I. Hong. “Understanding Challenges for Developers to Create Accurate Privacy Nutrition Labels”. In: *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. CHI ’22. New Orleans, LA, USA: Association for Computing Machinery, 2022. DOI: 10.1145/3491102.3502012 (cit. on pp. 49, 64).
- [LWE19] Ze Shi Li, Colin Werner, and Neil Ernst. “Continuous Requirements: An Example Using GDPR”. In: *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)*. 2019, pp. 144–149. DOI: 10.1109/REW.2019.00031 (cit. on p. 114).
- [Luo+21] Linghui Luo, Felix Pauck, Goran Piskachev, et al. “TaintBench: Automatic real-world malware benchmarking of Android taint analyses”. In: *Empirical Software Engineering* (2021). DOI: 10.1007/s10664-021-10013-5 (cit. on pp. 15, 72, 81, 83, 84, 87, 95).
- [M J19] M. Jensen, C. Lauradoux, and K. Limnietis. *Pseudonymisation techniques and best practices*. Tech. rep. European Union Agency for Cybersecurity (ENISA), 2019 (cit. on p. 8).
- [Mao+24] Ke Mao, Cons Åhs, Sopot Cela, et al. “PrivacyCAT: Privacy-Aware Code Analysis at Scale”. In: *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*. ICSE-SEIP ’24. Lisbon, Portugal: Association for Computing Machinery, 2024, pp. 106–117. DOI: 10.1145/3639477.3639742 (cit. on p. 114).

- [MK18] Yod-Samuel Martin and Antonio Kung. “Methods and Tools for GDPR Compliance Through Privacy and Data Protection Engineering”. In: *2018 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. 2018, pp. 108–111. DOI: 10.1109/EuroSPW.2018.00021 (cit. on p. 114).
- [NSB22] Marcus Nachtigall, Michael Schlichtig, and Eric Bodden. “A large-scale study of usability criteria addressed by static analysis tools”. In: *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA 2022*. Virtual, South Korea: Association for Computing Machinery, 2022, pp. 532–543. DOI: 10.1145/3533767.3534374 (cit. on p. 2).
- [Nan+15] Yuhong Nan, Min Yang, Zhemin Yang, et al. “UIPicker: User-Input Privacy Identification in Mobile Applications”. In: *24th USENIX Security Symposium (USENIX Security 15)*. Washington, D.C.: USENIX Association, Aug. 2015, pp. 993–1008 (cit. on pp. 27, 39).
- [NNH99] Flemming Nielson, Hanne R. Nielson, and Chris Hankin. *Principles of Program Analysis*. Berlin, Heidelberg: Springer-Verlag, 1999 (cit. on p. 10).
- [NSS25] Akila Niroshan, Suranga Seneviratne, and Aruna Seneviratne. *An Empirical Study of Code Obfuscation Practices in the Google Play Store*. 2025. arXiv: 2502.04636 [cs.CR] (cit. on p. 73).
- [Pan+24] Harshvardhan J. Pandit, Beatriz Esteves, Georg P. Krog, et al. *Data Privacy Vocabulary – Version 2*. 2024. arXiv: 2404.13426 [cs.CY] (cit. on pp. 4, 13, 76, 113, 114, 133).
- [Pan+19] Harshvardhan J. Pandit, Axel Polleres, Bert Bos, et al. “Creating a Vocabulary for Data Privacy”. In: *On the Move to Meaningful Internet Systems: OTM 2019 Conferences: Confederated International Conferences: CoopIS, ODBASE, C&TC 2019, Rhodes, Greece, October 21–25, 2019, Proceedings*. Rhodes, Greece: Springer-Verlag, 2019, pp. 714–730. DOI: 10.1007/978-3-030-33246-4_44 (cit. on p. 113).
- [Pan+13] Rahul Pandita, Xusheng Xiao, Wei Yang, William Enck, and Tao Xie. “WHY-PER: Towards Automating Risk Assessment of Mobile Applications”. In: *Proceedings of the 22nd USENIX Conference on Security. SEC’13*. Washington, D.C.: USENIX Association, 2013, pp. 527–542 (cit. on p. 39).
- [PW21] Felix Pauck and Heike Wehrheim. “Jicer: Simplifying Cooperative Android App Analysis Tasks”. In: *2021 IEEE 21st International Working Conference on Source Code Analysis and Manipulation (SCAM)*. 2021, pp. 187–197. DOI: 10.1109/SCAM52516.2021.00031 (cit. on pp. 11, 70, 71, 78, 113).
- [Per+24] Anthony Peruma, Timothy Huo, Ana Catarina Araújo, Jake Imanaka, and Rick Kazman. *A Developer-Centric Study Exploring Mobile Application Security Practices and Challenges*. 2024. arXiv: 2408.09032 [cs.CR] (cit. on p. 49).

- [Pis+20] Goran Piskachev, Lisa Nguyen Quang Do, Oshando Johnson, and Eric Bodden. “SWANAssist: Semi-Automated Detection of Code-Specific, Security-Relevant Methods”. In: *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering*. ASE ’19. San Diego, California: IEEE Press, 2020, pp. 1094–1097. DOI: 10.1109/ASE.2019.00110 (cit. on p. 39).
- [PNB19] Goran Piskachev, Lisa Nguyen Quang Do, and Eric Bodden. “Codebase-Adaptive Detection of Security-Relevant Methods”. In: *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2019 (cit. on p. 39).
- [RAB14] Siegfried Rasthofer, Steven Arzt, and Eric Bodden. “A Machine-learning Approach for Classifying and Categorizing Android Sources and Sinks”. In: *2014 Network and Distributed System Security Symposium (NDSS)*. Feb. 2014 (cit. on pp. 9, 22, 24, 25, 39).
- [Rei04] Frederick Reichheld. “The One Number you Need to Grow”. In: *Harvard business review* 81 (June 2004), pp. 46–54, 124 (cit. on pp. 88, 95, 112, 121, 124).
- [Rod+23] David Rodriguez, Akshath Jain, Jose M. Del Alamo, and Norman Sadeh. “Comparing Privacy Label Disclosures of Apps Published in both the App Store and Google Play Stores”. In: *2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. 2023, pp. 150–157. DOI: 10.1109/EuroSPW59978.2023.00022 (cit. on p. 65).
- [RH09] Per Runeson and Martin Höst. “Guidelines for conducting and reporting case study research in software engineering”. In: *Empirical Software Engineering* 14 (2009), pp. 131–164 (cit. on pp. 51, 65).
- [SMJ15] Ilaah Salman, Ayse Tosun Misirli, and Natalia Juristo. “Are Students Representatives of Professionals in Software Engineering Experiments?” In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 1. 2015, pp. 666–676. DOI: 10.1109/ICSE.2015.82 (cit. on pp. 87, 92, 120, 130).
- [Sam+23] Jordan Samhi, Maria Kober, Abdoul Kader Kabore, et al. “Negative Results of Fusing Code and Documentation for Learning to Accurately Identify Sensitive Source and Sink Methods”. In: *2023 30th edition of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 2023 (cit. on p. 39).
- [SN23] Sk Golam Saroar and Maleknaz Nayebi. *Developers’ Perception of GitHub Actions: A Survey Analysis*. 2023. arXiv: 2303.04084 [cs.SE] (cit. on p. 49).
- [Sch+07] Stuart E. Schechter, Rachna Dhamija, Andy Ozment, and Ian Fischer. “The Emperor’s New Security Indicators”. In: *2007 IEEE Symposium on Security and Privacy (SP ’07)*. 2007, pp. 51–65. DOI: 10.1109/SP.2007.35 (cit. on p. 112).

- [SSM25] Grishma Shrestha, Shristi Shrestha, and Anas Mahmoud. “No Country for Indie Developers: A Study of Google Play’s Closed Testing Requirements for New Personal Developer Accounts”. In: *ACM Trans. Softw. Eng. Methodol.* (May 2025). Just Accepted. DOI: 10.1145/3736578 (cit. on pp. 49, 51, 66).
- [Sla+16] Rocky Slavin, Xiaoyin Wang, Mitra Bokaei Hosseini, et al. “Toward a framework for detecting privacy policy violations in android application code”. In: *Proceedings of the 38th International Conference on Software Engineering*. ICSE ’16. Austin, Texas: Association for Computing Machinery, 2016, pp. 25–36. DOI: 10.1145/2884781.2884855 (cit. on p. 16).
- [SFB07] Manu Sridharan, Stephen J. Fink, and Rastislav Bodik. “Thin slicing”. In: *SIGPLAN Not.* 42.6 (June 2007), pp. 112–122. DOI: 10.1145/1273442.1250748 (cit. on p. 113).
- [Su+25] Chia-Yi Su, Aakash Bansal, Vijayanta Jain, et al. *Which Code Statements Implement Privacy Behaviors in Android Applications?* 2025. arXiv: 2503.02091 [cs.SE] (cit. on p. 79).
- [Su+17] Jessica Su, Ansh Shukla, Sharad Goel, and Arvind Narayanan. “De-Anonymizing Web Browsing Data with Social Networks”. In: *Proceedings of the 26th International Conference on World Wide Web*. WWW ’17. Perth, Australia: International World Wide Web Conferences Steering Committee, 2017, pp. 1261–1269. DOI: 10.1145/3038912.3052714 (cit. on p. 22).
- [SAW08] Mikael Svahnberg, Aybüke Aurum, and Claes Wohlin. “Using students as subjects - an empirical evaluation”. In: *Proceedings of the Second ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*. ESEM ’08. Kaiserslautern, Germany: Association for Computing Machinery, 2008, pp. 288–290. DOI: 10.1145/1414004.1414055 (cit. on pp. 87, 92, 120, 130).
- [TVS20] Mohammad Tahaei, Kami Vaniea, and Naomi Saphra. “Understanding Privacy-Related Questions on Stack Overflow”. In: *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. CHI ’20. Honolulu, HI, USA: Association for Computing Machinery, 2020, pp. 1–14. DOI: 10.1145/3313831.3376768 (cit. on pp. 51, 65, 66).
- [TS23] Zeya Tan and Wei Song. “PTPDroid: Detecting Violated User Privacy Disclosures to Third-Parties of Android Apps”. In: *2023 IEEE/ACM 45th IEEE International Conference on Software Engineering*. Melbourne, Australia, 2023 (cit. on pp. 16, 114, 139).
- [TØ22] Feiyang Tang and Bjarte M. Østvold. “Assessing Software Privacy Using the Privacy Flow-Graph”. In: *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*. MSR4P&S 2022. Singapore, Singapore: Association for Computing Machinery, 2022, pp. 7–15. DOI: 10.1145/3549035.3561185 (cit. on pp. 39, 114).

- [Val+10] Raja Vallée-Rai, Phong Co, Etienne Gagnon, et al. “Soot: a Java bytecode optimization framework”. In: *CASCON First Decade High Impact Papers*. CASCON '10. Toronto, Ontario, Canada: IBM Corp., 2010, pp. 214–224. DOI: 10.1145/1925805.1925818 (cit. on pp. 12, 27, 71, 113).
- [Wan+18] Xiaoyin Wang, Xue Qin, Mitra Bokaei Hosseini, et al. “GUILeak: Tracing Privacy Policy Claims on User Input Data for Android Applications”. In: *Proceedings of the 40th International Conference on Software Engineering*. ICSE '18. Gothenburg, Sweden: Association for Computing Machinery, 2018, pp. 37–47. DOI: 10.1145/3180155.3180196 (cit. on pp. 16, 39).
- [Wei82] Mark Weiser. “Programmers use slices when debugging”. In: *Commun. ACM* 25.7 (July 1982), pp. 446–452. DOI: 10.1145/358557.358577 (cit. on p. 10).
- [Wei84] Mark Weiser. “Program Slicing”. In: *IEEE Transactions on Software Engineering* SE-10.4 (1984), pp. 352–357. DOI: 10.1109/TSE.1984.5010248 (cit. on pp. 10, 12).
- [Wer+18] Dominik Wermke, Nicolas Huaman, Yasemin Acar, et al. “A Large Scale Investigation of Obfuscation Use in Google Play”. In: *Proceedings of the 34th Annual Computer Security Applications Conference*. ACSAC '18. San Juan, PR, USA: Association for Computing Machinery, 2018, pp. 222–235. DOI: 10.1145/3274694.3274726 (cit. on pp. 30, 73).
- [Xu+05] Baowen Xu, Ju Qian, Xiaofang Zhang, Zhongqiang Wu, and Lin Chen. “A brief survey of program slicing”. In: *SIGSOFT Softw. Eng. Notes* 30.2 (Mar. 2005), pp. 1–36. DOI: 10.1145/1050849.1050865 (cit. on pp. 10, 71, 113).
- [Yan+25] Jingwen Yan, Mohammed Aldeen, Jalil Harris, et al. “Understanding User Acceptance of Privacy Labels: Barriers and Enhancements”. In: *Proceedings of the 24th Workshop on Privacy in the Electronic Society*. WPES '25. Association for Computing Machinery, 2025, pp. 223–230. DOI: 10.1145/3733802.3764055 (cit. on p. 16).
- [Yog+21] Vahan Yoghoudjian, Yalong Yang, Tim Dwyer, et al. “Scalability of Network Visualisation from a Cognitive Load Perspective”. In: *IEEE Transactions on Visualization and Computer Graphics* 27.2 (2021), pp. 1677–1687. DOI: 10.1109/TVCG.2020.3030459 (cit. on p. 82).
- [Yu+16] Le Yu, Xiapu Luo, Xule Liu, and Tao Zhang. “Can We Trust the Privacy Policies of Android Apps?” In: *2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2016, pp. 538–549. DOI: 10.1109/DSN.2016.55 (cit. on p. 16).
- [Zha+23] Kaifa Zhao, Xian Zhan, Le Yu, et al. *Demystifying Privacy Policy of Third-Party Libraries in Mobile Apps*. 2023. DOI: 10.48550/ARXIV.2301.12348 (cit. on pp. 114, 139).

- [Zim+17] Sebastian Zimmeck, Ziqi Wang, Lieyong Zou, et al. “Automated Analysis of Privacy Requirements for Mobile Apps”. English (US). In: *Proceedings 2017 Network and Distributed System Security Symposium*. Proceedings 2017 Network and Distributed System Security Symposium. Korea, Republic of Korea: Korea Society of Internet Information, 2017. DOI: 10.14722/ndss.2017.23034 (cit. on p. 16).

Webpages

- [22a] *AdMob*. 2022. URL: <https://admob.google.com/home/> (cit. on p. 58).
- [26a] *Amazon Shopping App*. 2026. URL: <https://play.google.com/store/apps/details?id=com.amazon.mShop.android.shopping&hl=de> (cit. on pp. 8, 9).
- [14a] *Android-apkmirror*. 2014. URL: <https://www.apkmirror.com/> (cit. on p. 24).
- [10] *Android-apktool*. 2010. URL: <https://ibotpeaches.github.io/Apktool/> (cit. on p. 27).
- [11] *AndroidRank*. 2011. URL: <https://androidrank.org/android-most-popular-google-play-apps?price=free> (cit. on p. 34).
- [19b] *Anonymization in Data Privacy*. 2019. URL: <https://www.theguardian.com/technology/2019/jul/23/anonymised-data-never-be-anonymous-enough-study-finds> (cit. on p. 45).
- [14b] *AppBrain*. 2014. URL: <https://www.appbrain.com> (cit. on pp. 24, 34, 42, 73).
- [20] *Apple: Transparency is the best policy*. 2020. URL: <https://www.apple.com/privacy/labels/> (cit. on p. 16).
- [22b] *AppLovin*. 2022. URL: <https://www.applovin.com/> (cit. on pp. 58, 119).
- [26b] *AppsFlyer*. 2026. URL: <https://www.appsflyer.com/> (cit. on p. 119).
- [25a] *Caralegal*. 2025. URL: <https://caralegal.eu/en/> (cit. on p. 100).
- [CH22] Nia Castelly and Fergus Hurley. *Introducing Checks: simplifying privacy for app developers*. 2022. URL: <https://blog.google/technology/area-120/checks/> (cit. on pp. 18, 45, 63, 66, 130).
- [Cav09] Ann Cavoukian. *Privacy by design: The 7 foundational principles*. 2009. URL: https://student.cs.uwaterloo.ca/~cs492/papers/7foundationalprinciples_longer.pdf (cit. on pp. 1, 7, 73, 114).
- [18a] *CNIL*. 2018. URL: <https://www.cnil.fr/en> (cit. on pp. 20, 100).
- [18b] *CNIL PIA Tool*. 2018. URL: <https://www.cnil.fr/en/PIA-privacy-impact-assessment-en> (cit. on p. 100).

- [19c] *CNIL RoPA*. 2019. URL: <https://www.cnil.fr/en/gdpr-toolkit/record-processing-activities> (cit. on p. 19).
- [25b] *Code Project*. 2025. URL: <https://www.codeproject.com/> (cit. on p. 65).
- [25c] *Collibra*. 2025. URL: <https://www.collibra.com/> (cit. on p. 100).
- [24a] *D3.js Library*. 2024. URL: <https://d3js.org/> (cit. on p. 78).
- [18c] *Data Privacy Vocabularies and Controls Community Group*. 2018. URL: <https://www.w3.org/community/dpvcg/> (cit. on pp. 76, 94, 96, 97).
- [25d] *Data Privacy Vocabulary Primer*. 2025. URL: <https://w3c.github.io/dpv/2.2/dpv/> (cit. on pp. 13, 14).
- [18d] *Data Protection Impact Assessments*. 2018. URL: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/accountability-and-governance/data-protection-impact-assessments-dpias/> (cit. on pp. 1, 4, 131).
- [22c] *Data Safety Section*. 2022. URL: <https://blog.google/products/google-play/data-safety/> (cit. on pp. 1, 4, 16, 22).
- [26c] *DataGuard RoPA blog*. 2026. URL: <https://www.dataguard.com/blog/ropa/> (cit. on p. 19).
- [22d] *Declare your app's data use*. 2022. URL: <https://developer.android.com/privacy-and-security/declare-data-use> (cit. on pp. 43, 47).
- [25e] *Demo application from TaintBench*. 2025. URL: https://github.com/TaintBench/godwon_samp (cit. on pp. 133, 134).
- [24b] *DotGraph*. 2024. URL: <https://www.sable.mcgill.ca/soot/doc/soot/util/dot/DotGraph.html> (cit. on p. 113).
- [18e] *DroidBench*. 2018. URL: <https://github.com/secure-software-engineering/DroidBench/tree/master> (cit. on pp. 81, 83, 84, 87).
- [25f] *DSS form example from Tuya Developer Platform*. 2025. URL: <https://developer.tuya.com/en/docs/iot/Google-release-datasafety?id=Kbmh43wsn5kyr> (cit. on p. 19).
- [25g] *DZone*. 2025. URL: <https://dzone.com/> (cit. on p. 65).
- [26d] *Evidence-Based GDPR Optimization: What Professionals Really Need*. 2026. URL: https://noyb.eu/sites/default/files/2026-03/noyb-Survey_Evidence%20Based%20GDPR-Optimisation.pdf (cit. on p. 131).
- [25h] *Firebase Analytics*. 2025. URL: <https://firebase.google.com/docs/analytics/get-started?platform=web> (cit. on pp. 58, 119).
- [18f] *GDPR*. 2018. URL: <https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32016R0679> (cit. on p. 1).
- [18g] *GDPR Article 13*. 2018. URL: <https://gdpr-info.eu/art-13-gdpr/> (cit. on pp. 1, 8, 16, 75).

- [18h] *GDPR Article 25*. 2018. URL: <https://gdpr-info.eu/art-25-gdpr/> (cit. on pp. 1, 7, 14, 75, 76).
- [18i] *GDPR Article 30*. 2018. URL: <https://gdpr-info.eu/art-30-gdpr/> (cit. on pp. 1, 18, 131).
- [18j] *GDPR Article 32*. 2018. URL: <https://gdpr-info.eu/art-32-gdpr/> (cit. on p. 102).
- [18k] *GDPR Article 35*. 2018. URL: <https://gdpr-info.eu/art-35-gdpr/> (cit. on p. 1).
- [18l] *GDPR Article 4*. 2018. URL: <https://gdpr-info.eu/art-4-gdpr/> (cit. on pp. 2, 7, 79, 120).
- [18m] *GDPR Article 5*. 2018. URL: <https://gdpr-info.eu/art-5-gdpr/> (cit. on pp. 7, 75, 102, 111).
- [18n] *GDPR Article 9*. 2018. URL: <https://gdpr-info.eu/art-9-gdpr/> (cit. on p. 19).
- [18o] *GDPR Chapter 5*. 2018. URL: <https://gdpr-info.eu/chapter-5/> (cit. on p. 15).
- [22e] *Get more information about your apps in Google Play*. 2022. URL: <https://blog.google/products/google-play/data-safety/> (cit. on p. 65).
- [25i] *GitHub API*. 2025. URL: <https://docs.github.com/en/rest?apiVersion=2022-11-28> (cit. on p. 52).
- [25j] *Google Play Console*. 2025. URL: <https://play.google.com/console/signup> (cit. on pp. 17, 73).
- [26e] *Google Play Statistics*. 2026. URL: <https://www.appbrain.com/stats/number-of-android-apps> (cit. on p. 1).
- [22f] *Google's data types for DSS*. 2022. URL: <https://support.google.com/googleplay/android-developer/answer/10787469#zippy=%2Cdata-types> (cit. on pp. 18, 56).
- [22g] *Google's definition of data collection*. 2022. URL: <https://support.google.com/googleplay/android-developer/answer/10787469?hl=en%5C#zippy=%2Cdata-collection> (cit. on pp. 45, 47).
- [22h] *Google's purposes for DSS*. 2022. URL: <https://support.google.com/googleplay/android-developer/answer/10787469?hl=en#zippy=%2Cpurposes> (cit. on p. 18).
- [25k] *Hacker news*. 2025. URL: <https://news.ycombinator.com/> (cit. on p. 52).
- [18p] *IAPP*. 2018. URL: <https://iapp.org/> (cit. on p. 96).
- [26f] *ICO RoPA*. 2026. URL: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/accountability-and-governance/documentation/how-do-we-document-our-processing-activities/> (cit. on p. 20).

- [26g] *Instagram*. 2026. URL: <https://play.google.com/store/apps/details?id=com.instagram.android&hl=en> (cit. on pp. 45, 121).
- [17] *LibD*. 2017. URL: <https://github.com/IIE-LibD/libd> (cit. on p. 29).
- [16] *LibRadar*. 2016. URL: <https://github.com/pkumza/LibRadar> (cit. on p. 29).
- [25l] *MoPub*. 2025. URL: <https://www.applovin.com/max/> (cit. on p. 119).
- [23a] *Nielsen Normal Group. Between-subjects vs. within-subjects study design*. 2023. URL: <https://www.nngroup.com/articles/between-within-subjects/> (cit. on pp. 87, 120).
- [25m] *NoScribe*. 2025. URL: <https://github.com/kaixxx/noScribe> (cit. on pp. 98, 136).
- [25n] *Notifee*. 2025. URL: <https://notifee.app/> (cit. on p. 59).
- [25o] *OneTrust*. 2025. URL: <https://www.onetrust.com/> (cit. on pp. 104, 111, 139).
- [07] *Opinion 04/2007 on the concept of personal data*. 2007. URL: <https://www.clinicalstudydatarequest.com/Documents/Privacy-European-guidance.pdf> (cit. on p. 21).
- [22i] *Privado Play Store Data Safety Report Builder*. 2022. URL: <https://www.privado.ai/data-safety-report> (cit. on pp. 66, 104, 131).
- [22j] *Privado.ai*. 2022. URL: <https://www.privado.ai/> (cit. on pp. 18, 45, 54, 61, 63, 64, 139).
- [26h] *ProGuard*. 2026. URL: <https://github.com/Guardsquare/proguard> (cit. on p. 30).
- [18q] *Records of processing activities*. 2018. URL: <https://gdpr-info.eu/issues/records-of-processing-activities/> (cit. on pp. 1, 5, 18, 99, 103, 131).
- [25p] *Reddit API*. 2025. URL: <https://www.reddit.com/dev/api/> (cit. on p. 52).
- [25q] *Salesforce Privacy Center*. 2025. URL: <https://www.salesforce.com/platform/privacy-center/> (cit. on p. 104).
- [23b] *See No Evil: Loopholes in Google's Data Safety Labels Keep Companies in the Clear and Consumers in the Dark*. 2023. URL: https://assets.mofoprod.net/network/documents/PNI_Google_Play_Store_Research_Report_2023.pdf (cit. on pp. 17, 45, 65, 66, 130).
- [22k] *Signal Private Messenger*. 2022. URL: <https://play.google.com/store/apps/datasafety?id=org.thoughtcrime.securesms&hl=en> (cit. on pp. 17, 42, 119–121, 127).
- [25r] *Site point*. 2025. URL: <https://www.sitepoint.com/> (cit. on p. 65).
- [25s] *SonarQube*. 2025. URL: <https://www.sonarsource.com/sem/products/sonarqube/> (cit. on pp. 104, 111, 139).

- [25t] *Stack Exchange API*. 2025. URL: <https://api.stackexchange.com/> (cit. on p. 52).
- [Swe00] Latanya Sweeney. *Simple Demographics Often Identify People Uniquely*. 2000. URL: <http://dataprivacylab.org/projects/identifiability> (cit. on p. 38).
- [26i] *Telegram*. 2026. URL: <https://play.google.com/store/apps/details?id=org.telegram.messenger&hl=en> (cit. on pp. 42, 121).
- [22l] *Unity*. 2022. URL: <https://unity.com/> (cit. on p. 58).
- [26j] *WALA*. 2026. URL: <https://github.com/wala/WALA> (cit. on p. 113).
- [18r] *What is GDPR?* 2018. URL: <https://gdpr.eu/what-is-gdpr/> (cit. on p. 7).
- [25u] *XDA forums*. 2025. URL: <https://xdaforums.com/> (cit. on p. 65).

List of Acronyms

ADG	Application Dependence Graph
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Application Package
CI/CD	Continuous Integration and Continuous Deployment
CVV	Card Verification Value
DPIA	Data Protection Impact Assessment
DPO	Data Protection Officer
DPV	Data Privacy Vocabulary
DSS	Data Safety Section
GDPR	General Data Protection Regulation
HTTP	Hypertext Transfer Protocol
IAPP	International Association for Privacy Professionals
IBAN	International Bank Account Number
IDE	Integrated Development Environment
IP	Internet Protocol
LLM	Large Language Model
MAC	Media Access Control
NLP	Natural Language Processing
NPS	Net Promoter Score
OS	Operating System
OTP	One-Time Password
PIN	Personal Identification Number
PRICE	Privacy-Relevant Input Classification Engine
PDG	Program Dependence Graph

RoPA Record of Processing Activities
SDG System Dependence Graph
SDK Software Development Kit
SIM Subscriber Identity Module
SPARK Soot Pointer Analysis Research Kit
SUS System Usability Scale
SVG Scalable Vector Graphics
TPL Third-Party Library
UI User Interface
URL Uniform Resource Locator
XML eXtensible Markup Language

List of Figures

1.1.	Overview of stakeholders and static analysis-based support for GDPR compliance in Android applications. Static analysis bridges technical artifacts and interdisciplinary stakeholders by supporting DSS reporting, DPIAs, and RoPA generation.	3
2.1.	Example: Input fields with labels and Edit Texts from Amazon shopping app [26a].	9
2.2.	Comparison of the original code and the forward slice with respect to variable <i>x</i>	13
2.3.	A small subset of DPV core concepts and the possible relationships between them. The complete documentation can be found in the DPV primer [25d].	14
2.4.	An example DPV model and DPV diagram.	14
2.5.	An example of the DSS of Signal [22k].	17
2.6.	An example of the DSS form for the data type <i>Email address</i> [25f], showing 1. data collection and/or sharing, 2. ephemeral processing, and 3. optional data collection.	19
3.1.	Workflow of Privacy-Relevant Input Classification Engine (PRICE). . . .	26
4.1.	Our empirical study design.	50
4.2.	Methods and resources participants use to categorize privacy-related data collected by their app into the DSS data categories (RQ1).	54
4.3.	Survey participants' responses to the questions about confidence levels for different processes (RQ2).	55
4.4.	Bar chart showing the distribution of challenges across survey participants (RQ3). Participants could choose multiple challenges through various questions.	57
5.1.	Workflow of Assessor View. Module B (shaded with gray) builds on Jicer [PW21], an existing slicer for Android apps.	70
5.2.	<i>Upload APK</i> page of Assessor View, showing ① input APK, and ② the option to analyze a sample APK provided by the back-end.	71

5.3.	Assessor View dashboard for Roidsec, a real-world app from the Taint-Bench suite [Luo+21].	72
5.4.	GDPR Warnings for Roidsec. These warnings correspond to rules 1 and 3 in Table 5.1.	75
5.5.	Different views for Roidsec.	77
5.6.	Developer View of a Roidsec slice. Nodes: source code statements, edges: control and data dependencies. Pop-ups reveal details for each node (pop-up for node 1 shown in the figure). Node 1 collects the network operator name (risk 2), which is copied (node 2), and appended to a string (node 3). Nodes 4–9 append other variables. The dashboard assigns a low-risk (B) warning since only string manipulations occur, suggesting potential for data minimization (GDPR Article §4 [18l]). . .	79
5.7.	An example mapping between Developer View and DPV View for the Roidsec example from Figure 5.5.	81
5.8.	Customization options for developers on the <i>Upload APK</i> page (cf. Figure 5.2), showing ① inclusion of control dependencies in the graph, ② maximum time limit for the analysis, and ③ maximum nodes allowed in a slice.	82
5.9.	Iterative user-centered design process of Assessor View. Developers evaluated the first implemented abstraction, the Developer View, which served as the basis for the later higher-level DPV View (cf. Section 5.2.1). After refinement of the system and interaction design, the extended prototype was evaluated with privacy professionals (cf. Section 5.2.2). The final refinement was evaluated again by DPOs (cf. Section 6.2.2). . . .	86
5.10.	Participants’ responses to the tool-based questions about dashboard and the views.	90
5.11.	Participants’ responses to the tool-based questions about DPV View and GDPR compliance warnings.	106
5.12.	Participants’ responses to the usability questions. For (+) statements, <i>Strongly Agree</i> represents a more favorable response. For (-) statements, <i>Strongly Disagree</i> represents a more favorable response.	107
5.13.	Assessor View’s rating for legal experts, privacy experts, and DPOs. (0: not recommended, 10: highly recommended)	108
6.1.	Our approach built on top of PRICE (Module B) (cf. Section 3.3). . . .	118

6.2.	AutoPRICE dashboard for Signal [22k], showing ① legends to identify privacy-related data and GDPR's personal data, ② data collected via system APIs, ③ data collected via permissions, ④ data collected via app UI, and ⑤ button to automatically populate the <i>Data collected</i> part of the DSS form.	119
6.3.	A part of the pre-filled DSS form for Signal [22k], showing ① data category, ② data type, and ③ explanation on how the data type has been collected.	120
6.4.	User study participants' responses to the usability questions. For (+) statements, <i>Strongly Agree</i> represents a more favorable response. For (-) statements, <i>Strongly Disagree</i> represents a more favorable response. . .	124
6.5.	AutoPRICE's rating for developers and users. (0: not recommended, 10: highly recommended)	124
6.6.	Different interfaces for the dashboard.	127
6.7.	Information about <i>where</i> data is collected from: third-party libraries or in-app methods.	128
6.8.	Enhanced DSS filler component, showing ① data category, ② data types, ③ explanation on how the data type has been collected, and ④ editable purposes of data collection (cf. Table 2.3).	129
6.9.	Comparison between the DSS generated by AutoPRICE and the corresponding Play Store DSS of Signal, showing ① manually entered URL of the Play Store DSS, ② DSS filled by AutoPRICE, ③ DSS extracted from the given link, and ④ a summary of under-reporting, over-reporting or other observations (cropped partially due to space constraints).	129
6.10.	Enhanced Assessor View architecture.	132
6.11.	Assessor View dashboard for the demo app [25e], showing ① data sources, ② warning scale for developers, ③ warning scale for privacy experts, ④ assessment flags for collaboration, ⑤ selection of sources to generate RoPA, and ⑥ generation of RoPA.	133
6.12.	RoPA formats for the demo app [25e], showing editable fields for ① the purpose of processing, ② special categories of data, and ③ DPO remarks (present in both formats but partially cropped in Figure 6.12b).	134

List of Tables

2.1.	Some examples of DPV from real-world apps.	15
2.2.	Google Play Store's DSS data categories and data types [22f].	18
2.3.	Google Play Store's predefined DSS purposes [22h].	18
3.1.	Classification of privacy-related data.	23
3.2.	Examples of privacy-related data.	23
3.3.	Third-Party Libraries in the Identifier API Dataset. Not an exhaustive list.	25
3.4.	Part of the <i>Identifier Keywords Dataset</i>	25
3.5.	Part of the <i>Identifier API Dataset</i>	26
3.6.	Statistics of 7,500 apps. All tables in this section present percentages between 50-100 in red, 20-50 in yellow, and below 20 in green.	32
3.7.	UI privacy-related data distribution (2182 apps).	32
3.8.	System API privacy-related data distribution (7150 apps).	32
3.9.	UI personal data distribution (2182 apps).	33
3.10.	System API personal data distribution (7150 apps).	33
3.11.	Statistics of 700 most downloaded apps (100 from each domain).	34
3.12.	UI privacy-related data distribution (552 most downloaded apps).	35
3.13.	Domain-wise UI privacy-related data distribution. Highlighted: highest percentage of apps collecting data.	35
3.14.	Domain-wise UI personal data distribution. Highlighted: highest per- centage of apps collecting data. Risk 1: directly identifiable, Risk 2: partially identifiable.	36
3.15.	Identifier combinations collected by most downloaded apps, Highlighted: highest percentage of apps collecting the identifier combination.	37
3.16.	UI privacy-related data distribution for random apps (RQ1) vs. most downloaded apps (RQ2).	38
4.1.	Comparison between our results and the Data Safety Sections of 20 popular apps. Highlighted apps are discussed in detail.	44
4.2.	Mapping between research questions and survey questions.	51
4.3.	Survey participants' demographics (based on optional responses).	51

4.4.	Summary of developer discussions collected across platforms. “Raw posts” refers to all extracted discussions (may include duplicates and irrelevant posts), and “Final posts” refers to those retained after filtering.	52
4.5.	Distribution of challenges across online developer discussions (RQ3). At the first level, discussions are grouped according to the high-level challenge they addressed. Each challenge is further subdivided into specific subtopics that captured recurring themes. Each online discussion is mapped to a single subtopic. Abbreviation: TPL = third-party library.	58
4.6.	Distribution of challenges across online developer discussions (continued).	58
5.1.	GDPR rules and warnings addressed by the DPV Transformer.	75
5.2.	A small subset of the mapping from source code statements in Developer View to DPV processing operations in DPV View	80
5.3.	Summary of participant experience. Filled circles indicate expertise from 1 (Beginner) to 5 (Expert).	88
5.4.	Summary of interview protocol. The first column is the topic and the number of questions for that topic.	96
5.5.	Summary of participant experience. * denotes members of the DPV maintenance group [18c]. Filled circles represent expertise levels on a Likert scale from 1 (Beginner) to 5 (Expert). P12 and P15 only participated in the non-tool based study (Sections 1 and 2 of the study). . . .	97
6.1.	Summary of participant experience. Filled circles indicate expertise from 1 (Beginner) to 5 (Expert).	122
6.2.	Summary of interview protocol. The first column is the topic and the number of questions for that topic.	136
6.3.	Summary of participant experience. Filled circles indicate expertise from 1 (Beginner) to 5 (Expert).	137

Appendix: Developer Survey Questionnaire

In this appendix, we detail the questions of the developer survey (cf. Section 4.2). The anonymized answers are available online at <https://doi.org/10.5281/zenodo.17416187>.

Section 1: Experience and Demographics

1.1. What is your email address?

- Free-text field.
- Optional.

1.2. What is your age?

- Multiple choice question.
- Optional.
- Choices:
 - * Less than 25
 - * 25-34
 - * 35-44
 - * 45-54
 - * 55-64
 - * Greater 64

1.3. What is your gender?

- Multiple choice question.
- Optional.
- Choices:
 - * Male
 - * Female

- * Non-binary person
- * Prefer not to answer

1.4. How many years of experience do you have in Android app development?

- Multiple choice question.
- Mandatory.
- Choices:
 - * No experience
 - * Less than 1 year
 - * 1-3 years
 - * 4-6 years
 - * 7-10 years
 - * More than 10 years

1.5. Is your app published in the European Union? (This applies regardless of your location.)

- Multiple choice question.
- Mandatory.
- Choices:
 - * Yes
 - * No

Section 2: DSS Form and GDPR

2.1. How challenging is it to fill out the Data Collection part of the Data Safety Section (DSS)? If challenging, specify the difficult parts in the comment box.

- Multiple choice question.
- Optional text-field for comments.
- Mandatory.
- Choices:
 - * Very easy
 - * Easy
 - * Neutral

- * Challenging
- * Very challenging
- * Never done that

2.2. How well do you understand Google's definitions and categorizations of data types within the Data Safety Section (DSS)? If you answered *Poorly* or *Very poorly*, specify which parts you did not understand.

- Multiple choice question.
- Optional text-field for comments.
- Mandatory.
- Choices:
 - * Very well
 - * Well
 - * Neutral
 - * Poorly
 - * Very poorly
 - * Never needed to understand

2.3. How familiar are you with GDPR (General Data Protection Regulation)? If you answered *Somewhat unfamiliar* or *Very unfamiliar*, specify which aspects are unclear.

- Multiple choice question.
- Optional text-field for comments.
- Mandatory.
- Choices:
 - * Very familiar
 - * Somewhat familiar
 - * Neutral
 - * Somewhat unfamiliar
 - * Very unfamiliar
 - * Never heard of GDPR

2.4. How confident are you that your app's data collection in the Data Safety Section (DSS) aligns with GDPR requirements? If you are not confident, specify the GDPR compliance challenges in the comment box.

- Multiple choice question.
- Optional text-field for comments.
- Mandatory.
- Choices:
 - * Very confident
 - * Confident
 - * Neutral
 - * Not confident
 - * Not confident at all

2.5. How well do you understand the privacy-related data your app collects (e.g., location, personal, financial) that must be disclosed in the Data Safety Section (DSS)? If you answered *Poorly* or *Very poorly*, specify which parts are unclear.

- Multiple choice question.
- Optional text-field for comments.
- Mandatory.
- Choices:
 - * Very well
 - * Well
 - * Neutral
 - * Poorly
 - * Very poorly

2.6. How do you categorize the privacy-related data collected by your app for the Data Safety Section (DSS) of the Google Play Store?

- Multiple choice question.
- Optional text-field for comments.
- Mandatory.
- Choices:

- * Manually
- * Using an in-house tool
- * Using third-party software or tools (if selected, please specify in the comment box)
- * Not categorized
- * Other

2.7. How confident are you in categorizing the privacy-related data your app collects (e.g., precise vs. approximate location) for the Data Safety Section (DSS) of Google Play? If not confident, please specify the challenges.

- Multiple choice question.
- Optional text-field for comments.
- Mandatory.
- Choices:
 - * Very confident
 - * Confident
 - * Neutral
 - * Not confident
 - * Not confident at all

2.8. Which aspects of the Data Safety Section (DSS) do you find challenging? Select **all** that apply.

- Multiple choice question.
- Optional text-field for comments.
- Mandatory.
- Choices:
 - * Understanding the DSS requirements
 - * Technical submission process
 - * Collecting the required privacy-related information for my app
 - * Ensuring compliance with data privacy laws
 - * Updating the DSS for every app version
 - * Others

2.9. What resources do you use to help you understand and complete the Data Safety section (DSS)? Select **all** that apply.

- Multiple choice question.
- Optional text-field for comments.
- Mandatory.
- Choices:
 - * Google’s official documentation
 - * Google’s Checks framework
 - * Online forums and communities
 - * Third-party tools and services
 - * Other

Section 3: General Feedback

3.1. What specific challenges have you faced while filling out the Data Safety Section (DSS) on Google Play Store?

- Free-text field.
- Mandatory.

3.2. Describe any technical or general issues you have encountered during the submission of the Data Safety Section (DSS) form on the Google Play Store.

- Free-text field.
- Optional.

3.3. Have you ever had an app rejected due to issues with the Data Safety Section (DSS)? If yes, please describe the situation.

- Free-text field.
- Optional.

Appendix: Study Materials for Assessor View Evaluation (Developers)

In this appendix, we detail study conducted in Section 5.2.1. The anonymized answers are available online at <https://zenodo.org/records/17395001>.

Section 1: Participant information

- 1.1. How much experience do you have with Java/Android development? (in years)
 - Free-text field.
 - Mandatory.
- 1.2. Rate your knowledge on the following on a scale of Beginner to Expert.
 - A. Android app development:
 - Likert scale from 1 (Beginner) to 5 (Expert).
 - Mandatory.
 - B. Java development:
 - Likert scale from 1 (Beginner) to 5 (Expert).
 - Mandatory.
 - C. Understanding of Jimple:
 - Likert scale from 1 (Beginner) to 5 (Expert).
 - Mandatory.
 - D. Using static analysis tools:
 - Likert scale from 1 (Beginner) to 5 (Expert).
 - Mandatory.
 - E. Graph visualization:
 - Likert scale from 1 (Beginner) to 5 (Expert).

- Mandatory.
- F. Data privacy:
 - Likert scale from 1 (Beginner) to 5 (Expert).
 - Mandatory.
- G. GDPR:
 - Likert scale from 1 (Beginner) to 5 (Expert).
 - Mandatory.

Section 2: Analyze data processing using Assessor View

- 2.1. Which types of privacy-related data does this app collect? (Select all that apply)
- Multiple choice question.
 - Mandatory.
 - Choices:
 - * Risk 1 directly identifiable data
 - * Risk 2 partially identifiable data
 - * Risk 3 access data
 - * Risk 4 context-dependent data
 - * Personal data
- 2.2. Which of the collected data types require protection according to GDPR guidelines?
- Free-text field.
 - Mandatory.
- 2.3. Which of the following privacy-related properties could you observe in the slices? (Select all that apply)
- Multiple choice question.
 - Mandatory.
 - Choices:
 - * Data minimization – app is collecting data which is not used. Data collection may not be necessary.
 - * Third-party API usage – app uses third-party API methods.

- * No data protection – no technical measures such as encryption are used by the app.
- 2.4. The dashboard contains enough information to identify privacy-related features.
- Multiple choice question.
 - Optional text-field for comments.
 - Mandatory.
 - Choices:
 - * Strongly disagree
 - * Disagree
 - * Neutral
 - * Agree
 - * Strongly agree

Section 3: Comparison between Static Analysis and Developer View

- 3.1. In general, program slice was easier to understand with:
- Multiple choice question.
 - Optional text-field for comments.
 - Mandatory.
 - Choices:
 - * Developer View
 - * Static Analysis View
- 3.2. In general, program slice was faster to understand with:
- Multiple choice question.
 - Optional text-field for comments.
 - Mandatory.
 - Choices:
 - * Developer View
 - * Static Analysis View
- 3.3. I am comfortable with Developer View.
- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).

- Optional text-field for comments.
 - Mandatory.
- 3.4. What are the positive points of Developer View?
- Free-text field.
 - Mandatory.
- 3.5. What are the negative points of Developer View?
- Free-text field.
 - Mandatory.
- 3.6. I am comfortable with Static Analysis View.
- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Optional text-field for comments.
 - Mandatory.
- 3.7. What are the positive points of Static Analysis View?
- Free-text field.
 - Mandatory.
- 3.8. What are the negative points of Static Analysis View?
- Free-text field.
 - Mandatory.
- 3.9. Developer View would be best suited for understanding the program.
- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Optional text-field for comments.
 - Mandatory.
- 3.10. Static Analysis View would be best suited for understanding the program.
- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Optional text-field for comments.
 - Mandatory.
- 3.11. How likely is it you would recommend Developer View over Static Analysis View to a friend?
- Likert scale from 0 (Not at all) to 10 (Go for it!).

- Optional text-field for comments.
 - Mandatory.
- 3.12. How likely is it you would recommend Static Analysis View over Developer View to a friend?
- Likert scale from 0 (Not at all) to 10 (Go for it!).
 - Optional text-field for comments.
 - Mandatory.

Section 4: Privacy-Related Properties of an App

- 4.1. Which types of privacy-related data does this app collect? (Select all that apply)
- Multiple choice question.
 - Mandatory.
 - Choices:
 - * Risk 1 directly identifiable data
 - * Risk 2 partially identifiable data
 - * Risk 3 access data
 - * Risk 4 context-dependent data
 - * Personal data
- 4.2. Which of the following privacy-related properties could you observe in the slices? (Select all that apply)
- Multiple choice question.
 - Mandatory.
 - Choices:
 - * Data minimization – app is collecting data which is not used. Data collection may not be necessary.
 - * Third-party API usage – app uses third-party API methods.
 - * No data protection – no technical measures such as encryption are used by the app.
- 4.3. I could understand the slice easily without access to the dashboard.
- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).

- Mandatory.
- 4.4. I would prefer to have an overview and the hints provided by the dashboard.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.

Section 5: Usability

- 5.1. I would like to use Assessor View for my privacy-aware development needs.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.2. I found Assessor View unnecessarily complex.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.3. I found Assessor View easy to use.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.4. I would need the support of a technical person to be able to use Assessor View.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.5. I found the various functions in Assessor View to be well integrated.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.6. I thought there was too much inconsistency in Assessor View.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.7. I would imagine that most people would learn to use Assessor View very quickly.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.8. I found Assessor View very cumbersome to use.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).

- Mandatory.
- 5.9. I felt confident using Assessor View.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.10. I needed to learn a lot of things before I could get going with Assessor View.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.11. How likely is it you would recommend Assessor View to an Android developer?
 - Likert scale from 0 (Not at all) to 10 (Go for it!).
 - Mandatory.

Section 6: General Feedback

- 6.1. What features would you like in Assessor View?
 - Free-text field.
 - Mandatory.
- 6.2. What are the positive points of Assessor View?
 - Free-text field.
 - Mandatory.
- 6.3. What are the negative points of Assessor View?
 - Free-text field.
 - Mandatory.
- 6.4. Was the given documentation useful?
 - Free-text field.
 - Mandatory.
- 5.5. Did Assessor View give you a better understanding of privacy-related data processing?
 - Free-text field.
 - Mandatory.
- 5.6. Other remarks.
 - Free-text field, optional.

Appendix: Study Materials for Assessor View Evaluation (Privacy Experts)

In this appendix, we detail the questions of the study conducted in Section 5.2.2. The anonymized answers are available online at <https://zenodo.org/records/17589238>.

C.1 Interview Briefing

Thank you for participating in our study. We are going to be asking you some questions today about privacy assessments and GDPR compliance. Thank you for signing the pre-study consent form. I just wanted to emphasize that what you say to us during the study will be kept confidential, and you can stop participating at any time. If you feel uncomfortable answering any question, we can skip them with no penalty. We will be recording the study. Before we start recording and begin the study, would you like to ask us any questions? We will be sharing an online study questionnaire with you, that you can open in a new browser window. Would you be comfortable sharing your browser window? That will help us guide you through the process. We will sign another consent form before starting the study. Consent form: <obtain consent from participant>

C.2 Interview Questionnaire

Section 1: Participant Information

- 1.1. What is your designation?
 - Free-text field.
 - Mandatory.
- 1.2. What is your primary role?

- Free-text field.
 - Mandatory.
- 1.3. How many years of experience do you have in privacy and/or law? (in years)
- Free-text field.
 - Mandatory.
- 1.4. What is the highest degree you have received? (If you're currently enrolled in school, please indicate the last degree you received)
- Multiple choice question with an "Other.." field.
 - Choices:
 - * Bachelor's degree (LL.B., BSc., BTech, etc)
 - * Master's degree (LL.M., MSc., MTech, etc)
 - * Doctoral degree (Ph.D., Ed.D., etc.)
 - * Other...
 - Mandatory.
- 1.5. Have you received any additional privacy-related certifications? If yes, which ones?
- Free-text field.
 - Optional.
- 1.6. Which country do you currently work in?
- Free-text field.
 - Optional.
- 1.7. Rate your knowledge on the following:
- A. Data visualization:
- Likert scale from 1 (Beginner) to 5 (Expert).
 - Mandatory.
- B. GDPR:
- Likert scale from 1 (Beginner) to 5 (Expert).
 - Mandatory.
- C. GDPR compliance:

- Likert scale from 1 (Beginner) to 5 (Expert).
- Mandatory.

D. Data Protection Impact Assessment (DPIA):

- Likert scale from 1 (Beginner) to 5 (Expert).
- Mandatory.

E. Data Privacy Vocabulary (DPV):

- Likert scale from 1 (Beginner) to 5 (Expert).
- Mandatory.

Section 2: Privacy Assessments and GDPR Compliance

- 2.1. Can you walk us through a privacy assessment or a DPIA process?
- 2.2. Who are the stakeholders in such a privacy assessment/ DPIA?
- 2.3. Which parts of the assessment are manual? Which parts aren't, and how are they assisted?
- 2.4. What is your role in the assessment process?
- 2.5. Do privacy assessors have to understand source code to successfully complete such an assessment/ to advice your clients? How do they navigate the process of understanding source code?
- 2.6. Is DPV or any such vocabulary actively used in the assessment process?
- 2.7. Any plans of long-term support version of DPV? (specifically for the DPV maintenance group)
- 2.8. Have you used any tools to verify whether a software complies with GDPR? Which tools?
- 2.9. Which documents do data controllers need to complete to adhere to GDPR?
- 2.10. Do your clients consult you to help complete these documents? Is this a manual process?
- 2.11. Are there any training programs specific to GDPR that are mandatory for the stakeholders involved in a privacy assessment process? Do they cover automation?

Section 3: Evaluating a Privacy-Relevant Program Slice

- 3.1. Assessor View's DPV View provides a clear overview of the privacy practices in the program slice.

- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Optional text-field for comments.
 - Mandatory.
- 3.2. The DPV View helps in identifying potential GDPR violations or areas requiring further investigation.
- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Optional text-field for comments.
 - Mandatory.
- 3.3. The DPV View provides detailed insights into the specific data processing activities and flows.
- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Optional text-field for comments.
 - Mandatory.

Section 4: Understanding GDPR Compliance Warnings

- 4.1. The warnings highlight potential GDPR compliance issues.
- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Optional text-field for comments.
 - Mandatory.
- 4.2. The warnings make it easier to assess GDPR principles such as data minimization, purpose limitation, or security measures.
- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Optional text-field for comments.
 - Mandatory.
- 4.3. The GDPR compliance warnings and suggestions are useful in guiding your analysis.
- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Optional text-field for comments.
 - Mandatory.
- 4.4. What additional compliance-related insights would you like to see?
- Free-text field.

- Optional.

Section 5: Usability and Interactivity

- 5.1. I would like to use Assessor View for my needs.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.2. I found Assessor View unnecessarily complex.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.3. I found Assessor View easy to use.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.4. I would need the support of a technical person to be able to use Assessor View.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.5. I found the various functions in Assessor View to be well integrated.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.6. I thought there was too much inconsistency in Assessor View.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.7. I would imagine that most people would learn to use Assessor View very quickly.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.8. I found Assessor View very cumbersome to use.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.9. I felt confident using Assessor View.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).

- Mandatory.
- 5.10. I needed to learn a lot of things before I could get going with Assessor View.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 5.11. How likely is it you would recommend Assessor View to a legal expert?
 - Likert scale from 0 (Not at all) to 10 (Go for it!).
 - Mandatory.
- 5.12. How likely is it you would recommend Assessor View to a privacy expert?
 - Likert scale from 0 (Not at all) to 10 (Go for it!).
 - Mandatory.
- 5.13. How likely is it you would recommend Assessor View to a DPO?
 - Likert scale from 0 (Not at all) to 10 (Go for it!).
 - Mandatory.

Section 6: Feedback and Recommendations

- 6.1. Do you think Assessor View would be the first step to reducing reliance on developers (for privacy experts)?
- 6.2. Do you think Assessor View can be used in privacy assessments or for conducting conversations about privacy assessments?

C.3 Codebook

In this section, we present our codebook.

- 1. GDPR Compliance
 - 1.1. Documents required
 - 1.1.1. RoPA
 - 1.1.2. Privacy notice
 - 1.1.3. Privacy policy
 - 1.1.4. Contracts
 - 1.1.4.1. Service provider contract
 - 1.1.4.2. Joint controller contract

- 1.1.4.3. Controller contract
 - 1.1.5. Agreements
 - 1.1.5.1. Service level agreement
 - 1.1.5.2. Non-disclosure agreement
 - 1.1.5.3. Data processing agreement
 - 1.1.5.4. Works council agreement (Germany)
 - 1.1.6. Assessment of partner activities
 - 1.1.7. Rule of conduct
 - 1.1.8. DPIA report
 - 1.1.9. Consent
 - 1.1.10. Report of data breach
 - 1.1.11. Data retention policy
 - 1.1.12. Data access request
 - 1.1.13. Data protection by design assessment
- 1.2. Documentation support
 - 1.2.1. Document generators
 - 1.2.2. Decision tree-based legal tech tools
 - 1.2.3. Standard templates
 - 1.2.4. None
 - 1.3. Compliance support
 - 1.3.1. Cookie compliance tools
 - 1.3.2. ChatGPT
 - 1.3.3. Data classification tools
 - 1.3.4. Source code analysis for security
 - 1.3.5. Data protection management system
 - 1.3.6. Privacy policy scanner
 - 1.3.7. None
 - 1.4. Training programs
 - 1.4.1. Organization-specific base level compliance

- 1.4.2. Training programs for specific roles
- 1.4.3. Formal privacy certifications
- 1.4.4. Country-specific programs
- 1.4.5. Tabletop training for clients
- 2. DPIA (Article 35 GDPR)
 - 2.1. Processes
 - 2.1.1. Pre-DPIA Screening
 - 2.1.2. Privacy by design
 - 2.1.3. Data mapping
 - 2.1.4. Information gathering
 - 2.1.5. Identification of personal data processing
 - 2.1.6. Stocktaking and recording in RoPA
 - 2.1.7. Risk assessment
 - 2.1.8. Discuss risk mitigation measures
 - 2.1.9. Technical discussion
 - 2.1.10. Interdisciplinary dialogue
 - 2.1.11. Data transfer assessment
 - 2.1.12. Document assessment
 - 2.1.13. App review
 - 2.1.14. Stakeholder consultation
 - 2.1.15. Stakeholder approval
 - 2.1.16. Regular monitoring and updates
 - 2.2. Support
 - 2.2.2. Charting tools
 - 2.2.3. Automated DPIA summarization
 - 2.2.4. Automated inventory of processing activities
 - 2.2.5. Automated process mining
 - 2.2.6. Data governance tools
 - 2.2.7. Metadata management tools

- 2.2.8. Automated security control tools
- 2.2.9. Questionnaire-based DPIA tools
- 2.2.10. None
- 2.3. Roles
 - 2.3.1. Documenter
 - 2.3.2. Advisor
 - 2.3.3. Organizer
 - 2.3.4. Researcher
 - 2.3.5. Assessee
 - 2.3.6. Liaison
- 2.4. Stakeholders
 - 2.4.1. Business functions
 - 2.4.1.1. Management board
 - 2.4.1.2. IT team
 - 2.4.1.3. Data processors
 - 2.4.1.4. Data controllers
 - 2.4.1.5. Marketing team
 - 2.4.1.6. Human Resources Department
 - 2.4.1.7. Project sponsors
 - 2.4.1.8. Works council (Germany)
 - 2.4.2. Privacy functions
 - 2.4.2.1. DPO
 - 2.4.2.2. Privacy team
 - 2.4.2.3. Legal counsel
 - 2.4.3. Data subjects
 - 2.4.4. Third parties
 - 2.4.4.1. Business partners
 - 2.4.4.2. Data protection supervisory authority
- 2.5. Vocabularies used

- 2.5.1. Organization-specific vocabulary
- 2.5.2. DPV
- 2.5.3. Law
- 2.5.4. Other standard vocabularies
- 2.6. Challenges
 - 2.6.1. Knowledge gap
 - 2.6.2. Communication gap
 - 2.6.3. Delay in privacy considerations
 - 2.6.4. Superficial legal recommendations
 - 2.6.5. Absence of source code analysis
 - 2.6.6. Lack of effort
 - 2.6.7. Lack of awareness
 - 2.6.8. Defining privacy roles
 - 2.6.9. Privacy-innovation trade-off
 - 2.6.10. Lack of standard rules

Appendix: Study Questionnaire for AutoPRICE Evaluation

In this appendix, we detail the AutoPRICE user study.

Section 1: Participant information

1.1. How much experience do you have with Android development? (in years)

- Free-text field.

- Mandatory.

1.2. Does your app have users in EU?

- Free-text field.

- Optional.

1.3. Rate your knowledge on the following on a scale of Beginner to Expert.

A. Android app development:

- Likert scale from 1 (Beginner) to 5 (Expert).

- Mandatory.

B. Java development:

- Likert scale from 1 (Beginner) to 5 (Expert).

- Mandatory.

C. Data privacy:

- Likert scale from 1 (Beginner) to 5 (Expert).

- Mandatory.

D. GDPR:

- Likert scale from 1 (Beginner) to 5 (Expert).

- Mandatory.

1.4. Have you completed the data safety section of an Android app? If yes, how?

- Free-text field.
- Optional.

Section 2: Analyze data collection using AutoPRICE

2.1. Which types of privacy-related data does this app collect? (Select all that apply)

- Multiple choice question.
- Mandatory.
- Choices:
 - * Risk 1 directly identifiable data
 - * Risk 2 partially identifiable data
 - * Risk 3 access data
 - * Risk 4 context-dependent data
 - * Personal data

2.2. Which of the collected data types require protection according to GDPR guidelines?

- Free-text field.
- Mandatory.

2.3. How does the data collected by this app compare with data collection practices of other apps in the same domain?

- Free-text field.
- Mandatory.

2.4. How would you rate this application based on how much data it collects? (Select one option)

- Multiple choice question.
- Mandatory.
- Choices:
 - * Safe – Collects very little data
 - * Moderate – Collects an appropriate amount of data
 - * Unsafe – Collects too much data

2.5. Do you feel confident about AutoPRICE's output? Why/why not?

- Free-text field.
- Mandatory.

Section 3: DSS

3.1. What differences do you observe between AutoPRICE's findings and the Google Play Store's Data Safety Section for this app?

- Free-text field.
- Mandatory.

3.2. Based on your findings, to what extent do you agree with AutoPRICE's DSS results?

- Free-text field.
- Mandatory.

3.3. Do you feel confident about AutoPRICE's output? Why/why not?

- Free-text field.
- Mandatory.

3.4. How was your experience with comparing AutoPRICE's DSS with Google Play Store's Data Safety Section (DSS) for this app? (Select one option)

- Multiple choice question.
- Optional text-field for comments.
- Mandatory.
- Choices:
 - * I could locate Play Store's DSS easily, and could compare it easily with AutoPRICE's DSS
 - * I could locate Play Store's DSS easily, but could not compare it easily with AutoPRICE's DSS
 - * I could not easily locate Play Store's DSS, but once found, I easily compare it with AutoPRICE's DSS
 - * I could not easily locate Play Store's DSS, and could not compare it easily with AutoPRICE's DSS

Section 4: Usability

4.1. I would like to use AutoPRICE for my privacy-aware development needs.

- Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).

- Mandatory.
- 4.2. I found AutoPRICE unnecessarily complex.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 4.3. I found AutoPRICE easy to use.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 4.4. I would need the support of a technical person to be able to use AutoPRICE.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 4.5. I found the various functions in AutoPRICE to be well integrated.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 4.6. I thought there was too much inconsistency in AutoPRICE.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 4.7. I would imagine that most people would learn to use AutoPRICE very quickly.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 4.8. I found AutoPRICE very cumbersome to use.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 4.9. I felt confident using AutoPRICE.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 4.10. I needed to learn a lot of things before I could get going with AutoPRICE.
 - Likert scale from 1 (Strongly disagree) to 5 (Strongly agree).
 - Mandatory.
- 4.11. How likely is it you would recommend AutoPRICE to an Android developer?

- Likert scale from 0 (Not at all) to 10 (Go for it!).
- Mandatory.

4.12. How likely is it you would recommend AutoPRICE to a user?

- Likert scale from 0 (Not at all) to 10 (Go for it!).
- Mandatory.

Section 5: General Feedback

5.1. What features would you like in AutoPRICE?

- Free-text field.
- Mandatory.

5.2. What are the positive points of AutoPRICE?

- Free-text field.
- Mandatory.

5.3. What are the negative points of AutoPRICE?

- Free-text field.
- Mandatory.

5.4. Was the given documentation useful?

- Free-text field.
- Mandatory.

5.5. Did AutoPRICE give you a better understanding of data collection?

- Free-text field.
- Mandatory.

5.6. Other remarks.

- Free-text field.
- Optional.

Appendix: Study Materials for Assessor View RoPA Generation

In this appendix, we present the details of the interview study conducted in Section 6.2.2. The anonymized answers are available online at <https://doi.org/10.5281/zenodo.16151588>.

E.1 Interview Briefing

Good morning/afternoon! Thank you for participating in our follow-up study. Today we will introduce you to Assessor View's new feature, RoPA generator. Before we start, I want to highlight that whatever you say to us during the study will be kept confidential, and you can stop participating at any time. If you feel uncomfortable answering any question, we can skip them with no penalty. Just let me know.

We will be recording the study. We will be sharing the link of the tool with you, that you can open and use in a new browser window. Would you be comfortable sharing your browser window? That will help us guide you through the process. If not, could you keep us updated about how you are using the tool? Before we start recording and begin the study, would you like to ask us any questions? I have shared the link to a consent form in the Zoom chat: <obtain consent from participant>

E.2 Interview Questionnaire

Section 1: State-of-the-art

- 1.1. How do you typically compile a Record of Processing Activities (RoPA) in your current work?
- 1.2. How often do you do this?
- 1.3. Do you use any templates? Which ones? What is your designation?

Section 2: Assessor View-generated RoPA

- 2.1. What were your first impressions when using the RoPA feature?
- 2.2. If they already complete RoPA in their work, ask: Do you think a partially pre-filled RoPA generated from source code would reduce manual workload during DPIA or pre-DPIA screening phases? Why/why not?

Else, ask: Imagine you need to fill out a RoPA for an initial version of an Android app. Do you think a partially pre-filled RoPA generated from source code would reduce manual workload during DPIA or pre-DPIA screening phases? Why/why not?
- 2.3. Do you think Assessor View-generated partial RoPA output would be a useful starting point for documenting processing activities in compliance with GDPR Article 30? Would it make the process easier for beginners or external DPOs?

Section 3: Usability, Feedback, and Suggestions

- 3.1. Could you understand and interpret the RoPA output generated by the tool?
- 3.2. Were there any parts of the output that you found unclear or that required additional validation?
- 3.3. If they already complete RoPA in their work, ask: Which representation (table or form) have you used more often for completing the RoPA? Which would you prefer to communicate with developers? Else, ask: Imagine you need to fill out a RoPA for an initial version of an Android app. Which representation (table or form) would you prefer to communicate with developers?
- 3.4. To what extent can a DPO rely on the RoPA output from Assessor View to meet legal obligations under Article 30?
- 3.5. What changes or additions would make the RoPA feature more useful for you?
- 3.6. What more would you like to see in Assessor View to better support RoPA generation?

E.3 Codebook

In this section, we present our codebook.

1. RoPA
 - 1.1. Support for completion
 - 1.1.1. Tool-guided

- 1.1.2. Documentation-based
 - 1.1.3. Interview-based
 - 1.1.4. Template-based
 - 1.1.5. Pre-DPIA Screening
- 1.2. Challenges
 - 1.2.1. Yearly review
 - 1.2.2. Lack of data granularity
 - 1.2.3. Balance abstraction layers
 - 1.2.4. Collect necessary information
 - 1.2.5. Time consuming
 - 1.2.6. Manual assessment
- 2. Assessor View-generated RoPA
 - 2.1. Advantages
 - 2.1.1. Purpose aggregation
 - 2.1.2. Starting point
 - 2.1.3. Beginner-friendly
 - 2.1.4. Enhanced understanding of data processing
 - 2.1.5. Reliable
 - 2.1.6. Facilitate stakeholder interviews
 - 2.1.7. Reduce manual workload
 - 2.1.8. Accessible to external DPOs
 - 2.1.9. Record of processing activities
 - 2.2. Table
 - 2.2.1. Overview of processed data
 - 2.2.2. Frequently used representation
 - 2.2.3. Inter-disciplinary communication
 - 2.3. Form
 - 2.3.1. Overview of processed data
 - 2.3.2. Frequently used representation